Robotics Research Lab
Department of Computer Science
University of Kaiserslautern-Landau

Master Thesis

Learning-Based Online Adaptation for Control of a Hydraulic Manipulator

Kevin Kudakwashe Murera

23 March 2026

Master Thesis

Learning-Based Online Adaptation for Control of a Hydraulic Manipulator

Robotics Research Lab
Department of Computer Science
University of Kaiserslautern-Landau

Kevin Kudakwashe Murera

Day of issue : 23 March 2026
Day of release : TODO: date of submission

Reviewer : Prof. Dr. Karsten Berns
Supervisor : TODO: supervisor

Hereby I declare that I have self-dependently composed the Master Thesis at hand. The sources and additives used have been marked in the text and are exhaustively given in the bibliography.

Tuesday 1st September, 2026 – Kaiserslautern

(Kevin Kudakwashe Murera)

Acknowledgements

"It always seems impossible until it's done." — Nelson Mandela

I would like to express my sincere gratitude to my supervisor, [**Supervisor's Name**], for their invaluable guidance, support, and encouragement throughout this Master's thesis. I am also deeply grateful to my family for their unwavering love and belief in me, and to my friends for their support and encouragement throughout this journey.

I sincerely thank everyone who contributed, directly or indirectly, to making this achievement possible.

Abstract

Hydraulic manipulators are difficult to control precisely: the actuator response is nonlinear, depends on load and supply pressure, and varies between machines. Faithful simulation is possible but far too slow for reinforcement learning, while commanding joint positions directly discards the behaviour that makes the machine difficult.

This thesis investigates how hydraulic actuator dynamics learned from high-fidelity simulation can be reduced, transferred into an efficient robotic simulation, and used for reinforcement-learning control of a verge mowing arm. Two actuator representations are identified from randomised campaigns: a modular chain of an inverse model, mapping desired cylinder motion to a valve command, and a forward model predicting the response; and a physics-structured reduced-order model keeping pressure, force and extension propagation explicit while learning covers valve realisation and loading. Both serve as the actuation interface for policy learning. A damped least-squares controller supplies privileged demonstrations and is then removed, so the distilled policy becomes the sole deployed controller.

The inverse and forward pair transfers to the deployment simulation without retraining in the one-step regime the policy uses, where extension error stays millimetre-scale across plant domains, and the reduced-order model transfers with a small bounded calibration against the deployment plant. Supervised accuracy proves an unreliable guide to control: a forward model scored highly while learning to ignore the valve command, and excitation design with input ablation are established as the tests that expose it. Both representations support closed-loop terrain following, and the choice of representation changes the control strategy the policy learns.

Online adaptation is examined last. Plant identity proves recoverable only from deliberate excitation, and under the randomised plant family the trained stacks absorb the variation unaided, so adaptation is tested on a plant whose load changes mid-episode. There a bounded correction on the inverse model's output changes nothing, and the reason is delivery rather than estimation: that command already rests against its limit for most of the scored window, so the authority to act is absent where the information arrives. Removing the inverse model, so that the policy emits the valve command and a bounded residual conditioned on causal history acts upon it, converts the same information into a substantial gain: cut-height error falls by roughly half and lateral error by more than two fifths against the frozen policy the stack is built from. The thesis contributes a decision procedure for whether online adaptation is worth building on a machine, and the finding that on a saturating actuator the placement of the correction, not the quality of the estimate, decides whether adaptation pays.

Contents

1 Introduction **1**

1.1 The Role of Mechanised Verge Maintenance 1

1.2 Advances in Learning-Based Control for Hydraulic Manipulators 1

1.3 Challenges of Precise Tracking with Hydraulic Mowing Arms 2

1.4 Contribution of This Thesis 3

1.5 Thesis Structure 6

2 Fundamentals and Related Work **9**

2.1 Manipulator Actuation and Control Fundamentals 9

2.1.1 Application Context and Implement Carriers 9

2.1.2 The Electro-Hydraulic Actuation Chain 10

2.1.3 High-Fidelity Simulation as a Data Source 10

2.2 Kinematics and Task-Space Tracking 11

2.2.1 Differential Kinematics 11

2.2.2 From Joint Space to Cylinder Space 11

2.3 Model-Based and Conventional Control Approaches 11

2.4 Data-Driven System Identification and Learned Dynamics 12

2.4.1 Learned Forward and Inverse Dynamics 12

2.4.2 Closed-Loop Identifiability 12

2.5 Reduced-Order Modeling 12

2.6 Domain Randomization and Latent Adaptation 13

2.6.1 Robustness Through Randomized Training 13

2.6.2 From Robustness to Adaptation 14

2.7 Reinforcement Learning and Structured Priors 14

2.7.1 Policy Optimization 14

2.7.2 Structured Priors for Policy Learning 15

2.7.3 Reinforcement Learning on Hydraulic Machines 15

2.8 Research Gap and Thesis Positioning 15

3 System Fundamentals and Problem Formulation 18
3.1 Manipulator Platform . . . 18
3.2 Kinematic Structure and Coordinate Mappings . . . 20
3.2.1 Joint Space and Cylinder Space . . . 20
3.2.2 Task Space and the Calibrated Cutter Jacobian . . . 21
3.3 Actuation and Measured State . . . 21
3.3.1 Sensing and Observability . . . 21
3.4 High-Fidelity Simscape Model . . . 23
3.4.1 The Command ROM as Deployment Actuator . . . 23
3.4.2 Mechanical Subsystem . . . 24
3.4.3 Hydraulic Subsystem . . . 24
3.4.4 Validated Scope of the Model . . . 25
3.5 IsaacLab Environment . . . 26
3.5.1 Simulation and Control Rates . . . 26
3.5.2 Environment Configuration Details . . . 26
3.6 The Verge-Following Task . . . 26
3.6.1 Task Definition . . . 26
3.6.2 Task Feasibility and Plant Capability . . . 27
3.7 Control Architecture . . . 27
3.8 Problem Formulation . . . 28
3.9 System-Level Evaluation Criteria . . . 29
3.9.1 The Eight-Gate Promotion Criterion . . . 29
3.9.2 Cylinder and Model Checks . . . 30

4 Data Generation and Preparation 32
4.1 Dataset Objectives and Design Requirements . . . 32
4.2 Randomized Parameter Space of the Simscape Campaign . . . 34
4.2.1 Sampling Design . . . 34
4.2.2 Hydraulic Parameters . . . 34
4.2.3 The Compressibility Axis . . . 35
4.2.4 Mechanical Parameters . . . 35
4.3 Closed-Loop Data Collection . . . 36
4.3.1 The Controller as a Data-Collection Instrument . . . 36
4.4 Open-Loop Excitation Experiments . . . 36
4.4.1 Limitations of the Closed-Loop Batch . . . 36
4.4.2 Design of the Excitation Signal . . . 37
4.4.3 Plant Draws and Acceptance . . . 38
4.5 Combining Closed-Loop and Open-Loop Datasets . . . 38
4.6 Closed-Loop Calibration Data for the Command ROM . . . 39
4.7 The IsaacLab Deployment Collections . . . 40
4.7.1 Provenance of the Deployment Data . . . 40
4.7.2 The Physical Factor Vector . . . 40
4.7.3 Domains and Factor Design . . . 40
4.7.4 Collection Procedure: The Task-Controller Runs . . . 41
4.7.5 The Open-Loop Excitation Collection . . . 43

4.7.6 The Frozen Dataset . . . 43
4.7.7 Learning Timebase and Hydraulic State . . . 43
4.8 Feature and Target Construction . . . 44
4.8.1 The Actuator Models . . . 44
4.8.2 Features and Targets for the Adaptation Module . . . 45
4.8.3 Features and Targets for the Prediction Models . . . 45
4.8.4 On-Policy Data for the Deployed Adaptation Residual . . . 45
4.9 Preprocessing and Data Splits . . . 46
4.10 Dataset Diagnostics . . . 47

5 Learned Models and Online Adaptation 49
5.1 Modelling Overview and Common Notation . . . 49
5.2 Inverse Dynamics Model . . . 51
5.2.1 Paired Control Fine-Tuning . . . 52
5.2.2 Transfer of the Frozen Inverse Model . . . 52
5.3 Forward Dynamics Model . . . 53
5.3.1 Assessment of the Supervised Score . . . 54
5.4 Reduced-Order Model . . . 55
5.4.1 Closed-Loop Calibration Against the Deployment Plant . . . 57
5.5 A Predictive Test of Plant Identity . . . 58
5.6 Arm Adaptation Module . . . 59
5.6.1 Information Content of the Estimated Latent . . . 60
5.7 Latent-Conditioned Inverse-Model Adapter . . . 60
5.7.1 Open-Loop Evidence . . . 61
5.7.2 Closed-Loop Evidence and Its Bound . . . 61
5.7.3 Propagation of the Latent Advantage . . . 62
5.8 Training and Model-Selection Protocol . . . 62

6 Control Framework and Simulation Integration 65
6.1 The DLS Teacher and Its Two Roles . . . 65
6.1.1 Scope and Role of the Controller . . . 65
6.1.2 Privileged Information . . . 65
6.1.3 From Task Error to Cylinder Command . . . 66
6.1.4 Execution Through the Student Interface . . . 67
6.1.5 Teacher Performance . . . 68
6.2 Policy Formulation . . . 68
6.2.1 Policy Definition . . . 68
6.2.2 The Observation Vector . . . 69
6.2.3 The Action and Its Bounds . . . 70
6.2.4 The Reward Function . . . 71
6.2.5 Optimization . . . 73
6.3 Training Protocol and the Distillation Finding . . . 74
6.3.1 Distillation by DAgger . . . 74
6.3.2 Objective Mismatch in Unconstrained Fine-Tuning . . . 76
6.4 Reference-Anchored PPO . . . 77

6.4.1 Critic Warm-Up . . . 77
6.4.2 The Reference Anchor . . . 77
6.4.3 Trust Regions and Rollback . . . 78
6.4.4 The Block Ratchet . . . 78
6.4.5 Theoretical Status . . . 78
6.5 R: Reference Actuator Baseline . . . 80
6.6 IF: Non-Adaptive Learned Baseline . . . 81
6.7 AF: The Adaptive Stack . . . 82
6.7.1 Why the Inverse Model Is Omitted . . . 83
6.7.2 Whether the Forward Model Alone Is Sufficient . . . 85
6.7.3 The Residual Controller . . . 86
6.7.4 Why the Base Policy Is Frozen . . . 87
6.7.5 How the Adaptation Encoder Is Trained . . . 87
6.7.6 Consistency With the Plant Dynamics . . . 87
6.7.7 What This Section Claims . . . 88
6.8 IsaacLab Integration and Timing . . . 88
6.9 Constraints and Fair-Comparison Rules . . . 89
6.9.1 Limits and Safeguards . . . 89
6.9.2 Matching Rules and What Each Comparison Carries . . . 89

7 Experimental Evaluation and Discussion 92
7.1 Experimental Protocol . . . 92
7.1.1 Test Conditions and Experimental Matrix . . . 92
7.1.2 Metrics, Statistics and Fairness . . . 93
7.1.3 What the Three Stacks Can and Cannot Be Compared For . . . 94
7.1.4 Checkpoint Selection and the Scoring Horizon . . . 95
7.2 Closed-Loop Task Performance . . . 95
7.3 Actuator Representation and Dynamic Response . . . 99
7.3.1 Dead Time in the Plant . . . 99
7.3.2 The Chain Compared Across the Three Stacks . . . 101
7.3.3 Step Response to a Commanded Change . . . 102
7.3.4 The Latency Budget of the Closed Loop . . . 104
7.3.5 Control Effort and Command Chatter . . . 104
7.4 Generalisation and Robustness . . . 105
7.4.1 How Much of the Requested Stress Reached the Plant . . . 107
7.4.2 What a Robustness Boundary Would Require . . . 108
7.5 Identifiability of Plant Variation . . . 108
7.5.1 Excitation Design and the Identifiability Envelope . . . 108
7.5.2 Closed-Loop Data Against Excitation Data . . . 110
7.5.3 Information Content of the Adaptive Latent . . . 110
7.5.4 The Model-Level Gain and the Frozen Inverse Model . . . 111
7.5.5 Estimated Against Privileged Latents . . . 113
7.6 Online Adaptation . . . 114
7.6.1 The Adaptation Experiment . . . 114
7.6.2 What Control and Adaptation Each Contribute . . . 115
7.6.3 Where the Advantage Comes From . . . 115
7.6.4 The Pressure Channel Across the Three Stacks . . . 117
7.6.5 Sensitivity to the Training Budget . . . 118
7.6.6 What This Experiment Does Not Yet Establish . . . 118
7.7 Integrated Discussion . . . 118

8 Conclusion and Future Work **122**
8.1 Summary of Findings . . . 122
8.2 Design Implications . . . 125
8.3 Contributions . . . 127
8.4 Limitations . . . 127
8.5 Future Work . . . 128

A System Parameters and Coordinate Mappings **130**
A.1 Rates . . . 130
A.2 Command-Path Limits . . . 131
A.3 Course and Episode Geometry . . . 131

B Dataset Manifests and Factor Ranges **132**
B.1 Trajectories . . . 132
B.2 Execution and Logging . . . 132
B.3 Parameter Ranges and Dataset Audit . . . 133
B.4 The Frozen Deployment Collections . . . 133

C Model Architectures, Normalizers and Checkpoint Hashes **135**
C.1 The Inverse and Forward Models . . . 135
C.2 The Adaptation Module . . . 136

D PPO and IsaacLab Configuration **137**
D.1 The Observation Vector . . . 137
D.2 The Action . . . 137
D.3 The Reward . . . 137
D.4 PPO Hyperparameters . . . 138

E Supplementary Results **140**
E.1 Per-Family Results on the Reference Mesh . . . 140
E.2 The Latent Substitution Arms . . . 141
E.3 Reproduction . . . 141

Bibliography **143**

1. Introduction

1.1 The Role of Mechanised Verge Maintenance

Road verges are cut because they are infrastructure. Vegetation left to grow closes sight lines at junctions and bends, obscures signs and barriers, blocks the drainage channels that keep water off the carriageway, and encroaches on the running surface itself. A national road network carries tens of thousands of kilometres of verge, and each kilometre is cut several times in a growing season, so the work is continuous, repetitive and performed almost entirely by machine.

It is also performed from a moving vehicle. Stopping to cut each section would require closing the lane, so a carrier drives the road at a few kilometres per hour while a boom-mounted mowing head works the verge alongside it. The head must hold a cut height of a few centimetres over ground that rises and falls continuously, follow a grass line that curves with the road, and do so without striking the surface hard enough to damage the tool or lift the machine.

The demands on the manipulator follow from that. The machine studied here, shown in Figure 1.1, carries a heavy rotating tool at the end of a long boom, works in intermittent contact with the ground, and must place that tool accurately while its own base is in motion. Hydraulic actuation is the standard answer at this scale: it supplies the force density and the tolerance to shock loading that electric drives do not, and it does so with components rugged enough for continuous outdoor operation. Precision, however, is not what hydraulics are good at, and that tension is the subject of this thesis.

1.2 Advances in Learning-Based Control for Hydraulic Manipulators

Two developments have changed what is achievable on machines of this kind.

The first is that policy optimisation in simulation has become practical for contact-rich manipulation. GPU-parallel physics engines run thousands of environments simultaneously, which makes reinforcement learning feasible on tasks that would take years of real-machine

experience, and randomising the simulator across a family of plants produces policies that transfer without being tuned to one machine. The second is that the actuator itself can be learned. Rather than deriving valve and cylinder behaviour from first principles and fitting its parameters, a network trained on recorded motion can represent the command-to-motion relationship directly, including the nonlinearities that analytical models approximate away.

Combining the two is attractive and not straightforward. A learned actuator model must be cheap enough to evaluate inside the training loop, which rules out the high-fidelity models used for offline analysis, and it must remain valid across the family of machines the policy will meet. This has motivated a line of work in which a compact code describing the current plant is inferred from recent interaction history and supplied to the controller, so that a single policy can serve a family rather than a single machine. Such methods report substantial gains in legged locomotion and in manipulation, and they are the direct antecedent of the architecture examined here.



Figure 1.1: MULAG FME 500 front mower on a Unimog during roadside verge maintenance. Source: [MULAG Fahrzeugwerk 26].

1.3 Challenges of Precise Tracking with Hydraulic Mowing Arms

A valve command does not produce a proportional cylinder motion. The relationship is nonlinear, depends on load and supply pressure, and is subject to dead time between the command and any measurable response. It also differs between machines: seal condition, fluid temperature, hose compliance and the attachment fitted all change it, so two machines of the same model respond differently and one machine responds differently across a season. A controller that inverts only the geometry therefore leaves the largest source of tracking error unaddressed.

That difficulty admits two responses, and they are routinely conflated. The first is robustness: train across a randomised family so that a single fixed model is adequate everywhere. It costs nothing at run time and gives up the difference between the family and the individual. The second is adaptation: infer which member of the family is present from the interaction history and condition the model or the policy on that estimate. It promises to recover that difference, at the cost of an estimator that must work from observable signals alone, since a deployed machine has no access to its own plant parameters.

The second response is only worth pursuing under conditions that are rarely checked before the work is done. Plant identity must be recoverable from the data the machine actually produces, which depends on how the machine is being driven while that data is collected. And the difference between knowing the plant and not knowing it must be large enough to matter once the estimate reaches a feedback controller. Neither condition is guaranteed, both are specific to the task, and both can be measured before an estimator is built. Establishing those measurements, and applying them to a hydraulic mowing arm, is the problem this thesis addresses.

1.4 Contribution of This Thesis

The difficulty described in Section 1.3 is a modelling problem before it is a control problem. A hydraulic actuator can be simulated accurately, but the models that do so are too slow to sit inside a reinforcement-learning loop that requires millions of environment steps. The alternative usually taken is to discard the hydraulics and command joint positions directly, which removes the load and pressure dependence that makes the machine difficult to control. What is needed is a representation that is cheap enough to train against and faithful enough that the resulting policy is controlling something close to the real actuator.

Learned actuator models and latent-conditioned adaptation are both established techniques, and Section 1.2 described the progress that makes them practical on machines of this class. What remains unclear is how such models should be identified so that they retain the command dependence a controller needs, whether they transfer between simulators without being retrained, and whether online adaptation adds anything once a policy has been trained against them. This thesis therefore treats representation and transfer as the primary questions, and treats online adaptation as the last of them rather than the first. The research statement of the thesis is as follows:

Comparison notation.

For readability, the three policy–actuator stacks carry both a short label and a full name, and the two are used interchangeably throughout this thesis: *R* is R, *IF* is IF, and *AF* is AF. The short labels are preferred where several stacks appear in one sentence and the full names where the actuator path itself is the subject. R uses the command-conditioned reduced-order model; IF uses the frozen inverse/forward pair; and AF retains the forward model alone, the policy emitting the valve command directly, with online adaptation supplied by a bounded history-conditioned residual. The labels name the complete stacks, not just their actuator models.

To determine how hydraulic actuator dynamics learned from high-fidelity simulation can be reduced, transferred into a computationally efficient robotic simulation and used for reinforcement-learning control, and under what conditions online adaptation improves on the robustness those learned models already provide.

The statement decomposes into six research questions, which are answered in the order given. The first three concern how the actuator is represented, the fourth concerns control through that representation, and the last two concern what remains for adaptation once the first four are settled. The six questions are:

1. How can data from a high-fidelity Simscape model be used to learn computationally tractable representations of hydraulic actuation while preserving the command dependence a controller requires?
2. To what extent do actuator models identified from Simscape transfer to an IsaacLab manipulator without deployment-specific retraining?
3. Can a physics-structured reduced-order hydraulic model supply dynamics that are accurate and stable enough for GPU-parallel policy training?
4. Can a policy trained from a privileged classical teacher become the sole deployed controller while acting through learned actuator dynamics?
5. How does the choice of actuator representation affect tracking, command behaviour, latency and robustness under plant variation?
6. Where plant variation survives that robustness, is it identifiable from recent observations, and does online adaptation by latent identification improve closed-loop performance?

The first is answered in Chapters 4 and 5, the second and third in Chapter 5, the fourth in Chapter 6, and the last two in Chapter 7.

RQ6 is the question named in the title, and it is deliberately placed last. It is only worth asking once the preceding questions have established that the learned models transfer, that a policy can control through them, and how much plant variation the resulting closed loop already absorbs unaided. An answer of any size to RQ6 is then a result about the system rather than a verdict on the method, because accurate models and a robust policy leave less for adaptation to recover. The answer this thesis reaches is that the size depends on where the adaptation is placed: on a command path that is already saturated it is indistinguishable from nothing, and on one with authority to spare it is substantial. Both halves are reported, and the second is intelligible only because the first was measured first. The work rests on the following contributions:

Transferable Hydraulic Actuator Learning: A pipeline that identifies hydraulic actuation from high-fidelity Simscape data and transfers it into IsaacLab through two

representations, a modular inverse and forward chain and a physics-structured reduced-order model. The inverse model is trained on Simscape data and evaluated on deployment runs drawn entirely outside that training distribution, where it reproduces most of the valve-command variance with an accuracy that is nearly flat across plant domains. It requires no deployment-specific retraining and remains frozen in every downstream stack. The reduced-order model is transferred differently and the difference is stated wherever it matters: it receives a small bounded calibration against the deployment plant, which is why it serves as a reference condition rather than as an independent competitor to the inverse and forward pair.

Control-Relevant Identification and Validation: A demonstration that predictive accuracy can conceal the loss of exactly the property a controller needs, together with the procedure that detects it. The first forward model reached a near-perfect supervised score while learning to ignore the valve command and reconstruct motion from the state history instead. The validation loss did not show this; ablating the command did. Redesigning the excitation and retraining recovered genuine command sensitivity, and the pairing of excitation design with input ablation is established here as a required test for learned hydraulic models intended for control rather than for prediction.

Reduced-Order Actuator Model: A command-conditioned grey-box actuator in which pressure, force, velocity and extension propagation remain explicit while learning covers valve realisation, unmodelled loading and bounded residual effects. It is accurate enough to serve as the deployment plant, stable in recursive rollout, and cheap enough to evaluate inside GPU-parallel policy training. Zeroing or shuffling its command collapses its extension and velocity accuracy, which establishes that the command authority is real rather than inferred from context.

Policy-Only Control Through Learned Hydraulics: A damped least-squares controller supplies privileged demonstrations during training and is then removed, so that the distilled policy emits desired cylinder motion directly into a learned actuator representation and constitutes the entire deployed controller. Both representations support closed-loop terrain following in this configuration, although two learned models placed in series could have compounded their errors.

A Decision Procedure for Online Adaptation: A matched evaluation protocol that scores the resulting stacks on tracking, control effort, latency, disturbance rejection and sensitivity to the magnitude of plant variation, together with a procedure that decides in advance whether online plant conditioning will repay the effort. The first of its two measurements establishes whether plant identity is present in the data the machine produces. The second supplies the true plant parameters through a privileged encoder and measures what the closed loop does with them. Both precede estimator training, and either can end the effort before it is incurred. This converts an open-ended engineering effort into a measurement, and it is the contribution of this thesis most likely to transfer to other machines and other latent-conditioned designs.

Online Adaptation on a Saturating Actuator: A demonstration that the binding constraint on latent-conditioned adaptation here is delivery rather than estimation, and a redesign that follows from it. With the inverse model in the command path, the estimate is informative and the closed-loop effect is absent, because the correction is added to a command already resting against its own limit for most of the scored window.

Removing that model, so that the policy emits the valve command itself and a bounded history-conditioned residual acts on it, converts the same information into a measured improvement: cut-height error falls from 7.964 mm to 3.772 mm and lateral error from 61.155 mm to 35.371 mm against the stack's own frozen base under a mid-episode plant change, paired across 528 environments on held-out meshes. The claim is about the placement of adaptation on saturating actuators, not about the superiority of one actuator representation over another.

The components on which these contributions rest, namely the causal plant estimator, the bounded adaptation path and the eight-gate evaluation harness, are described where they are used. They are technical enablers rather than separate claims.

1.5 Thesis Structure

This thesis investigates how hydraulic actuator dynamics learned from high-fidelity simulation can be reduced, transferred into a computationally efficient robotic simulation and used for reinforcement-learning control, grounding its investigation in the identifiability of the actuator models and in the robustness of the controllers built on them. The document is organised as follows:

Chapter 2 establishes the background against which the remainder is read. It introduces the machine class and its electro-hydraulic actuation, recalls the kinematic tools underlying the classical controller, and reviews how conventional model-based control handles hydraulic uncertainty. It then treats the learning methods this work builds on, namely data-driven dynamics models and the pitfalls of identifying them from closed-loop data, reduced-order and grey-box modelling, domain randomisation and latent adaptation, and policy optimisation with structured priors. The chapter closes by stating the gap that remains and the position of the three control stacks examined here.

Chapter 3 analyses the platform, using the MULAG FME 500 mounted on a Unimog carrier as the case study. It describes the rigid bodies and the serial chain they form, the coordinate mappings between task space and cylinder space, and the closed kinematic loops introduced by the cylinder mounts. It specifies what the controller can observe, separating measured quantities from quantities produced by a model, and presents the high-fidelity Simscape model from which the actuator data derives. The chapter also states the task, the plant capability limits and the acceptance criteria against which every later result is scored.

Chapter 4 presents the data generation strategy and answers the first research question. It describes the excitation design, the randomised factor set defining the family of plants, and the procedures that keep training, validation and reserved test configurations separate. It then reports the identifiability audit applied to the resulting collection, including the measured finding that excitation gathered under a task controller does not separate plants, and the open-loop collection that repairs this.

Chapter 5 develops the learned models and answers the first three research questions. It presents the inverse model that maps a desired cylinder motion to a valve command, the forward model that predicts the response, and the evidence that supervised accuracy alone does not establish that either model retains command authority. It then presents the command-conditioned reduced-order model that serves as the deployment plant inside

GPU-parallel training. The chapter closes with the material required for the final research question, namely the privileged latent and its measured limits, the causal estimator that infers a deployable substitute from observable history, and the bounded adapter through which that estimate first reached the actuator path. That adapter produces no closed-loop effect, and the chapter ends by establishing why: the limit is the delivery of the correction rather than the quality of the estimate, which is the finding the following chapter acts on.

Chapter 6 describes the control framework and its integration with the simulator, and answers the fourth research question. It presents the damped least-squares teacher used for demonstration and data collection, the shared observation interface and the action interface the two non-adaptive stacks hold in common, the distillation of the teacher into a student policy, and the reference-anchored fine-tuning that makes further policy optimisation admissible without discarding the distilled behaviour. It then assembles the three deployed control stacks, including the removal of the inverse model from the adaptive command path that the preceding chapter motivates, and states which comparison each stack can carry.

Chapter 7 reports the experimental evaluation and answers the last two research questions. It establishes the excitation envelope within which plant identity is recoverable, the sufficiency of the frozen inverse model, and what the inferred latent encodes. It measures latency through the actuation chain and the response of each stack to a commanded change, then compares the three deployed controllers on closed-loop tracking and control effort. It closes by measuring the disturbance envelope of each controller, an oracle reference for what plant-identity conditioning contributes within this architecture, and the sensitivity of each stack to the magnitude of plant variation.

Chapter 8 concludes the thesis. It summarises what the work establishes and what it does not, states the design implications that follow from the measured results, records the limitations of the evidence, and sets out the future work, which centres on transferring the learned actuator models and the distilled policy to the physical machine and fine-tuning them on data collected from it.

Appendix A provides the system parameters and the coordinate mappings between task space, joint space and cylinder space referred to in Chapter 3. **Appendix B** lists the dataset manifests and the factor ranges that define the family of plants used in Chapters 4 and 7. **Appendix C** records the model architectures, the normalisers and the checkpoint hashes for every learned model reported in Chapter 5. **Appendix D** documents the PPO and IsaacLab configuration used for the training described in Chapter 6. **Appendix E** collects supplementary results supporting Chapter 7.

2. Fundamentals and Related Work

Chapter 1 argued that precise task-space tracking with a hydraulic mowing arm is limited less by kinematics than by the actuator. The path from a valve command to actual cylinder motion is nonlinear and load dependent, and it differs from one machine, season, or attachment to the next. This chapter collects the background needed to judge that argument and to understand the design decisions made later in the thesis. Section 2.1 introduces the machine class and its electro-hydraulic actuation. Section 2.2 recalls the kinematic tools behind the classical controller, and Section 2.3 reviews how conventional model-based control deals with hydraulic uncertainty. The remaining sections cover the learning methods this work builds on. Section 2.4 treats data-driven dynamics models and the pitfalls of identifying them from closed-loop data. Section 2.5 covers reduced-order and grey-box modeling, Section 2.6 domain randomization and latent adaptation, and Section 2.7 policy optimization with structured priors. Section 2.8 then states the gap that remains and how the three control stacks of this thesis (R, IF, and AF) address it. As defined in Chapter 1, these abbreviate PPO–ROM, PPO–ID–FD, and PPO–AAM–FD, respectively. Only material that is needed for the later chapters is included. The concrete system definition and problem formulation follow in Chapter 3.

2.1 Manipulator Actuation and Control Fundamentals

2.1.1 Application Context and Implement Carriers

The MULAG FME 500 mounts a multi-link hydraulic boom and a 1.2 m flail head on a Unimog carrier [MULAG Fahrzeugwerk 26]. It reaches about 6.5 m laterally, below road level and over obstacles, while an operator adjusts cutter height, tilt and lateral position as the vehicle moves. Automating this continuous tracking task is the practical motivation of the thesis.

Hydraulic cylinders suit the required force, shock tolerance and existing vehicle power supply, but their command gain changes strongly with load and configuration. This trade-off between force density and precision is central to hydraulic manipulator control [Mattila 17].

2.1.2 The Electro-Hydraulic Actuation Chain

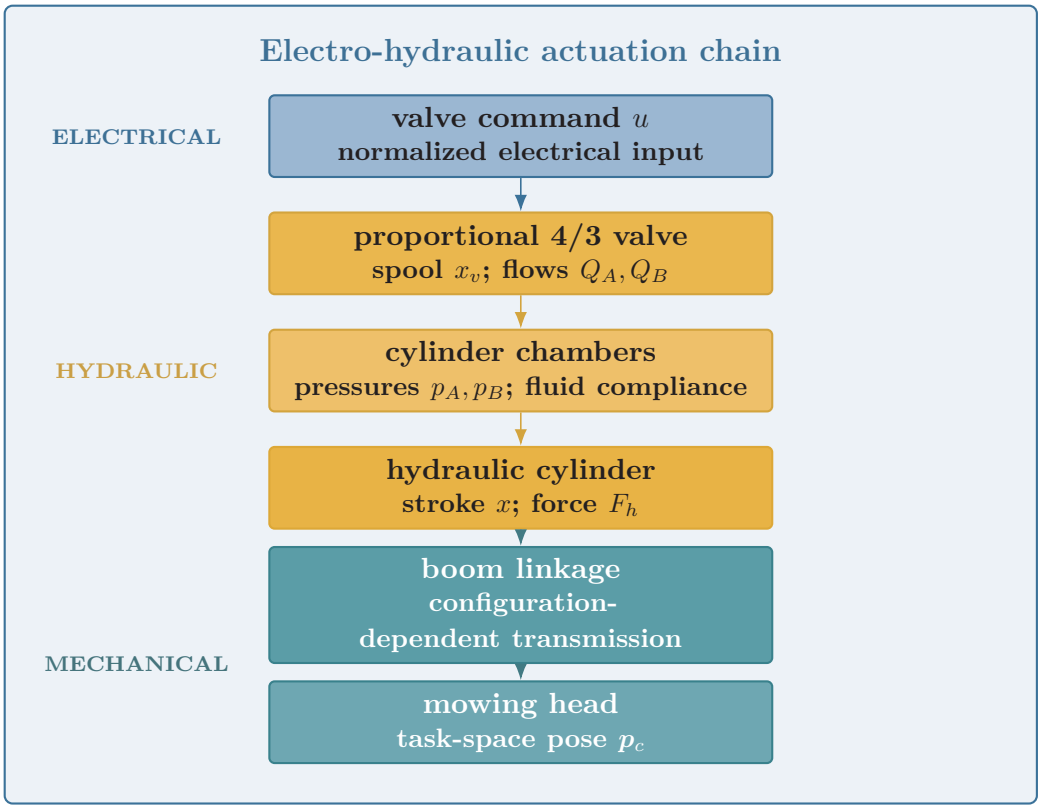


Figure 2.1: The electro-hydraulic actuation chain from valve command to mowing-head motion. The block colors distinguish the electrical, hydraulic and mechanical stages; each subtitle identifies the principal quantity passed to the next stage.

In Figure 2.1, an electrical command moves a proportional valve spool and meters flow to the cylinder chambers. The orifice law [Merritt 67] makes flow depend on spool position and the square root of the pressure margin. Valve authority is therefore load-dependent and directional; overlap creates a dead zone near null and the spool saturates at large commands [Jelali 03].

Compressibility, changing chamber volume, relief limits, leakage and seal friction further separate flow from motion. Several of these effects are non-smooth, while bulk modulus, leakage and friction drift with temperature and wear. The mechanical linkage then converts cylinder translation through configuration-dependent lever arms. Chapter 5 consequently models the hydraulic realization in cylinder space, before kinematic effects are mixed into cutter error.

2.1.3 High-Fidelity Simulation as a Data Source

The component-level Simscape plant [The MathWorks, Inc. 24a] resolves the stiff hydraulic dynamics but runs one machine at small integration steps. Policy optimization instead benefits from thousands of GPU-parallel plants [Makoviychuk 21]. The detailed model is therefore used offline to generate identification data; compact actuator models carry its behavior into parallel training, as summarized in Figure 2.2.

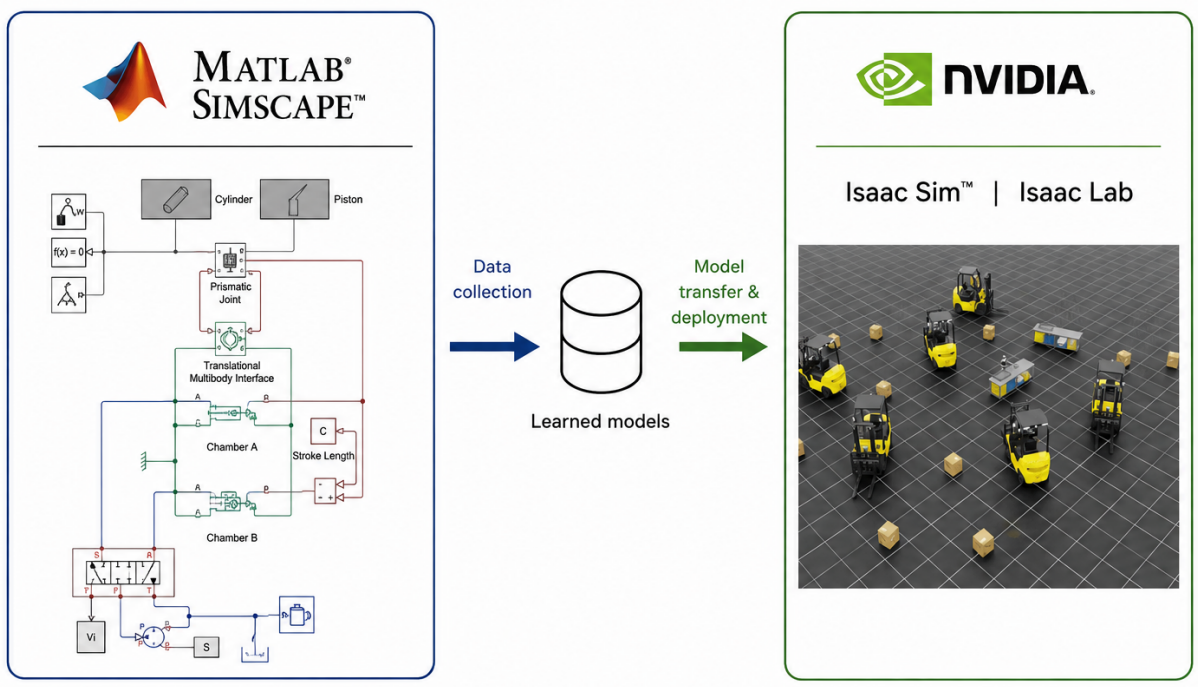


Figure 2.2: Simscape supplies high-fidelity hydraulic data; compact actuator models transfer that behavior to GPU-parallel training in IsaacLab. Product images: MathWorks and NVIDIA.

2.2 Kinematics and Task-Space Tracking

2.2.1 Differential Kinematics

The cutter path is specified in vehicle coordinates, so the manipulator Jacobian maps joint rates to the required tool velocity [Siciliano 09]. Near singular configurations the ordinary pseudo-inverse produces excessive commands. The damped least-squares inverse of Wampler [Wampler 86] and Nakamura and Hanafusa [Nakamura 86] trades some local accuracy for bounded commands and is retained as the classical controller.

2.2.2 From Joint Space to Cylinder Space

The actuated coordinates are cylinder lengths, and their nonlinear cylinder-to-joint map has a configuration-dependent lever arm. A passive head joint further limits independently controllable directions. The calibrated cutter Jacobian in Chapter 3 composes these mappings. It still assumes that commanded cylinder motion is realized; the learned actuator models address that remaining gap.

2.3 Model-Based and Conventional Control Approaches

Industrial systems commonly map joystick input directly to valve current. Fixed-gain position–velocity or position–pressure cascades improve on this practice [Jelali 03], but tuning across large changes in valve gain and linkage leverage produces either weak response or oscillation.

Nonlinear model-based control instead cancels known rigid-body dynamics [Slotine 91], estimates uncertain parameters online [Åström 95], or bounds their effect. Yang et al. [Yang 26], for example, combine state constraints with explicit valve dead-band inversion. Such methods can track accurately, but depend on valve, friction and inertia models that are costly to maintain on a commercial implement [Mattila 17].

This thesis therefore retains reliable structure and learns the uncertain part. The DLS kinematic controller acts as teacher and data-collection controller; learning covers the mapping from cylinder command to realized motion and the policy that issues that command (Section 2.7).

2.4 Data-Driven System Identification and Learned Dynamics

2.4.1 Learned Forward and Inverse Dynamics

Robot model learning distinguishes forward models, which predict the next state from state and command, from inverse models, which predict the command for a desired transition [Nguyen-Tuong 11]. Section 5.1 fixes both interfaces. One-step fit is insufficient when a forward model is recursively rolled out, because small errors compound [Nagabandi 18]. The models here are therefore trained with multi-step objectives. Probabilistic ensembles represent growing rollout uncertainty [Chua 18]; recurrent architectures instead retain unobserved pressure and valve history [Hochreiter 97, Cho 14], as used by the reduced-order model.

2.4.2 Closed-Loop Identifiability

Informative identification requires inputs that excite the plant independently of feedback [Ljung 99, Isermann 11]. In closed-loop data, command and state history are correlated; a flexible predictor can score well by ignoring the command, and validation from the same distribution will not reveal the shortcut [Schoukens 19].

That failure occurred here: a closed-loop forward model reached one-step $R^2 \approx 0.999$ yet showed almost no command dependence under ablation. Chapter 4 therefore uses open-loop valve excitation, and every learned model is checked with zeroed and shuffled commands. Prediction score alone is not treated as evidence of control authority.

2.5 Reduced-Order Modeling

Reduced-order models retain dominant behavior at lower cost [Benner 15]. Here the full Simscape plant is reduced to per-cylinder extension, velocity and pressure difference. The construction is grey-box: known structure is embedded so that data fits what remains unknown [Karniadakis 21]. As in the structured dynamics network of Lutter et al. [Lutter 19], explicit physical layers improve interpretability. The hydraulic ROM retains the orifice, pressure and integration structure; learned terms represent valve realization, configuration-dependent load and bounded residuals.

Hwangbo et al. [Hwangbo 19] combined analytic rigid-body mechanics with a learned actuator response, a pattern later scaled in GPU-parallel simulation [Rudin 22]. Yamamori

et al. [Yamamori 26] instead shaped the physical actuator to match the simulation. A commercial valve and pump cannot be redesigned in that way, so this thesis learns the hydraulic realization while PhysX retains the rigid-body mechanics (Figure 2.3).

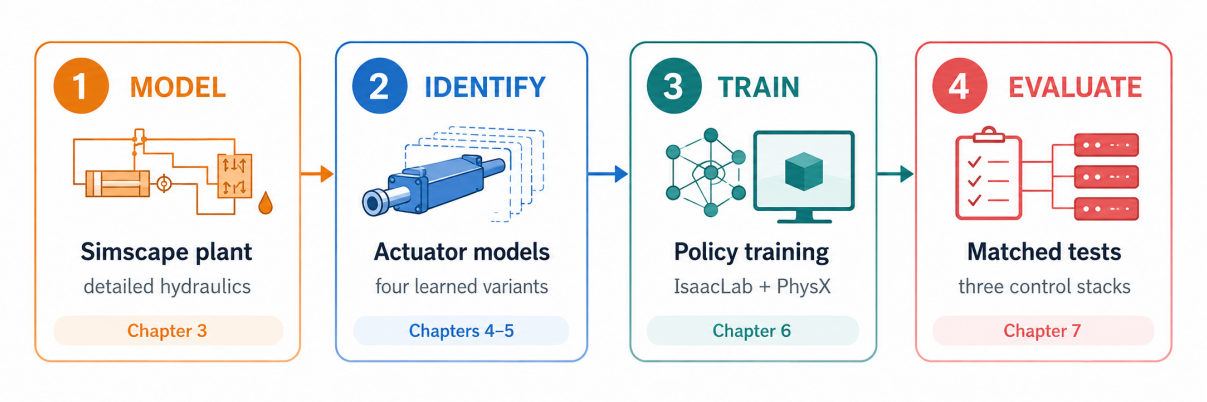


Figure 2.3: Workflow from the hydraulic plant model to matched evaluation. The separation between rigid-body physics and learned actuation follows the actuator-network approach in [Hwangbo 19].

2.6 Domain Randomization and Latent Adaptation

2.6.1 Robustness Through Randomized Training

A model trained on one simulated plant inherits its parameters as assumptions. Domain randomization instead trains across a family, first for visual appearance [Tobin 17] and then for dynamics such as mass, friction, gain and latency [Peng 18]. The hydraulic family in Chapter 4 varies bulk modulus, valve gain, leakage, friction and payload, producing robustness to physically plausible plant variation.

2.6.2 From Robustness to Adaptation

Figure 2.4 contrasts a policy trained for one robust response across the plant family with an adaptive stack that conditions its response on the current plant.

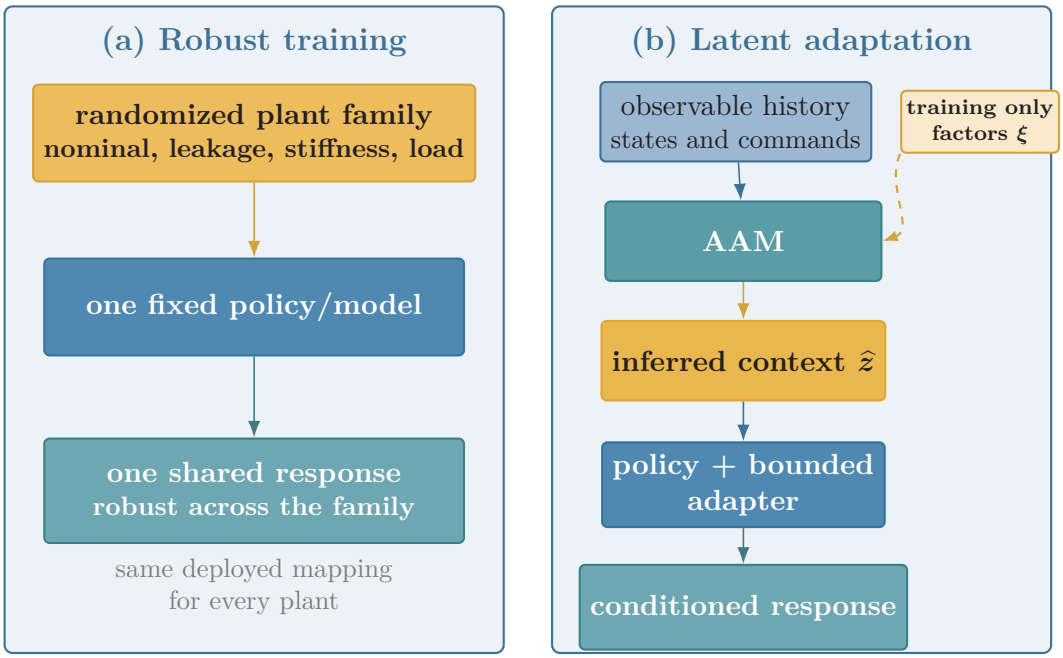


Figure 2.4: Comparison of robust training and latent adaptation. Robust training applies one fixed response across the randomized plant family. The adaptive stack uses privileged factors only to supervise training; during deployment, the AAM infers context from observable history and conditions the downstream response.

Teacher–student methods encode known simulation factors into a privileged latent and train an estimator to recover it from observable history. Lee et al. [Lee 20] used this construction for hidden terrain properties; Kumar et al. [Kumar 21] developed rapid motor adaptation. Unlike classical online parameter estimation [Åström 95], the learned latent need not have a direct physical interpretation. It can also condition a dynamics model rather than a policy, as in the contextual neural ODE of Yu et al. [Yu 26].

The adaptation module in Chapter 5 encodes one second of causal observable history. In AF, that encoding conditions a bounded residual on the valve command the policy itself emits, while the base policy remains frozen; it does not condition the ROM, the forward model or the PhysX plant. The experiments keep two distinctions explicit: randomized training is robust but not automatically adaptive, and an informative privileged latent does not prove that an inferred one improves closed-loop control. The second distinction is not rhetorical here, since the privileged-context route was tried on this stack and returned a null while the history-conditioned route did not.

2.7 Reinforcement Learning and Structured Priors

2.7.1 Policy Optimization

Reinforcement learning optimizes a policy from observations to actions for expected cumulative reward [Sutton 18]. This thesis uses PPO [Schulman 17] with generalized

advantage estimation [Schulman 16]; Section 6.2.5 gives the exact objective. GPU-parallel simulation supplies the required throughput [Makoviychuk 21, Rudin 22], which is why compact actuator surrogates replace the full hydraulic plant during training.

2.7.2 Structured Priors for Policy Learning

Learning from scratch would discard reliable kinematics. Residual RL instead adds a bounded learned correction to a controller at runtime [Johannink 19, Silver 18]. Policy distillation moves the prior to training: a privileged DLS teacher supplies demonstrations, then the student acts alone [Ross 11]. This thesis takes the second route. The teacher is distilled and PPO updates are anchored to the frozen student (Section 6.4), and the deployed actor remains the sole controller: no analytic controller runs underneath it at deployment, so the runtime correction of residual RL is not the construction used here.

One clarification belongs here, because the vocabulary collides. The adaptive stack of Chapter 6 does add a bounded term to its own frozen actor, but that term is not a residual in the sense of this subsection. There is no classical controller beneath it to correct, and its purpose is not to supply a prior that accelerates learning. It is an online adaptation mechanism internal to the reinforcement-learning policy, whose only function is to carry the dependence on inferred plant context that a frozen actor cannot express. It therefore belongs to the latent-adaptation line of Section 2.6.2 and is compared against that literature, not against residual policy learning.

The idea parallels real-world fine-tuning that retains a simulation-trained value function [Yin 25], but here the retained object is the reference policy. Anchoring was introduced because unconstrained PPO improved reward while degrading the physical tracking criterion (Section 6.3).

2.7.3 Reinforcement Learning on Hydraulic Machines

Deep RL has reached full-size hydraulic excavators in simulation-to-machine tracking [Egli 20] and general automation [Egli 22]. Spinelli et al. [Spinelli 24] extended it to commercial material handlers with underactuated tools. Maitan et al. [Maitan 26] learned force control on a physical cylinder, using a contraction-metric filter to bound exploration. These studies establish scale and feasibility, but do not combine a randomized plant family, online actuator adaptation, a privileged kinematic teacher and a matched comparison of actuator representations.

A policy can also compensate for a poor actuator model. Chapter 7 therefore reports rollout error, command magnitude and smoothness, and cylinder-level residuals alongside task tracking. Equal task error alone is not evidence of equal model quality.

2.8 Research Gap and Thesis Positioning

The literature leaves a specific combination unresolved. Hydraulic control provides structure but requires maintained plant models; learned dynamics replace derivation with data but depend on identifiable excitation; actuator surrogates and latent adaptation are established mainly for electric legged robots; and hydraulic RL studies do not isolate actuator representation under a matched protocol. Table 2.1 summarizes this position.

The five criteria are: (I) learned hydraulic actuation, (II) data audited for causal input identifiability, (III) online latent adaptation, (IV) a structured controller retained as a bounded prior, and (V) a matched actuator comparison. Parenthesized marks denote partial or implicit coverage.

Table 2.1: Positioning against the reviewed lines of work.

Line of work	I	II	III	IV	V
Model-based hydraulic control [Mattila 17, Jelali 03]	—	(✓)	(✓)	✓	—
Actuator networks for legged robots [Hwangbo 19, Rudin 22, Yamamori 26]	—	✓	—	—	—
RL for hydraulic machines [Egli 20, Egli 22, Spinelli 24, Maitan 26]	✓	(✓)	—	—	—
Latent online adaptation [Lee 20, Kumar 21, Yu 26, Liu 25, Hu 26]	—	—	✓	—	—
Residual RL and policy distillation [Johannink 19, Silver 18, Ross 11, Yin 25]	—	—	—	✓	—
This thesis (R / IF / AF)	✓	✓	✓	✓	✓

Column III is claimed in full, and the qualification it once carried is worth stating because it was earned rather than assumed. Causal recovery of the latent was established first; the closed-loop benefit was not, and on the inverse-model path it could not be, for the saturation reason developed in Section 6.7.1. It is claimed here because the adaptive residual is measured against its own frozen base under a paired protocol and separates from it. The contribution remains the complete experimental chain—identifiable cylinder data, physics-structured models fast enough for parallel training, online adaptation and a common evaluation protocol—rather than an unqualified claim that adaptation always improves control; the conditions under which it does not are reported alongside those under which it does. The three tested stacks are:

R. A command-conditioned ROM realizes the policy's desired cylinder motion. It is the reference and the only actuator calibrated to the deployment plant (Section 6.9).

IF. The same policy interface drives the frozen inverse model followed by the strict forward model, forming the robust learned baseline.

AF. The policy emits the valve command directly and the inverse model leaves the path. A causal history encoder conditions a bounded residual on that command, which the same forward model realizes ahead of PhysX.

All stacks share the task, terrain, trajectories, deployment factors and acceptance gates. R and IF additionally share the action contract and form the matched pair; AF does not, because its policy chooses a valve command rather than a cylinder increment, and it is therefore compared with its own frozen base rather than ranked against the others. The ROM is calibrated to the deployment plant, so causal architectural claims rely on the matched pair and on within-stack paired ablations rather than on a three-way contrast (Section 6.9.2). Chapter 3 now grounds this comparison in the concrete machine.

3. System Fundamentals and Problem Formulation

Chapter 2 established the control and modeling concepts used in this thesis. This chapter converts that background into a fixed system definition for the remaining work. It specifies the physical mechanism, coordinate mappings, available measurements, actuator models, task geometry, and evaluation interfaces before the controllers are compared. Keeping these definitions together prevents later chapters from changing the plant, observation set, or success criteria when interpreting results.

The physical machine and its two simulation representations are distinguished explicitly. Simscape is the high-fidelity hydraulic source used for actuator identification. IsaacLab/PhysX is the parallel environment used for policy training, adaptation-data collection, and closed-loop evaluation. Both use the cylinder ordering, kinematics, action contract, and cutter-tip task variables defined here, but they are not assumed to be identical plants.

Sections 3.1–3.3 define the mechanism, coordinate spaces, and signals. Sections 3.4 and 3.5 describe the two simulation models. The remaining sections define the verge-following task, the R, IF, and AF control stacks, the control objective, and the acceptance gates used in the experiments.

3.1 Manipulator Platform

The MULAG FME 500 is modeled as a serial-link mechanism with five revolute joints. Four hydraulic cylinders actuate the slew, two boom, and head-pivot joints. The *Messerkopf* rotation has no independent actuator and is locked at the mowing configuration; it is therefore not a controlled degree of freedom. The cylinder order used throughout the thesis is slew (*Drehzapfen*), boom I (*Ausleger I*), boom II (*Ausleger II*), and head pivot (*Messerkopf-Schwenk*).

Table 3.1 gives the cylinder stroke ranges used in the training data and the fixed limit of the *Messerkopf* joint. Task-space errors are measured at the cutter tip, which lies 0.79 m below the *Messerkopf* body origin.

Table 3.1: Cylinder stroke ranges in the training data (Chapter 4) and the fixed *Messerkopf* joint limit used for the mowing task.

i	Joint	Function	Limit
1	*Drehzapfen*	slew	0.018–0.493 m
2	*Ausleger I*	boom I	0.275–0.791 m
3	*Ausleger II*	boom II	0.309–0.857 m
4	*Messerkopf-Schwenk*	head pivot	0.269–0.689 m
5	*Messerkopf*	head rotation (locked)	$q_{5,\min} = q_{5,\max} = q_{5,\text{lock}}$

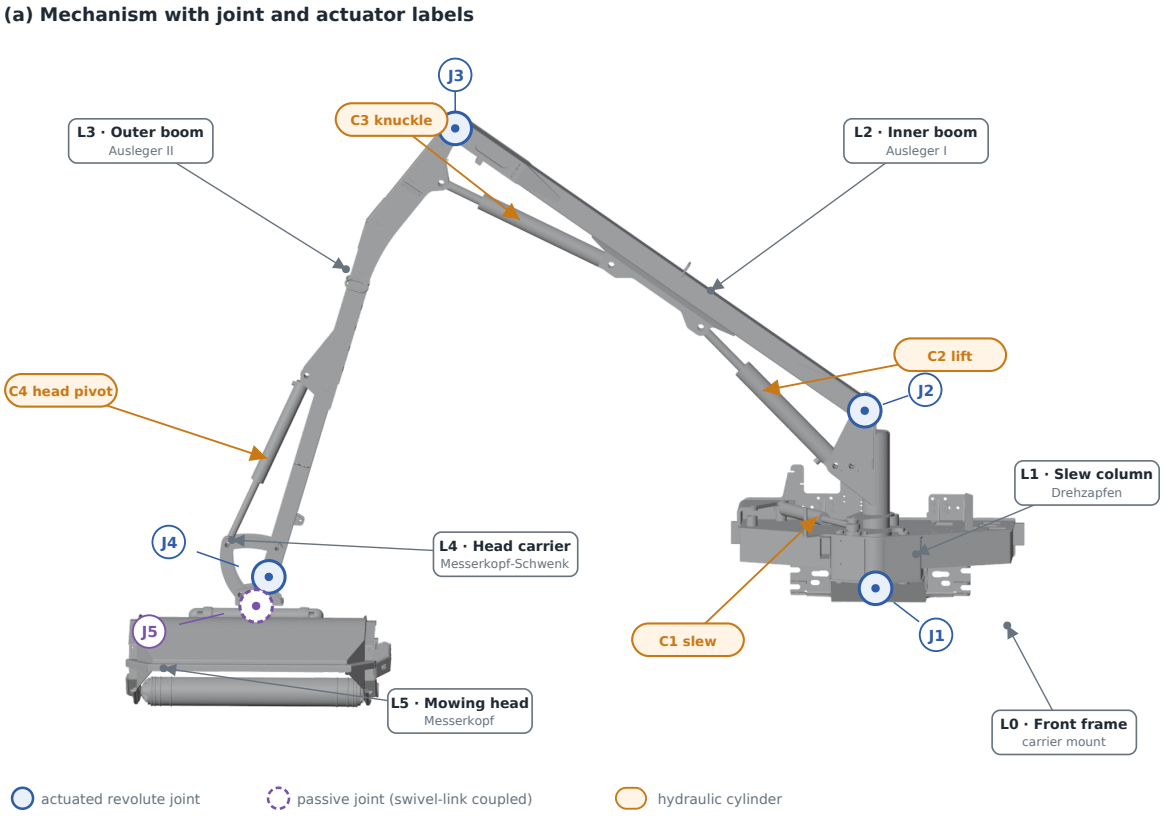


Figure 3.1: Annotated CAD view of the MULAG FME 500. Links L0–L5, joints J1–J5, and the four hydraulic cylinders C1–C4 are identified; J5 denotes the *Messerkopf* rotation, which is locked in the mowing configuration used in this thesis.

Figure 3.1 identifies the links, joints, and four actuator channels used throughout the model.

Figure 3.2 separates the rigid bodies that form the assembled mechanism. The catalog makes the L0–L5 body assignment explicit and shows the representative barrel–rod geometry used for cylinders C1–C4.

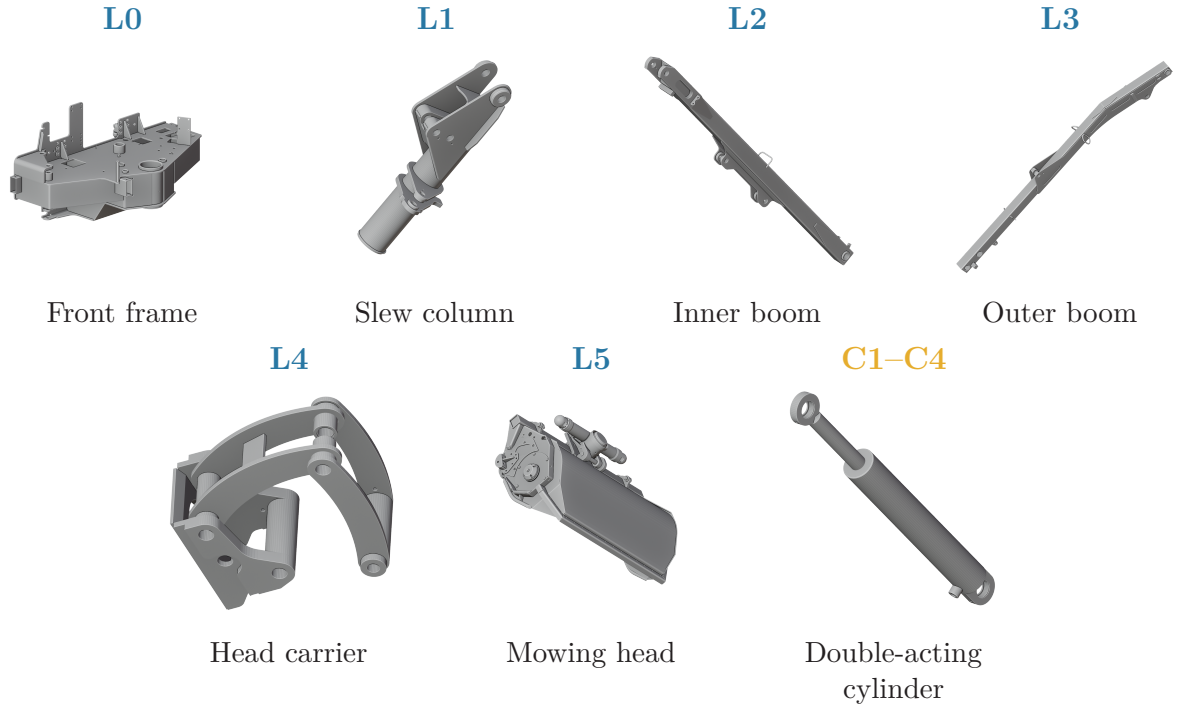


Figure 3.2: Individual CAD-derived bodies of the MULAG FME 500 shown at a common orthographic scale. L0–L5 are the front frame, slew column, inner boom, outer boom, head carrier, and mowing head. The C1–C4 panel shows the representative double-acting cylinder barrel–rod pair.

Manufacturer CAD supplies the link geometry, joint locations, masses, centers of mass, and inertia tensors through Phobos [von Szadkowski 20]. The same CAD-derived bodies and joint frames form the mechanical basis of the Simscape and IsaacLab models, keeping dimensions, axes, masses, and cylinder lever arms consistent while the hydraulic implementations remain model-specific.

3.2 Kinematic Structure and Coordinate Mappings

3.2.1 Joint Space and Cylinder Space

The configuration is $\boldsymbol{q} \in \mathbb{R}^5$; the actuated coordinates are the four cylinder extensions $\boldsymbol{x} \in \mathbb{R}^4$. Cylinder i joins CAD attachment points $\boldsymbol{p}_A$ and $\boldsymbol{p}_B$ on adjacent links. Its length and extension are

$$\ell_i(\boldsymbol{q}) = \left\| \boldsymbol{p}_B(\boldsymbol{q}) - \boldsymbol{p}_A(\boldsymbol{q}) \right\|, \qquad x_i = \ell_i(\boldsymbol{q}) - \ell_{0,i}, \tag{3.1}$$

with $\ell_{0,i}$ the retracted reference length. Differentiating (3.1) gives the scalar mapping between joint motion and extension rate,

$$\dot{x}_i = J_{\mathrm{cyl},i}(\boldsymbol{q})\, \dot{\boldsymbol{q}}, \qquad J_{\mathrm{cyl},i} = \boldsymbol{e}_i^{\top} \left(\boldsymbol{J}_B - \boldsymbol{J}_A \right), \tag{3.2}$$

where $\boldsymbol{e}_i$ is the cylinder-axis unit vector and $\boldsymbol{J}_A$ and $\boldsymbol{J}_B$ are attachment-point Jacobians. Virtual work gives $\boldsymbol{\tau}_{\text{act}} = \boldsymbol{J}_{\text{cyl}}^{\top}\boldsymbol{F}_h$. The mapping varies with configuration and stroke.

3.2.2 Task Space and the Calibrated Cutter Jacobian

The cutter tip is expressed in the truck chassis frame (`base_link`); using the moving slew-base frame would corrupt the tracking error. The controlled point includes the $-0.79\,\text{m}$ tool-tip offset. Vehicle motion supplies the along-track direction.

The calibrated Jacobian $\boldsymbol{J}_c \in \mathbb{R}^{3\times4}$ maps cylinder extension rates to cutter-tip velocity. It was measured at the head-down mowing pose by stepping each cylinder target and recording the settled cutter displacement. Its singular values are 8.24, 5.69, and 2.79, giving rank 3. The damped least-squares controller in Chapter 6 uses this same Jacobian.

3.3 Actuation and Measured State

Table 3.2 lists the signals and their availability.

Table 3.2: Signal inventory of the modeled system. "S" marks signals present in the Simscape data exports, "I" signals available live in the IsaacLab environment, "M" quantities that exist there only through the learned models of Chapter 5, and "P" *privileged* quantities that exist in simulation but must never enter a deployable policy or estimator input.

Signal	Dimension	Availability	Meaning
$\boldsymbol{u}$	$[-1,1]^4$	S, M	normalized proportional-valve commands
$\boldsymbol{x}$, $\dot{\boldsymbol{x}}$	$\mathbb{R}^4$	S, I	cylinder extensions and velocities
$\Delta\boldsymbol{p}$	$\mathbb{R}^4$	S, M	chamber pressure differences
$\boldsymbol{F}$	$\mathbb{R}^4$	S	logged rod forces
$\boldsymbol{q}$, $\dot{\boldsymbol{q}}$	$\mathbb{R}^5$	S, I	joint positions and velocities
$\boldsymbol{p}_c$	$\mathbb{R}^3$	S, I	cutter-tip position, truck frame
$\boldsymbol{h}$	$\mathbb{R}^9$	I	forward terrain-height profile ahead of the cutter
$\boldsymbol{c} = [\Delta\boldsymbol{x}^*, \Delta\dot{\boldsymbol{x}}^*]$	$\mathbb{R}^8$	I	desired cylinder command
$\mathbf{e}$	$\mathbb{R}^6$	P	physical randomization factors of the active plant
$\mathbf{z}_{\text{SD}} = \mu(\mathbf{e})$	$\mathbb{R}^{d_z}$	P	privileged dynamics latent, $d_z = 2$
$\mathbf{z}_{\text{AAM}}$	$\mathbb{R}^{d_z}$	M	AAM estimate of $\mathbf{z}_{\text{SD}}$ from observable history

In IsaacLab, valve commands and differential pressures are learned-actuator states, not direct measurements. R produces them internally; IF obtains them from the inverse/forward pair, and AF from the frozen forward model alone. $\mathbf{e}$ and $\mathbf{z}_{\text{SD}} = \mu(\mathbf{e})$ are privileged; only $\mathbf{z}_{\text{AAM}}$ may enter AF.

3.3.1 Sensing and Observability

The deployable observation contains articulation state, terrain rays, contact signals, the desired cylinder command, and actuator-model pressure state. Plant parameters are not observed; AF infers their effect from causal history.

Figure 3.3 separates deployable measurements from privileged plant information. Articulation state is read from the physics engine without encoder noise or quantisation.

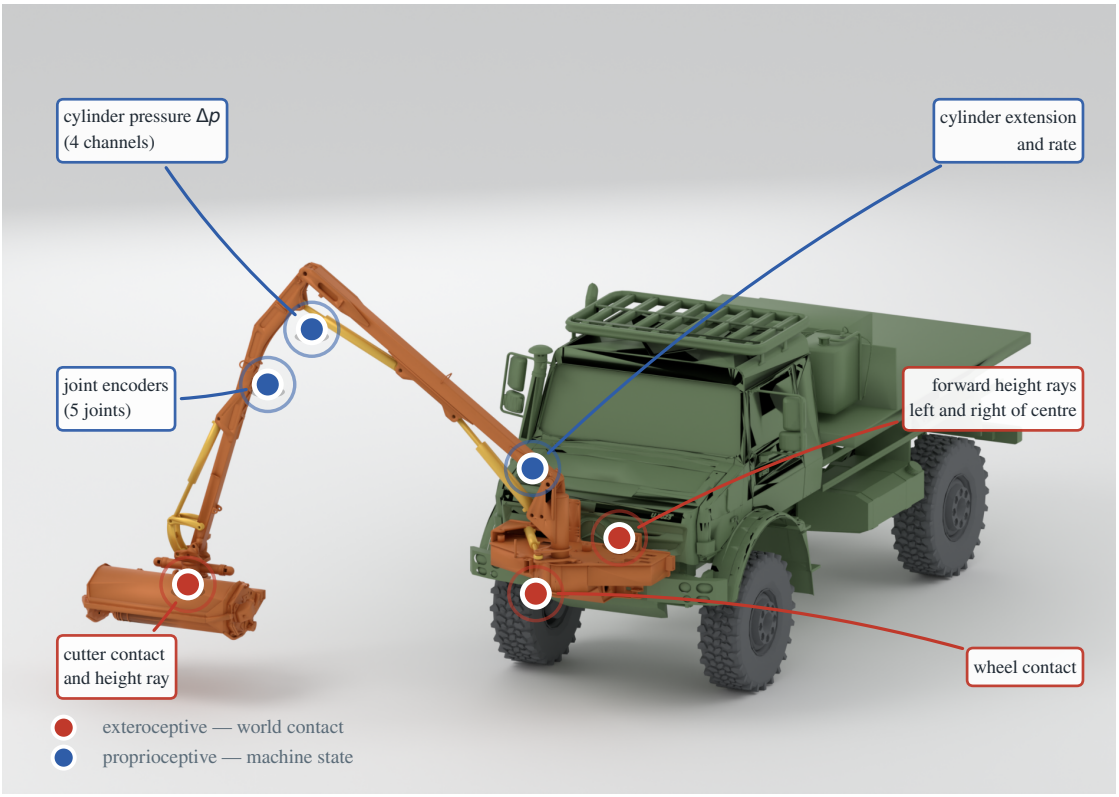


Figure 3.3: Sensor and observability map for the deployed controller. Blue callouts identify proprioceptive inputs: joint encoders, cylinder pressure differences, and cylinder extension and rate. Red callouts identify exteroceptive inputs: forward terrain-height rays and cutter and wheel contact.

3.4 High-Fidelity Simscape Model

The actuator models use a multi-domain Simscape model of the FME 500 developed in the preceding master project. Table 3.3 distinguishes the data source of each frozen component.

Table 3.3: Provenance of each frozen actuator component. The command ROM is the only one that has been calibrated against the deployment plant; the inverse/forward pair is Simscape-identified throughout. This asymmetry is a property of the comparison and is carried into the discussion of Chapter 7, not glossed over.

Component	Identified on	Calibrated on
Command ROM	Simscape closed-loop export (grey-box, full model)	IsaacLab/PhysX closed-loop rollouts, via a bounded command-gated correction head only
Control ID	Simscape closed-loop export	—
Strict FD	Simscape closed-loop export, retrained on the strict open-loop excitation export	—
SD_0, $\mu(\mathbf{e})$, SD_z, AAM	IsaacLab/PhysX adaptation collection (from scratch)	—

Simscape supplies the frozen ID and strict FD checkpoints. The ROM receives an additional bounded IsaacLab calibration (Section 5.4.1). The adaptation models are trained from IsaacLab data collected through the ROM and PhysX.

3.4.1 The Command ROM as Deployment Actuator

The command ROM plus PhysX is the deployment plant. Simscape is too slow for policy training and bare PhysX has no hydraulics. The ROM is therefore both the plant model and the actuator of R:

- R uses the deployment plant itself and is the reference condition.
- IF replaces it with an independently identified inverse/forward pair.
- AF drops the inverse model, has the policy emit the valve command directly, and adds online adaptation on the forward model.

The R–IF gap measures the transfer cost of the independently identified pair; it is not a general ranking of the two model classes.

3.4.2 Mechanical Subsystem

The mechanism uses the CAD-derived links, joints, and cylinder attachment frames of Section 3.1. It is simulated in Simscape Multibody [The MathWorks, Inc. 24b]. The joint-space dynamics take the standard manipulator form

$$\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q},\dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) + \boldsymbol{\tau}_{\text{fric}}(\dot{\boldsymbol{q}}) = \boldsymbol{\tau}_{\text{act}} - \boldsymbol{J}^{\top}(\boldsymbol{q})\,\boldsymbol{F}_{\text{ext}}, \tag{3.3}$$

with inertia $\boldsymbol{M}$ from the imported CAD. Friction combines viscous, Coulomb, and Stribeck terms. The cutting load scales with feed velocity and vegetation density. Each cylinder uses a spherical–prismatic–cylindrical loop and hard stops enforce the limits in Table 3.1.

3.4.3 Hydraulic Subsystem

Figure 3.4 shows four 4/3 valve-driven asymmetric cylinders fed by a variable-speed, fixed-displacement pump ($Q_p = D_p\omega$), accumulator, and pressure-relief valve.

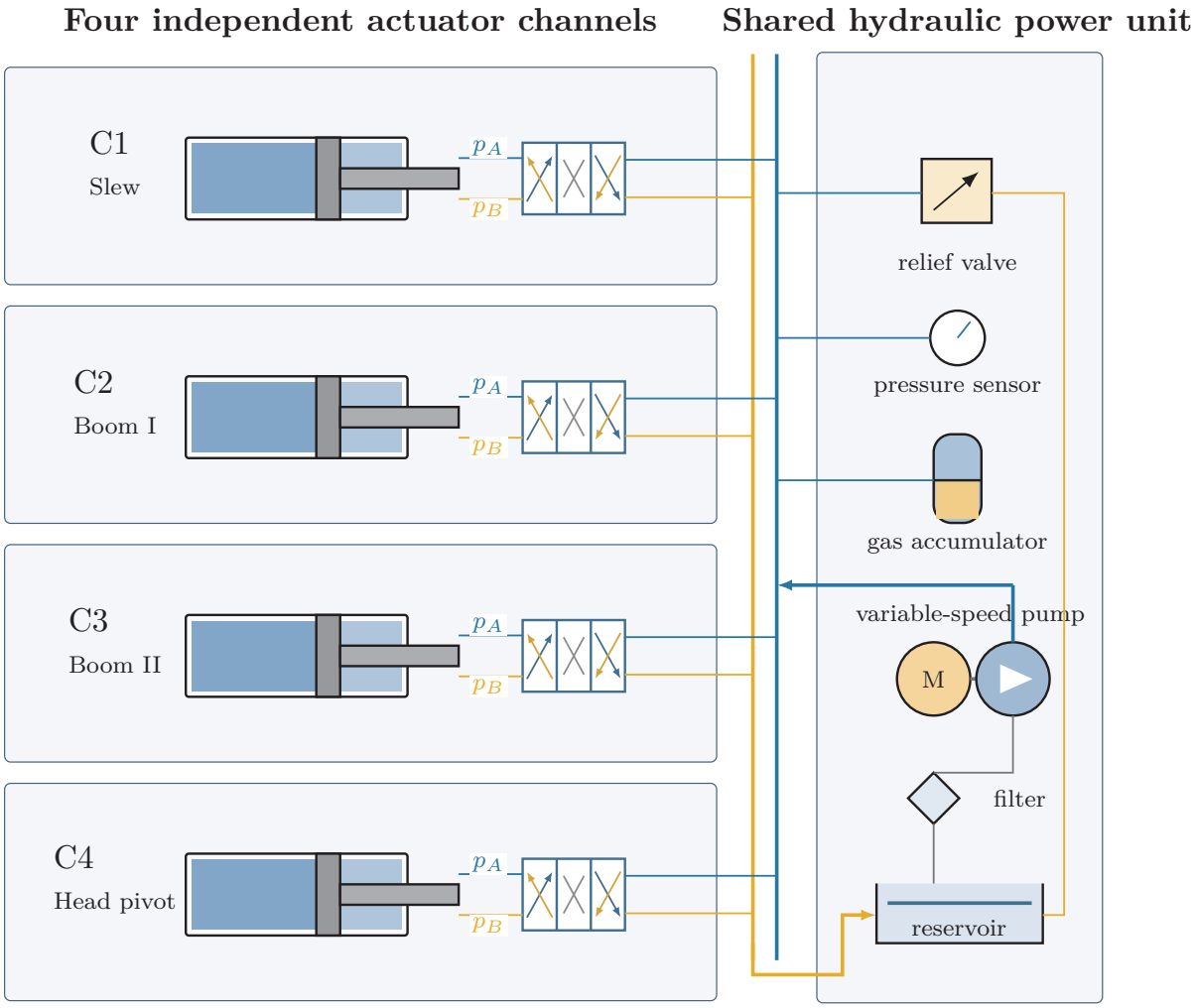


Figure 3.4: Hydraulic system schematic. Four proportional-valve cylinder channels share a variable-speed pump, reservoir, accumulator, sensor, and pressure-relief valve. Blue and orange rails denote supply and return.

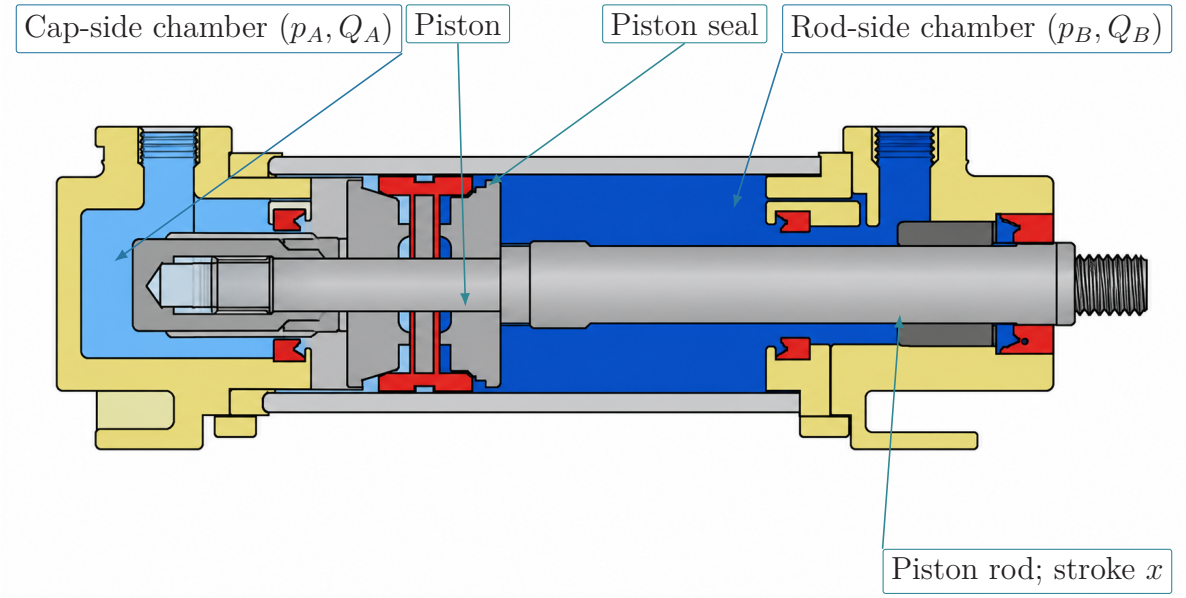


Figure 3.5: Annotated cutaway of an asymmetric double-acting hydraulic cylinder. The cap-side and rod-side chamber pressures generate the piston force and rod stroke.

Figure 3.5 identifies the cylinder chambers, piston, seal, and rod used in the actuator model.

The 4/3 valve in Figure 3.4 meters flow to the cylinder chambers. The chamber pressures follow

$$\dot{p}_1 = \frac{E_{\text{eff}}}{V_1(x)}\big(Q_1 - A_1\dot{x} - C_{\text{leak}}(p_1 - p_2)\big), \qquad \dot{p}_2 = \frac{E_{\text{eff}}}{V_2(x)}\big(-Q_2 + A_2\dot{x} + C_{\text{leak}}(p_1 - p_2)\big), \tag{3.4}$$

where $V_{1,2}(x)$ include dead volumes, $A_1 \neq A_2$, and C_{leak} is internal leakage. Entrained air is represented by

$$E_{\text{eff}}(p) = \left(\frac{1}{E_f} + \frac{\alpha}{p}\right)^{-1}, \tag{3.5}$$

Valve flow follows the orifice law of Section 2.1.2; spool dynamics are $\dot{x}_v = (u - x_v)/\tau_v$. The piston force is $F_h = p_1 A_1 - p_2 A_2 - F_{\text{fric}}(\dot{x})$. The coupled model is stiff and uses a variable-step implicit solver, which prevents RL-time simulation.

3.4.4 Validated Scope of the Model

Validation uses a fixed integral-backstepping controller at 0.01 s while sweeping $\beta \in \{0.5, 1.0, 1.5, 2.0\}$ GPa. Slew-axis delay falls from about 0.85 s to 0.65 s, the system delay index from 0.45 s to 0.35 s, and task-space tracking error by about 11 %. The improvement saturates beyond 1.5 GPa.

The model represents one machine and omits thermal dynamics. The logged rod-force channel is not carried into the learned actuator models: force is generated internally by the PhysX plant at deployment, and the logged channel also fails the piston-area consistency audit (Section 5.4). Chapter 4 introduces variation through parameter randomization.

3.5 IsaacLab Environment

Policy training, adaptation-data collection, and evaluation run in IsaacLab [Mittal 23] on PhysX. The USD articulation contains the Unimog carrier, five arm links, and four prismatic cylinder joints. Learned actuator models supply the hydraulic behavior between the controller and PhysX cylinder targets.

3.5.1 Simulation and Control Rates

Table 3.4 lists the four simulation clocks. PhysX runs at 120 Hz and the policy at 60 Hz. The learned actuator models and AAM update every three policy steps (0.05 s); their prediction is held across the intervening PhysX updates. Data collection uses this same 50 ms grid.

Table 3.4: The four rates of the simulated system. They are reported separately throughout the thesis; "the control frequency" is not a well-defined quantity here.

Layer	Step	Rate
PhysX physics	1/120 s	120 Hz
Policy / control (decimation 2)	1/60 s	60 Hz
Learned actuator model (3 control steps)	0.05 s	20 Hz
AAM latent update	0.05 s	20 Hz

3.5.2 Environment Configuration Details

The arm root is spawned at $z = 1.26$ m; ground-height spawning buries the vehicle. Cylinder targets are integrated in reference space so gravity does not erase commanded increments. Vehicle motion is scripted because the authored tyres and suspension do not provide reliable PhysX traction; a planar fixture holds lateral position and heading.

3.6 The Verge-Following Task

3.6.1 Task Definition

The task is moving verge following. The truck drives a 26 m strip at a speed sampled from 2 to 7 km/h; development uses 4 km/h. Episodes allow 2808 steps (46.8 s), while scoring stops when the strip is completed. The arm must keep the cutter at the target height, on the verge centre line, in contact with the ground, and within its travel limits. Terrain height is randomized per environment (Figure 3.6).

Eleven reference-path families span straight, periodic, spline, sweep, and lane-change motion. Results use the worst path family and terrain mesh rather than a pooled score. A regression test verifies that all eleven families are distinct.



Figure 3.6: IsaacLab realization of the moving verge-following task. The scripted truck travels on the level road while the four-cylinder arm places the cutter above the procedurally generated verge and follows the white reference path. Randomized grass, bushes, soil patches, twigs, and logs form a non-colliding visual layer; the underlying verge mesh supplies the terrain disturbance and height-sensing geometry used by the controller.

3.6.2 Task Feasibility and Plant Capability

The action contract limits cylinders 1–3 to 4 mm per policy step, which bounds lateral cutter speed at

$$v_{\text{lat}}^{\max} = \frac{4\,\text{mm}}{1/60\,\text{s}} = 0.240\,\text{m/s}. \tag{3.6}$$

This uses the 1/60 s policy interval. Early path settings exceeded this limit; the worst case (`boom_sweep`) required 3.7× the available rate. Path and terrain parameters are now derived from (3.6), and a preflight check rejects infeasible tasks.

Equation (3.6) is a property of the cylinder-increment contract, so it describes the action chains of R and IF directly. AF bounds its command in valve space instead, and the same arithmetic does not apply to it. The paths and terrain derived from the ceiling are nonetheless shared by all three stacks, so what the task *demands* is identical in every case; what differs is only the space in which the request is bounded.

3.7 Control Architecture

The vehicle advances at the scripted speed and the reference generator supplies the verge centre line and cut height. The policy maps normalized observations to four bounded actions and has no runtime baseline controller. In R and IF those actions are desired cylinder increments and the action contract bounds them as such; R uses the command ROM and IF the frozen inverse/forward pair to realise them. AF instead emits the valve

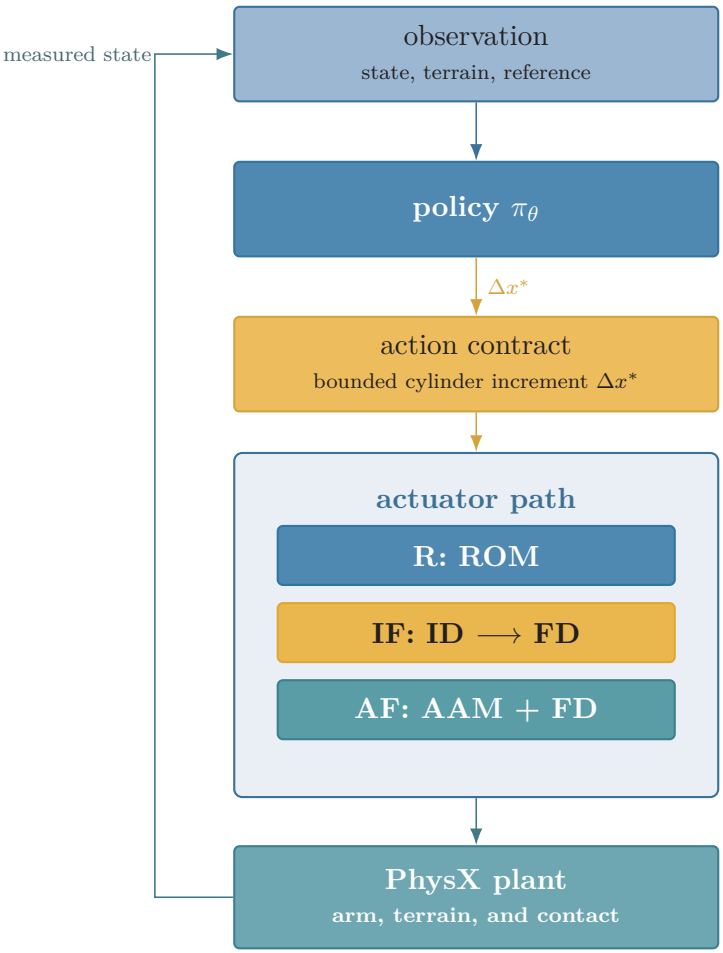


Figure 3.7: Deployed control stack. The policy produces a bounded cylinder increment; R, IF, or AF realizes it in the shared PhysX plant. Measured state closes the loop.

command directly and realises it through the frozen forward model, adding a bounded residual conditioned on causal history. All paths set the same PhysX cylinder target. The DLS controller is used only for teaching and identification.

3.8 Problem Formulation

The reference is the verge centre line $y_{\mathrm{ref}}(t)$ and cut height h_{ref}, expressed in the truck frame. With cutter-tip position $\boldsymbol{p}_c(t)$ and terrain height $g(\cdot)$, the scored errors are

$$e_{\mathrm{lat}}(t) = p_{c,y}(t) - y_{\mathrm{ref}}(t), \qquad e_{\mathrm{cut}}(t) = \Big(p_{c,z}(t) - g(\boldsymbol{p}_c(t))\Big) - h_{\mathrm{ref}}. \tag{3.7}$$

The objective is to minimize the mean and tail of both errors across path families and terrain while maintaining contact, head alignment, and smooth motion. Learned models are evaluated on one-step and recursive cylinder predictions. The stacks are tested across the randomized plant family. R and IF are compared with each other on held-out factors, and AF is compared with its own frozen base under a plant change, for the reasons given in Section 6.9.2.

Constraints are cylinder limits and headroom (Table 3.1), the action contract, checkpoint output limits, and the head-down condition.

3.9 System-Level Evaluation Criteria

Controllers and models are evaluated separately. Controller scores can hide actuator-model errors through policy compensation.

The system-level criteria cover three aspects of operation. Cut-height and lateral errors measure task accuracy. Contact, head alignment, impact rate, and cutter acceleration describe tool–terrain interaction and motion quality. Cylinder headroom and safety terminations check whether the task is completed without driving the mechanism into its limits. The metrics are evaluated over the scored part of each episode and are kept in physical units where possible. This makes a failed gate traceable to a specific limitation rather than to a single combined reward value.

3.9.1 The Eight-Gate Promotion Criterion

Table 3.5 defines the eight physical gates used throughout this thesis. Their role is to supply a criterion that is *not* the reward. A policy is optimised against the scalar return of Section 6.2.5, but it is selected and promoted against this vector of physical quantities, and Section 6.3.2 shows that the two point in measurably different directions. The gates are therefore an instrument for checkpoint selection and for the block-ratchet promotion of Section 6.4, and that is how every result in this thesis uses them. They are used to rank and to promote candidates, not to certify a stack as accepted, and no claim of certified acceptance is made anywhere in this work.

All controller stacks are evaluated with the same path, terrain, and randomization conditions. A candidate is preferred when it satisfies more of the gates at a wider margin; strong mean tracking does not compensate for loss of contact, inadequate cylinder headroom, excessive impacts, or safety terminations. Worst-case aggregation prevents easy terrain or path cases from masking a local failure. Mean absolute errors quantify sustained tracking performance, while the lower and upper percentiles capture short alignment, clearance, or acceleration events that a mean can hide. Contact and termination measures are reported as fractions so that episodes of different completed length remain comparable.

Table 3.5: Eight system-level acceptance gates. Each gate uses at least three terrain meshes and the worst mesh; tracking-critical gates also use the worst path family. †Hard impacts is a wear constraint.

Gate	Metric	Threshold	Aggregation
Cut height	mean $\lvert e_{\mathrm{cut}} \rvert$	$\leq 0.040\,\mathrm{m}$	worst mesh
Lateral tracking	mean $\lvert e_{\mathrm{lat}} \rvert$	$\leq 0.300\,\mathrm{m}$	worst mesh, worst family
Ground contact	contact fraction	≥ 0.700	worst mesh, worst family
Head alignment	p01 head-down alignment	≥ 0.900	worst mesh
Hard impacts†	hard-impact step fraction	≤ 0.350	worst mesh
Tool acceleration	p95 cutter acceleration	$\leq 120\,\mathrm{m/s^2}$	worst mesh
Cylinder headroom	p01 travel margin	$\geq 0.005\,\mathrm{m}$	worst mesh
Safety terminations	termination rate	≤ 0.080	worst mesh

3.9.2 Cylinder and Model Checks

Cylinder checks use fresh $\pm 10\,\mathrm{mm}$ commands, coordinated $80\,\mathrm{mm}$ motions, return error, and saturation. Model checks use one-step and rollout accuracy, input ablation, and Jacobian conditioning (Chapter 5). Chapter 7 reports these with the acceptance contract. The small command checks local response and deadband; the coordinated motion checks coupling between actuator channels. Return error tests reversibility, and saturation checks whether the learned model respects the command and stroke limits. These diagnostics are interpreted alongside the system gates, not as substitutes for closed-loop task performance.

The system is now specified. The next chapter turns to the data the learned models are built from.

4. Data Generation and Preparation

Chapter 3 defined two plant representations: detailed hydraulics in Simscape and fast rigid-body dynamics in IsaacLab. This chapter documents the datasets that connect them, including collection, auditing, feature construction and splitting. Table 3.3 records their provenance.

Simscape campaigns. Closed-loop runs provide realistic operating data for the inverse and reduced-order models. Separate open-loop valve excitation provides the causal command information needed by the strict forward model (Section 4.4).

ROM calibration. After Simscape training, the command ROM was fine-tuned on five closed-loop IsaacLab reference shapes. Only its bounded correction head remained trainable, with Simscape replay retained (Section 4.6). The ID–FD pair was not calibrated in IsaacLab.

IsaacLab deployment data. The system-dynamics surrogate, the privileged encoder and the adaptation module use two 118-run collections recorded in the deployed command ROM–PhysX plant, including dedicated PRBS windows (Section 4.7). No earlier adaptation or system-dynamics model was reused. Figure 4.1 summarizes the common preparation workflow.

4.1 Dataset Objectives and Design Requirements

Four requirements governed collection and preparation.

R1: realistic operating distribution. The learned models are deployed inside a tracking controller, so they must be accurate in the states that controller visits: coordinated multi-axis motion, gravity-loaded poses, moderate speeds. Data collected far from that distribution would be accurate where it does not matter.

R2: causal identifiability of the valve. A forward model is only useful to a policy if it responds to the command. This requires that the valve signal in the training data carry information that the state history does not already carry, which is a property of the *experiment*, not of the network [Ljung 99, Isermann 11].

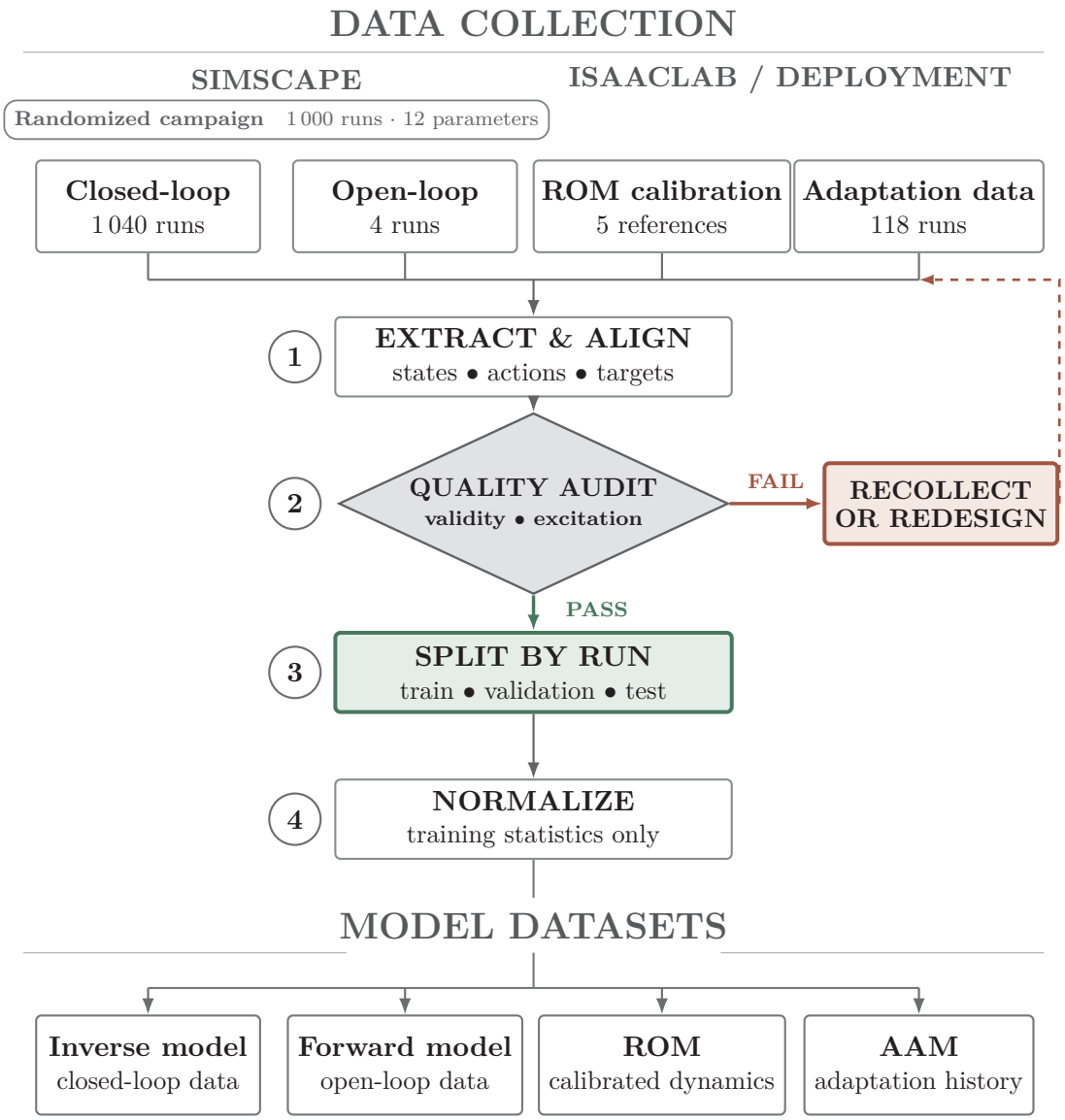


Figure 4.1: Data-preparation workflow. Two Simscape collections and two IsaacLab collections pass through common extraction, audit, run-level splitting and training-statistics normalization stages. The randomized Simscape campaign is provenance metadata rather than a pipeline stage. A failed audit returns the experiment to collection; accepted material forms the inverse, forward, ROM and AAM datasets.

R3: variation across a plant family. Robust training and, later, latent adaptation both need many plants rather than one. The variation has to be physically meaningful, it has to demonstrably change the measured response, and it has to be recorded per run, because the privileged latent of Section 5.5 is computed from those recorded factors.

R4: clean separation. Train, validation, and test material must be separable at the level of complete runs, so that no transition window straddles a split boundary and no run contributes to two splits.

R1 and R2 require different experiments. Closed-loop control gives realistic motion but correlates command with state; open-loop excitation breaks that correlation but is less representative of deployment. The two campaigns are therefore retained and combined as described in Section 4.5.

4.2 Randomized Parameter Space of the Simscape Campaign

This section records the Simscape randomization design. No reported model is trained on this curriculum; it is retained as collection provenance and because its compliance analysis (Section 4.2.3) informed the IsaacLab factor design. Readers concerned only with trained models may continue at Section 4.3.

4.2.1 Sampling Design

The closed-loop campaign draws its plants from a twelve-dimensional parameter space using Latin hypercube sampling [McKay 79], which spreads the draws more evenly over each marginal than independent uniform sampling would at this sample size. The twelve coordinates are fixed once for the whole batch, before any simulation runs, so the parameter table is reproducible and can be audited independently of the data it produced.

The batch is organised as a curriculum rather than a single homogeneous pool. Run 1 is the untouched nominal plant. Runs 2–251 randomize only the hydraulics, runs 252–501 only the mechanics, and runs 502–1000 both together, giving the composition shown in Figure 4.2(a). This structure is deliberate: it makes it possible to attribute a later modelling failure to the hydraulic or the mechanical axis by retraining on one block at a time, which a uniformly mixed batch would not allow.

4.2.2 Hydraulic Parameters

Table B.1 lists the hydraulic axes. They were chosen to correspond to the physical mechanisms that make one machine differ from another in service, in the sense discussed in Section 2.1.2: air works its way into the fluid, spools and seals wear, and relief settings drift.

The seal-wear coordinate deserves comment because it is the one place where a single sampled number is deliberately allowed to move several model parameters together. A worn cylinder does not become leakier without also becoming slower, so $w_{\mathrm{seal},i}$ scales the

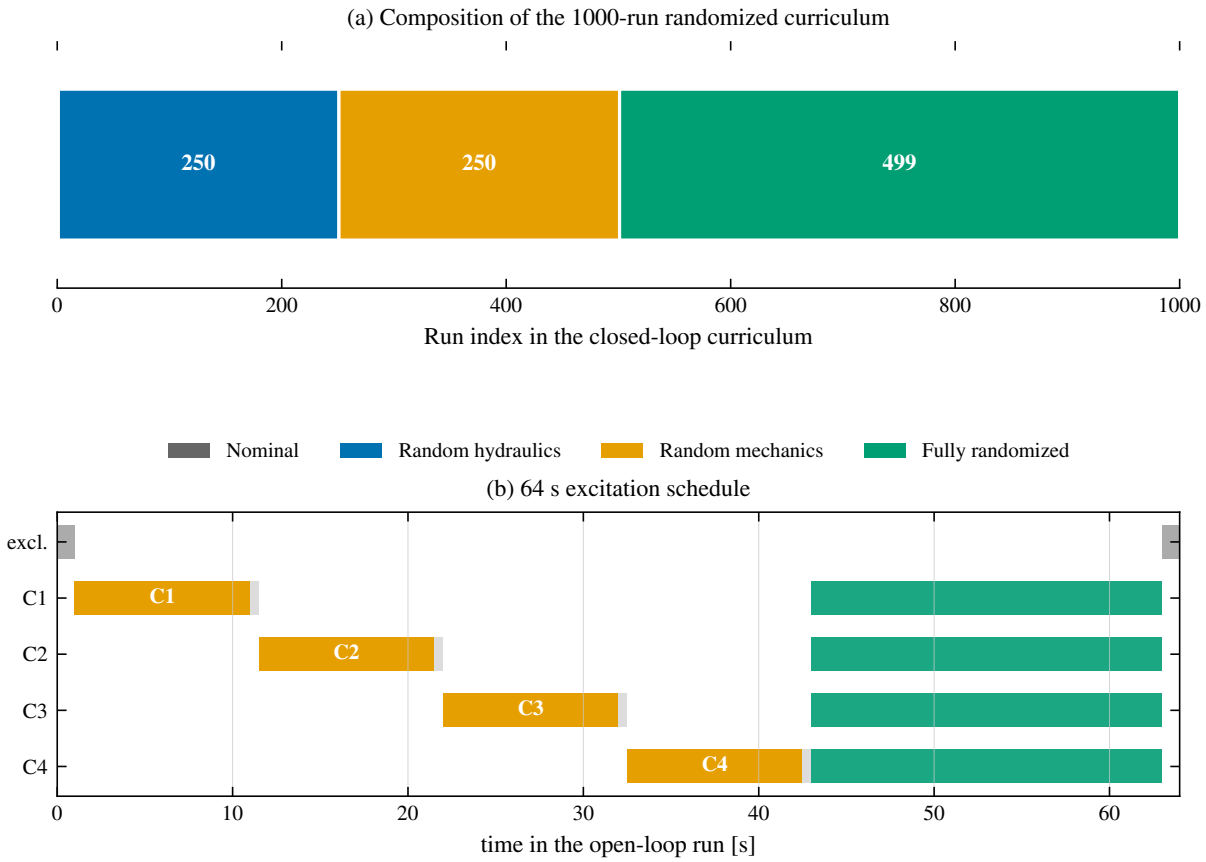


Figure 4.2: Design of the two collection campaigns. (a) Composition of the closed-loop curriculum: one nominal run, then 250 hydraulics-only, 250 mechanics-only and 499 fully randomized draws. (b) Timeline of one open-loop excitation run.

piston clearance linearly, the valve leakage area with the cube of the same factor, and the spool time constant from 8 ms to about 13 ms at the top of the range. Sampling those three independently would have produced physically incoherent machines, some of them very leaky and yet perfectly fast.

4.2.3 The Compressibility Axis

The entrained-air and dead-volume coordinates randomize the chamber dead volume, which spans 505 to 579 mL across the batch. This reaches the pressure dynamics through the continuity equation (3.4), which carries the chamber volume: a larger chamber is more compliant and slower, a smaller one stiffer and faster. The compressibility axis therefore enters the batch as capacitance variation.

4.2.4 Mechanical Parameters

The mechanical axes, listed in Table B.2, are applied to all fourteen rigid bodies of the mechanism: the six links, the four cylinder barrels, and the four pistons. For each body the mass is perturbed by up to ±20 %, the centre of mass by up to ±5 % of its nominal offset, and the inertia tensor together with its products of inertia is perturbed consistently

Algorithm 4.1: Closed-loop curriculum generation.

input : run budget $N = 1000$; joint limits $\boldsymbol{q}_{\min}, \boldsymbol{q}_{\max}$; nominal parameters $\boldsymbol{\theta}_0$
output : dataset $\mathcal{D} = \{(\text{trajectory}_i, \boldsymbol{\xi}_i)\}_{i=1}^{N}$

$\mathbf{L} \leftarrow \textsc{LatinHypercube}(N, 12)$ `// one design for the whole batch`
for $i \leftarrow 1$ **to** N **do**
  assign i to a curriculum block (nominal / hydraulic / mechanical / full)
  $\boldsymbol{\xi}_i \leftarrow \textsc{DrawParameters}(\mathbf{L}_i, \text{block})$ `//` $\varepsilon_{\text{air}}, \varepsilon_{\text{vol}}, c_d, w_{\text{seal}}, p_{\text{set}}$ `; masses, CoM, inertia, stiffness`
end
for $i \leftarrow 1$ **to** N **do**
  inject $\boldsymbol{\xi}_i$ into the Simscape model
  $n_w \leftarrow \textsc{RandInt}(12, 20)$; $\boldsymbol{W} \leftarrow \textsc{LatinHypercube}(n_w, \text{JDOF})$ within $[\boldsymbol{q}_{\min}, \boldsymbol{q}_{\max}]$
  assign each segment a speed class (slow / medium / fast / variable); **no** dwell
  $\boldsymbol{q}_{\text{ref}}(t) \leftarrow \textsc{Interpolate}(\boldsymbol{W})$ over 22 s
  simulate with `ode23t`; log on the 0.05 s grid
  store $(\boldsymbol{u}, \boldsymbol{x}, \dot{\boldsymbol{x}}, \Delta\boldsymbol{p}, \boldsymbol{F}, \boldsymbol{q}, \dot{\boldsymbol{q}}, \boldsymbol{q}_{\text{ref}}, \boldsymbol{\xi}_i)$
  if $i \bmod 100 = 0$ **then** checkpoint $\mathcal{D}$ to disk
end

with the mass change, so that the body remains physically realisable rather than acquiring an inertia that its mass could not produce.

The joint stiffness and damping terms model compliance in the bearings and linkage rather than in the fluid. They matter here because they set how much of a pressure transient appears as boom oscillation instead of clean motion, which is the coupling that a purely kinematic controller cannot anticipate.

4.3 Closed-Loop Data Collection

Algorithm 4.1 summarizes the procedure. The parameter table is fixed before simulation, and each run receives a new reference path.

4.3.1 The Controller as a Data-Collection Instrument

Every closed-loop run uses the same integral-backstepping controller at 0.005 s. Its gains remain fixed, so response differences are caused by the plant draw rather than controller retuning.

Joint 2 uses PI control with velocity damping and gravity bias because it carries the boom load. A barrier term enforces the URDF limits. Per-axis command limits are [0.95, 0.95, 0.80, 0.80, 0.90]; the first four are also stored in the inverse-dynamics checkpoint (Section 5.2).

4.4 Open-Loop Excitation Experiments

4.4.1 Limitations of the Closed-Loop Batch

The first forward model achieved one-step $R^2 \approx 0.999$ on closed-loop data but was almost insensitive to valve input: zeroing the input changed long-horizon error by only a few

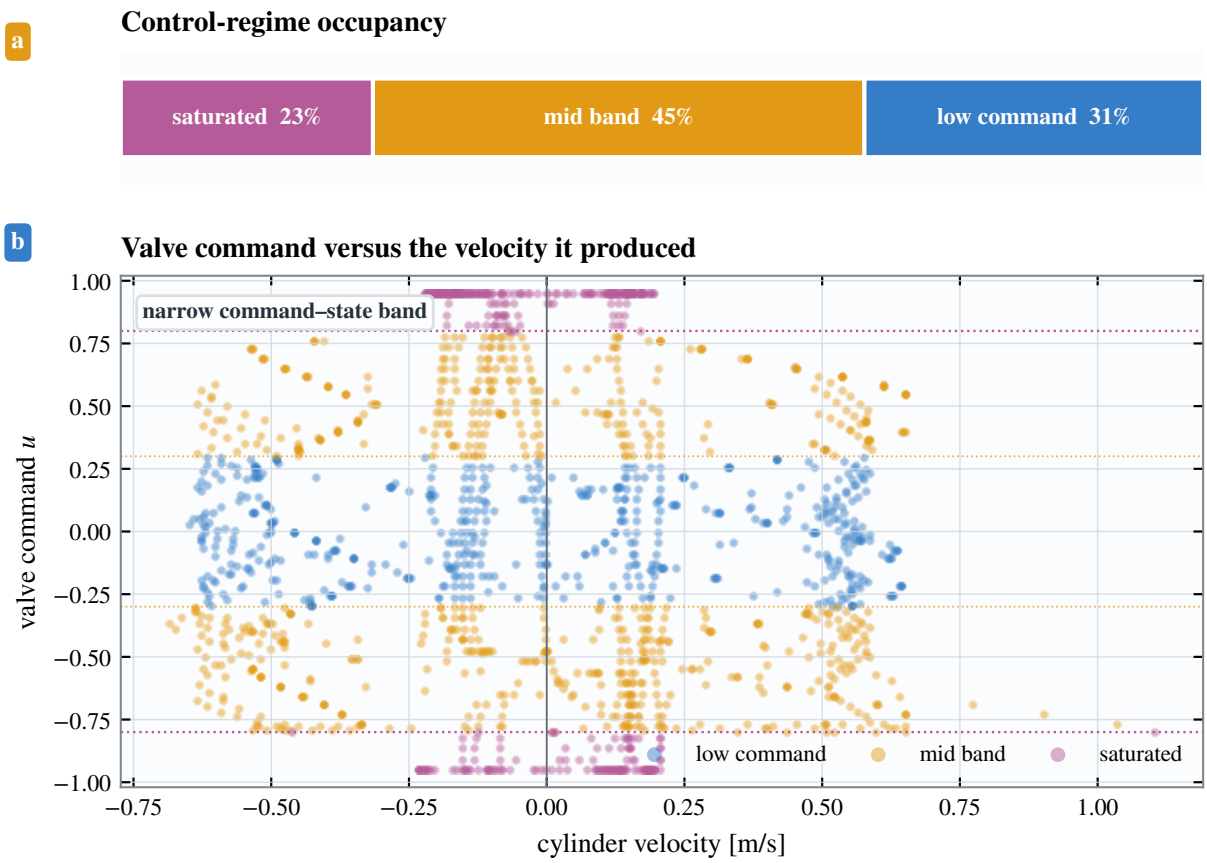


Figure 4.3: The closed-loop batch is diverse in control effort but not identifiable. (a) Fraction of time the valve spends saturated, in the mid band and in low-command creep. (b) The same run in the command–state plane, where the correlation that blocks identification is visible.

percent (Section 5.3.1). In closed loop, command is largely determined by the state history already given to the model, so the data do not separate command causality from state correlation [Ljung 99].

Figure 4.3 confirms that coverage alone is not the issue. The valve is saturated for about one quarter of a pooled run, operates at mid-authority for roughly half, and otherwise remains in low-command creep. Despite this spread, command and cylinder velocity remain concentrated along a narrow band. The data are diverse in control effort but not identifiable, motivating a separate open-loop experiment.

4.4.2 Design of the Excitation Signal

The excitation run lasts 64 s and follows the fixed schedule in Figure 4.2(c). One second of startup is simulated and then discarded, because the plant is settling from its initial condition and those samples describe the initialisation rather than the dynamics. Four sequential slots follow, each giving one cylinder ten seconds of excitation followed by a half-second zero-command gap. The gaps make the boundaries between single-axis segments unambiguous. A twenty-second phase then drives all four channels at once, and a final second of zero command closes the run. Commands are limited to ± 0.90 of full authority.

An initial rapidly switching signal met the formal persistence condition but kept each piston near one stroke position. The final design instead uses long full-authority plateaus in the sequential phase and slow, high-authority motion in the simultaneous phase. A slow near-square cycle is combined with distinct medium-frequency channel components, with raised-cosine entry and exit ramps. This provides both input excitation and useful stroke coverage.

4.4.3 Plant Draws and Acceptance

The same excitation is applied to four plant draws taken from the same parameter space as the closed-loop campaign: nominal, randomized hydraulics, randomized mechanics, and fully randomized. Holding the input fixed and varying only the plant is what makes the four runs directly comparable, since any difference in the response is attributable to the parameters.

Each run is checked for stroke coverage before it is accepted. The collector requires at least 10 % of the available stroke on cylinders 1–3 and 8 % on cylinder 4, and records the achieved coverage per channel in the run summary. The runs retained here cover 33 % of the stroke on cylinder 1, and 12 % on cylinder 4, which is the narrowest of the four. Every channel clears the acceptance floor, and the audit of Section 4.10 confirms that the retained material is informative at these coverages: the instantaneous command matrix has full rank four and the lagged regressor matrix reaches rank eighty.

An earlier export was rejected because half its runs contained duplicated state trajectories. The acceptance checks in Section 4.10 now detect this failure directly.

4.5 Combining Closed-Loop and Open-Loop Datasets

The inverse model uses only closed-loop data because its target is the controller command for a measured state. The forward model is retrained on the strict open-loop export while retaining closed-loop replay at weight 0.25. Without replay, four excitation runs do not provide enough pose coverage.

Figure 4.4 shows the difference: closed-loop data covers the reachable range, while open-loop data concentrates at identifiable valve plateaus. The forward model therefore uses both.

For ROM training, the inverse and forward exports are joined by run identifier, time index and sample time. The join is accepted only when their sixteen-dimensional cylinder states match sample for sample. It contains 452 400 transitions at 0.05 s from 1 040 runs, split into 936/52/52 complete runs. Valve command is used only to supervise the ROM's internal valve head; it is not a runtime input (Section 5.4).

This aligned nominal-plant export trains the inverse, forward and ROM models. It predates the thesis and its collection script is unavailable, so its measurable contents and audits define its reproducibility boundary. Exact alignment, increment consistency and the identifiability failure were checked directly on the export.

The separate thousand-run randomized curriculum was collected for an earlier adaptation plan. No reported result is trained on or evaluated against it; the identifiability chain uses the IsaacLab collection of Section 4.7, and the deployed residual is trained on-policy (Section 4.8.4). The curriculum is retained as provenance and documented in Appendix B.

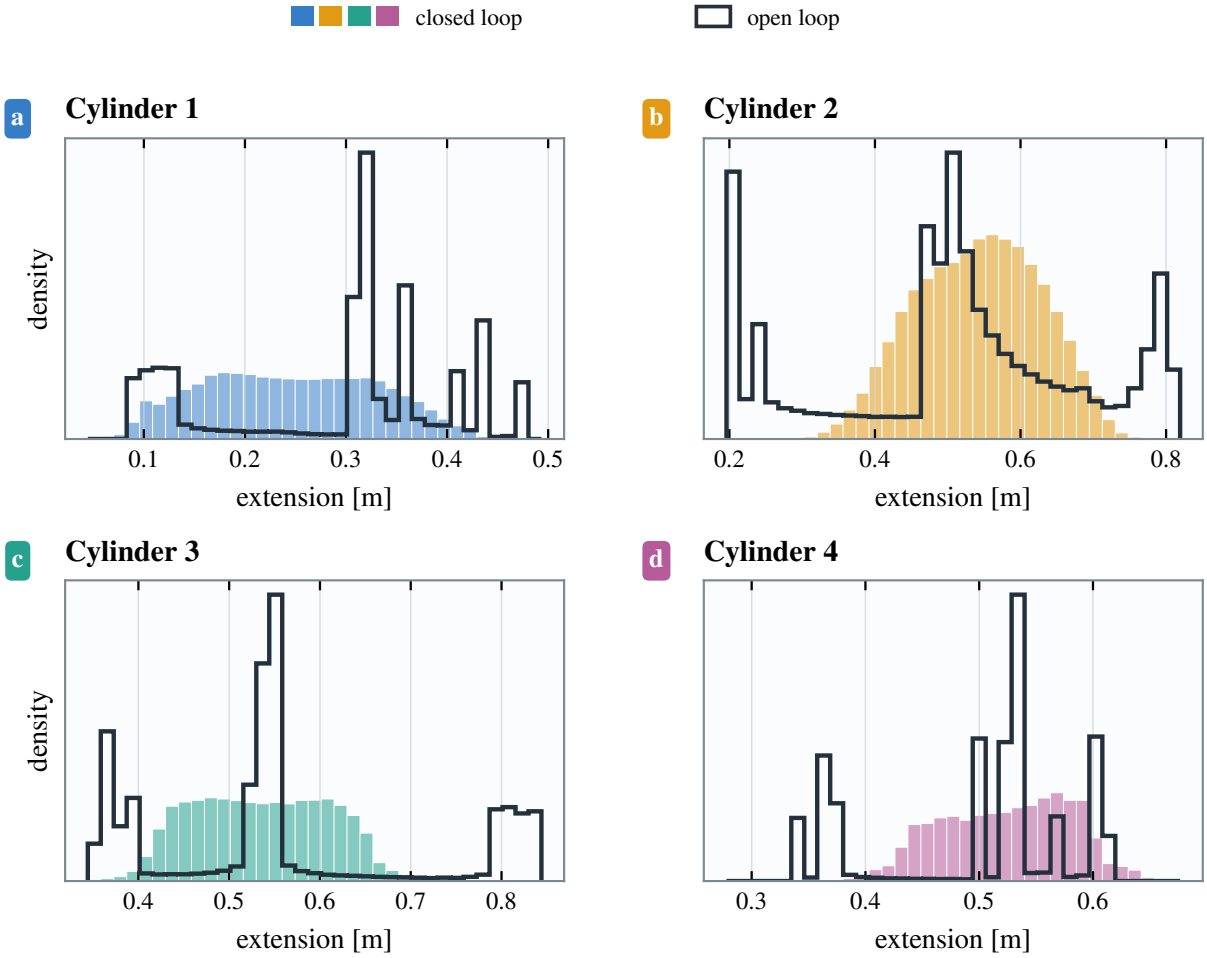


Figure 4.4: Extension coverage of the two campaigns, per cylinder and pooled over runs (closed-loop filled, open-loop outline). The closed-loop data fills the reachable range smoothly; the open-loop data concentrates into the plateaus where the valve is identifiable.

4.6 Closed-Loop Calibration Data for the Command ROM

The ROM drives a PhysX articulation whose compliance and contact response are not present in Simscape. Further Simscape training cannot remove the resulting deployment offset.

The calibration set therefore comes from closed-loop ROM–PhysX operation on the native 0.05 s grid. Its 4 084 transitions store the six-step runtime history and measured PhysX transition.

Only the command-gated correction head is trainable; all other parameters are frozen and the correction is capped at 3 mm. Original-data replay at weight 0.25 preserves wider pose coverage. The gate makes the correction zero at zero command, retaining stationary hold.

The trajectories predate the moving-verge benchmark and exclude cutter–ground contact. The bounded head limits, but does not remove, this coverage gap. Section 3.4.1 explains the consequence for the three-way comparison.

4.7 The IsaacLab Deployment Collections

Two collections are recorded in the deployment plant. They share their factor design, their domains, their splits and their freezing procedure, and differ in one respect only: how the arm is driven. The first is driven by the task controller and is the collection the inverse and forward models are scored against in Section 5.2.2; the second is driven by open-loop excitation and is the one the adaptation module is trained on. Why both exist is itself a result, and it is given in Section 4.7.5.

4.7.1 Provenance of the Deployment Data

The models that carry the identifiability argument (the system-dynamics surrogate, the privileged factor encoder, and the offline history encoder) describe the plant that the adaptive controller is closed around. That plant is not the Simscape model. It is the command ROM plus PhysX, the same four-dimensional cylinder command port used at deployment. A surrogate identified on Simscape data would be a model of a model, and the latent it defines would describe variation the deployment plant does not necessarily exhibit. Both collections are therefore made *in* the deployment plant, and the frozen ROM, ID and FD checkpoints enter this chain only as fixed components, never as things to be retrained.

4.7.2 The Physical Factor Vector

The randomized quantity is a six-dimensional physical factor vector

$$\mathbf{e} = \left[e_{\text{flow}},\ e_{\text{compliance}},\ e_{\text{seal}},\ e_{\text{pressure}},\ e_{\text{mass}},\ e_{\text{COM}_z}\right], \tag{4.1}$$

with the ranges of Table B.3. Four coordinates are hydraulic and two mechanical, which is what makes the hydraulic-only, mechanical-only, and combined domains of Section 4.7.3 separable. Each candidate was admitted only after a one-factor sensitivity screen confirmed that moving it alone produces a measurable change in cylinder extension, velocity, pressure, or end-effector motion. Parameters whose effect could not be measured, or which could not be varied safely, were rejected rather than carried as decoration.

One omission is deliberate. Actuation delay is not a randomized coordinate. Delay in this system is an *emergent* property of flow, compliance, and load rather than a free parameter, so it is measured as a response property of each run (Chapter 7) instead of being supplied to any model as a known factor. Handing an estimator a factor that the real machine would never publish is the same mistake as handing it the true latent.

4.7.3 Domains and Factor Design

Runs are organized into five domains (nominal, hydraulic-only, mechanical-only, combined-training, and an *unseen* combined-test domain), and one factor vector is fixed for a complete run, so that no run mixes plants. Factor configurations are drawn with deterministic scrambled Sobol sequences [Sobol' 67] rather than independent uniform sampling, with explicit boundary anchors so that the corners of the space are represented and not merely approached, and with fixed per-domain seeds so the design is reproducible independently

of the data it produced. Combined draws that compound into an excessive response gain are rejected at draw time rather than being collected and discarded later.

The resulting design is auditable: 98.3 % of the factor keys are unique, the active-factor matrix is full rank, every factor quartile is populated, and the factor keys of the training, validation, and test splits are disjoint. That last property is the one that matters most for the claims of Chapter 7: a test run does not merely come from a different episode, it comes from a plant the models have never seen.

4.7.4 Collection Procedure: The Task-Controller Runs

Every run is collected through the same interface the policy uses. The DLS controller closes the loop on measured cylinder state and issues its command through the four-dimensional action port into the command ROM and PhysX. No joint-target bypass is used, and no PPO policy is used to collect identification data, because a policy trained on the same plant would reintroduce the closed-loop correlation that Section 4.4 spent a campaign removing.

Independent excitation is supplied by dedicated four-channel PRBS windows applied through that same action port, with a small amplitude and a hold of ten control steps, so that the excitation is causally independent of the state without ever leaving the safe envelope. Probe actions are logged separately from controller output, so any analysis can distinguish the two. Each run follows a fixed eleven-segment structure (settle, three postures, three PRBS windows, DLS task motion, reach-and-return, and stow), giving 11 304 control steps and 188.4 s of simulated time per run. The separate open-loop excitation collection of Section 7.5.1 uses a different schedule and runs to 11 040 steps. Three postures plus line, coupled-sine, and reach-and-return motion were verified in a pilot to give full rank over all four action channels under varied geometry and load.

Figure 4.5 summarizes the IsaacLab collection configuration and its frozen run-level outputs.



Figure 4.5: IsaacLab adaptation-data collection. Each run uses one fixed six-factor plant draw and combines DLS task motion with independent PRBS probes. State, action and factor keys are logged, audited and frozen into disjoint train, validation and held-out test splits.

4.7.5 The Open-Loop Excitation Collection

A second collection of 118 runs was recorded in the same deployment plant, with the same six-factor design and the same domain and split structure, and it is the one the adaptation module is trained on. It differs from the collection above in a single respect, and that respect is the point of it: there is no task controller. Each run is driven by open-loop excitation alone, the vehicle is stationary, and no policy is in the loop. The excitation is the same four-channel PRBS used as a probe above, at amplitude 0.02 and held for ten control steps, but it now drives the whole run rather than three windows inside one. Each run is 11 040 control steps, giving 1 302 720 steps across the collection.

The reason for collecting twice is an identifiability result rather than a preference, and it is the sharpest one in this thesis. Under a task controller the command is computed from the state, so command and response are close to redundant and the plant has little left to explain. Measured on the earlier task-controller collection, a latent-conditioned state predictor scored *behind* its own no-latent baseline, 0.17596 against 0.17278, with the privileged latent worse still at 0.18515; the same architecture, objective and selection criterion on this collection lower free-running error by a median of 22.3 % (Section 7.5.2). Nothing about the model changed. What changed was whether the data contained an answer. Training the adaptation module on task-controller data would therefore have produced a confident negative about an architecture that works.

The collection records its own feature contract, and what each consumer takes from it is set out separately: Section 4.8.2 for the adaptation module and Section 4.8.3 for the prediction models.

4.7.6 The Frozen Dataset

Both collections contain 118 complete runs under the identical domain design: 2 nominal, 40 hydraulic-only, 12 mechanical-only, 48 combined-training, and 16 unseen combined-test. Collection ran in four vectorized IsaacLab environments with per-environment factors and logging. Slots that tripped a safety or validity check were discarded and retried individually rather than being silently kept. All 118 runs passed the final collection gate, which checks finite physics, exact model-grid alignment, command rank, factor-response authority, state coverage, and saturation limits.

Each dataset is then *frozen*: the raw runs are checksummed and the run-level split is fixed at 82 training, 20 validation, and 16 reserved combined-test runs, with zero factor-key leakage between them. The reserved sixteen do not influence history length, checkpoint, hyperparameter or architecture selection. They are scored only after each model is fixed and are treated as a single-use test set. Test results are reported with validation results, not in place of them.

4.7.7 Learning Timebase and Hydraulic State

The learning timebase is the actuator model grid, not the control clock. Samples are exact 50 ms ROM transitions taken at control indices $0, 3, 6, \ldots$: state row t and command row t predict state row $t + 3$. Nothing is interpolated. This is the same alignment rule as Section 3.5.1, and it is enforced rather than assumed, because a model trained across a mismatched interval is wrong by a fixed factor in the quantity it exists to supply.

The hydraulic state used by the system-dynamics surrogates is

$$\boldsymbol{s}_t = \left[\boldsymbol{x}_t^{1:4},\ \dot{\boldsymbol{x}}_t^{1:4},\ \Delta\boldsymbol{p}_t^{1:4}\right] \in \mathbb{R}^{12}, \tag{4.2}$$

with joint position and velocity carried as *context* only, and the prediction target the next 50 ms state delta. Differential pressure is not a measured signal. At runtime it is the actuator model's recursive state, generated by the command ROM or frozen forward model (Section 3.3). The system-dynamics surrogate and AAM may use it because it exists inside the deployed loop, but it inherits actuator-model error. Causal history is therefore retained to represent pressure transients that one sample cannot summarize.

4.8 Feature and Target Construction

4.8.1 The Actuator Models

The three learned actuator models take cylinder-space features, in the ordering and notation fixed in Section 5.1. Their input contracts differ because their questions differ. The inverse model receives the measured cylinder state, the logged rod forces, the desired increments, and the previously applied command, twenty-eight numbers in all. The forward model receives the applied valve command and the measured state, twenty numbers. The reduced-order model receives a six-sample history of state, previously predicted valve, and desired command, twenty-four numbers per step.

Three conventions apply throughout, and they are the same ones the models enforce at inference time.

Increment Targets

Every dynamic target is a within-run one-step increment, $\Delta\boldsymbol{x}_k = \boldsymbol{x}_{k+1} - \boldsymbol{x}_k$, never an absolute state. This removes the large static component of the signals and, more importantly, makes the trivial persistence solution an explicit baseline that a model can be measured against rather than a hiding place it can settle into.

Causal Alignment

Features at step k contain only quantities available at step k. The extraction stage verifies this by recomputing every logged target from the within-run difference of the states and requiring exact agreement, a check reported in Section 4.10.

Time-Base Conversion

The open-loop campaign is logged at 0.01 s but the models run at 0.05 s, so five consecutive raw samples are combined into a single transition. The valve input attributed to that transition is the command held at the start of the window, which is the causally correct choice. The mean command over the window is also recorded, but only for analysis. Cylinder extension velocity is not logged directly in the closed-loop archive and is obtained by differentiating extension on the output grid, which is consistent with the trapezoidal identity that the strict data satisfies to $R^2 = 0.994$–0.9997 (Equation 5.6).

4.8.2 Features and Targets for the Adaptation Module

The adaptation module is trained on the open-loop collection of Section 4.7.5. Its history window carries only quantities the deployed stack produces at runtime: the five joint positions, the five joint velocities, the four cylinder pressure differences and the four previously applied valve commands, giving the eighteen-value row of Section 5.6. The pressure entry is the learned actuator's own recursive state rather than a measurement (Section 3.3, Table 3.2); it is admitted because the model that generates it runs inside the deployed loop, and the consequence of that choice is examined where the estimator is evaluated. Four classes of signal are excluded by construction, and the exclusion is enforced by an explicit input mask rather than by convention:

- the physical factors $\mathbf{e}$ and the true latent $\mathbf{z}_{\mathrm{SD}}$, because recovering them is the estimator's job;
- any future observation, because the estimator is causal;
- the hidden internal state of the command ROM, because the deployment interface does not expose it;
- the desired task reference, so that the latent describes the plant rather than the trajectory being asked of it.

Windows are contained within a single run and never cross a run boundary, and each window carries the stopped-gradient target $\mathbf{z}_{\mathrm{SD}} = \mu(\mathbf{e})$ of its own run together with the run identifier, domain, split, and valid history bounds. Applied to the 118 frozen runs this yields 444 624 causal history ticks. What is built from them, how the history length is chosen, and how the result is validated downstream belong to Chapter 5.

The residual encoder deployed in AF shares this row but uses no collection at all, for the reason given in Section 4.8.4.

4.8.3 Features and Targets for the Prediction Models

The prediction models of Section 5.5 read the same runs through a different contract. Each sample is a transition at the 50 ms tick rather than a history window, pairing the previous and current valve command with the state, and training draws five consecutive transitions at a time. They additionally receive the privileged factor vector $\mathbf{e}$, which the adaptation module never sees and which is what allows them to define the target the module is then asked to recover.

4.8.4 On-Policy Data for the Deployed Adaptation Residual

The residual that carries the adaptation result in AF is not trained from the frozen collection above, and the difference is one of kind rather than of scale. It has no dataset, no stopped-gradient latent target and no run-level splits, because its encoder never regresses a target at all: the encoder and the residual head are optimized together under the reinforcement objective, so their training data is the on-policy experience the stack generates as it learns.

The reason for the change is a measured failure of the offline route, reported in Section 6.7.5. An encoder fitted to the dynamics target of this collection and then frozen carried about 0.4 % plant identity into the controller and scored 7.1 % worse than simply supplying the mean context. A latent that is accurate against a dynamics target is not thereby useful to a controller, and no amount of additional offline data addresses that, because the deficiency is in the objective rather than in the sample count.

What replaces the dataset is a plant design, since on-policy training makes the training distribution a property of the environment rather than of a file. The cutter mass is drawn from two well-separated modes and flips between them *mid-episode*, so that no single compromise response is correct in both halves of a run. The flip instant is jittered per environment, so it cannot be anticipated on a timer, and the direction is balanced across environments, so it cannot be assumed. The consequence is that history is not merely available to the residual but necessary to it: an arm that cannot see the history is measured, and it does not merely fail to improve on the frozen base but falls well below it (Section 7.6.2).

Two properties of the frozen collection are nonetheless inherited unchanged. The history row is the same eighteen-value row defined above, with the same four exclusion classes enforced by the same input mask, so nothing enters the on-policy encoder that was barred from the offline one. And the plant is still the deployment plant, the command ROM plus PhysX, so the change of training regime does not smuggle in a different system.

4.9 Preprocessing and Data Splits

Splitting is done at the level of complete runs, never of individual samples. A random sample-level split would place transitions from the same trajectory on both sides of the boundary and report a validation score that mostly measures interpolation within a run. For the command-conditioned dataset the 1 040 runs are divided into 936 training, 52 validation, and 52 test runs, giving 407 160, 22 620, and 22 620 transitions respectively, with no run identifier appearing in more than one split. The IsaacLab deployment collection is split by run *and* by factor configuration (82/20/16, Section 4.7), which is the stronger condition: separating runs alone would still let two runs of the same plant fall on opposite sides of the boundary and turn a claim about unseen plants into a claim about unseen episodes.

The strict open-loop export needs a different treatment because it contains only four runs, and holding one out would remove an entire plant draw from training. Instead each run is divided into twenty temporal blocks, every fifth block is assigned to validation, and a five-sample guard interval is discarded at each block boundary so that no 0.05 s transition window spans the split. A simple last-twenty-percent split was tried first and rejected: it changed the spread of the targets between training and validation by a factor of 1.3 to 2.3, because the final fifth of the run is the simultaneous excitation phase and is dynamically unlike everything before it. That split would have measured extrapolation to a different regime and called it generalisation.

Normalization statistics are computed on the training split only and stored inside each checkpoint, so that a model always normalizes with the statistics it was fitted with. Mixing statistics between checkpoints silently invalidates any comparison, and the loader enforces the stored contract rather than recomputing it.

Finally, samples are carried with an explicit validity flag rather than being silently dropped. In the strict export 93.7 % of samples are marked valid. The remainder are the excluded startup and shutdown seconds and the zero-command gaps. Keeping them in the archive with a flag, instead of deleting them, means the time base stays contiguous and the within-run difference checks remain meaningful.

4.10 Dataset Diagnostics

No dataset in this thesis is used before it passes an explicit audit, for the reason given at the end of Section 4.4. The audit of the strict open-loop export is reproduced in Table B.4.

The two rank figures are the ones that speak directly to requirement R2. The instantaneous command matrix has rank four, so no channel is a linear combination of the others, and the lagged regressor matrix built from the command history has rank eighty, so the excitation is persistently exciting to the order the models need. The condition number of that lagged matrix is about 4.6×10^3, which is finite and workable rather than comfortable. It is a consequence of using long plateaus, and it is the price paid for the stroke coverage those plateaus buy.

The exactness of the three increment checks deserves a word, since a maximum error of exactly zero invites suspicion. It is expected here: the targets are computed from the logged states by the same within-run difference that the check recomputes, so the check verifies that the extraction stage did what it claims, not that the simulation is accurate. It catches misalignment, off-by-one indexing, and rows that have been reordered or concatenated across run boundaries, which are the failure modes that occur. Physical fidelity is a separate question, addressed by the validation study in Section 3.4.4.

Two properties of the Simscape material are recorded here because later chapters refer to them. Stroke coverage is 12 % on the head-pivot cylinder, the narrowest of the four channels, and that cylinder is also the one the strict forward model predicts least well. The rod-force channel carries zero supervised weight in the reduced-order model. That exclusion is a design choice rather than a data defect: force is generated internally by the PhysX plant at deployment, so the actuator model has no need to reproduce it, and the logged channel additionally fails a piston-area consistency check on this export, which confirms the choice. No result rests on it.

The IsaacLab collection is audited against different failure modes. Design adequacy is checked before use (unique factor keys, full active-factor rank, disjoint keys across splits), physical validity per run (finite physics, exact model-grid alignment, full command rank, bounded saturation), and leakage structurally, with privileged channels removed by an explicit input mask rather than by trusting the feature list. Its limitation is scale: 118 runs over six factors is a sparse design, which is why the identifiability claims of Chapter 7 are reported with paired bootstrap intervals and per-run detail rather than pooled point estimates. The limitation is specific to what this collection supports. It does not bound the closed-loop adaptation result, which is not estimated from these runs at all but from paired environments generated on-policy under the plant design of Section 4.8.4.

With the data described and audited, the next chapter turns to what was learned from it: the inverse and forward models that form the IF actuator path, the command-conditioned reduced-order model that replaces them in R, and the system-dynamics surrogate and encoders built on the IsaacLab collection.

5. Learned Models and Online Adaptation

Chapter 4 established how the datasets were generated and why both closed-loop operation and open-loop valve excitation were required. This chapter develops the learned representations that consume them: the inverse-dynamics (ID) and forward-dynamics (FD) networks of the IF actuator path, the command-conditioned reduced-order model (ROM) that replaces them in R, and the system-dynamics surrogate, privileged factor encoder and Arm Adaptation Module (AAM) that together carry the identifiability argument.

The chapter has a direction, and it is worth declaring at the outset because the final section is a negative result. Each link of the chain is established in turn: the latent exists, it is recoverable causally from observable history, the recovered estimate is as informative as a privileged one, and it improves a model that consumes it. The chain then fails at its last link, where the correction must reach a command that is already saturated. That failure is diagnosed rather than reported, because the diagnosis is what specifies the redesign carried out in Chapter 6 and evaluated in Chapter 7.

Two commitments organise the chapter. The first is that the surrogate is kept strictly separate from the deployed actuator: the system-dynamics surrogate is an instrument for establishing whether a compact dynamics latent exists and is useful, never a component of any control stack. The second is that no model is accepted on a supervised score, because a recurring result of this work is that such scores can be not merely optimistic but uninformative: a forward model reached a one-step R^2 near 0.999 while having learned almost nothing about valve authority (Section 5.3). Every model is therefore presented with its contract, training objective, and the offline evidence that accepted or refused it. All models operate on cylinder-space signals; the mapping to task space belongs to Chapter 3, the control integration to Chapter 6, and deployment-level results to Chapter 7.

5.1 Modelling Overview and Common Notation

All learned models act on the four hydraulic cylinders of the MULAG arm, indexed $i \in \{1, \ldots, 4\}$ in the fixed order slew cylinder (*Drehzapfen*), boom cylinder I (*Ausleger I*),

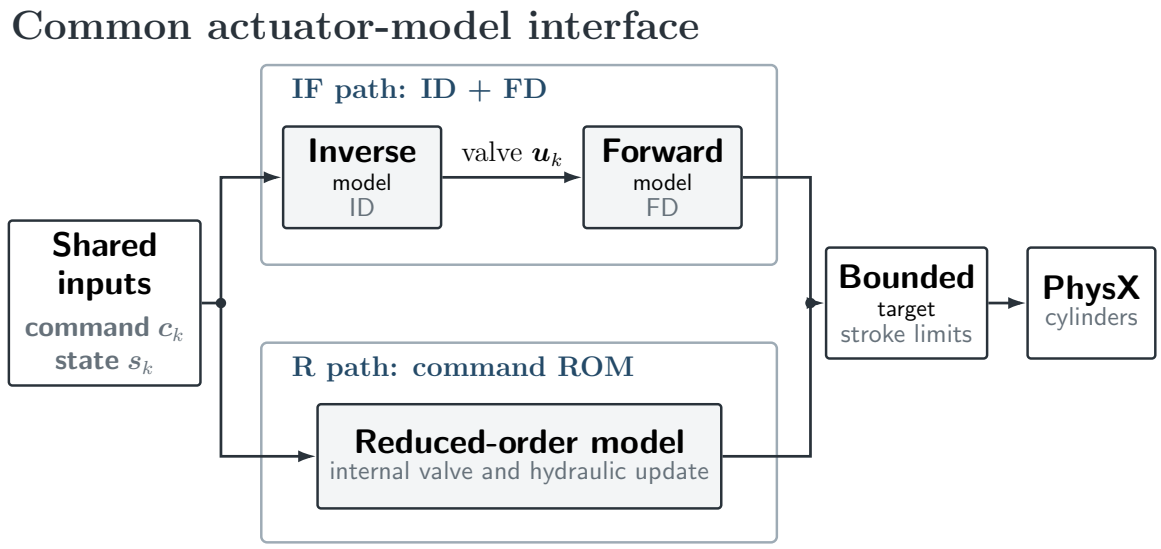


Figure 5.1: Common interface of the two alternative actuator paths. Both receive the desired cylinder command and measured hydraulic state, and both produce the same bounded PhysX cylinder target. The IF path composes the inverse and forward models; the R path uses one command-conditioned reduced-order model.

boom cylinder II (*Ausleger II*) and head pivot cylinder (*Messerkopf-Schwenk*). The passive fifth joint is not actuated and enters no model input. At the learned-model sample time $T = 0.05\,\mathrm{s}$ the discrete index k refers to the cylinder extension $\boldsymbol{x}_k$, the extension velocity $\dot{\boldsymbol{x}}_k$, the chamber pressure difference $\Delta\boldsymbol{p}_k$ and the logged rod force $\boldsymbol{F}_k$, each in $\mathbb{R}^4$. The valve command $\boldsymbol{u}_k \in [-1, 1]^4$ is the normalised spool command of the four proportional valves. The desired cylinder command produced by the task controller is the pair

$$\boldsymbol{c}_k = \left[\Delta\boldsymbol{x}_k^*,\ \Delta\dot{\boldsymbol{x}}_k^*\right] \in \mathbb{R}^8, \tag{5.1}$$

the extension and extension-velocity increments to be realised over the next sample. Predictions carry a hat, and $\Delta(\cdot)_k = (\cdot)_{k+1} - (\cdot)_k$ denotes a within-run one-step increment throughout.

Figure 5.1 compares the actuator paths. In IF, the ID maps the state and desired increment to a valve command, and the FD predicts the resulting increment. In R, one command-conditioned model predicts the valve internally and propagates pressure, velocity and extension. Both paths produce the same bounded PhysX cylinder target in IsaacLab [Mittal 23] under the same policy action contract [Schulman 17].

Three conventions apply throughout. Dynamic targets are within-run increments, with $\Delta\hat{\boldsymbol{x}} = \boldsymbol{0}$ as the persistence baseline. Each checkpoint uses its own training-set normalisation. Inputs at step k are causal, and targets are measured transitions to step $k + 1$.

Simscape supplies the identification data described in Chapter 4. The exported arrays are audited, frozen and used to train the models in PyTorch [Paszke 19]; Simscape is not part of training or deployment.

The closed-loop Simscape export [The MathWorks, Inc. 24a] contains 452 400 transitions at $T = 0.05\,\mathrm{s}$ from 1 040 runs, split by run into 936/52/52 training, validation and test runs. It supplies the supervised ID, FD and command-ROM data. The FD repair uses four open-loop 64-s runs at 0.01 s with rank-4 valve excitation on four plant draws.

5.2 Inverse Dynamics Model

The ID network maps the current hydraulic state and desired cylinder increment to a valve-command change:

$$\Delta \boldsymbol{u}_k = f_{\mathrm{ID}}\left(\boldsymbol{x}_k,\, \dot{\boldsymbol{x}}_k,\, \Delta \boldsymbol{p}_k,\, \boldsymbol{F}_k,\, \Delta \boldsymbol{x}^*_k,\, \Delta \dot{\boldsymbol{x}}^*_k,\, \boldsymbol{u}_{k-1}\right) \in \mathbb{R}^4, \tag{5.2}$$

a 28-dimensional input. The deployed command is $\boldsymbol{u}_k = \mathrm{clip}(\boldsymbol{u}_{k-1} + \Delta \boldsymbol{u}_k,\, -\boldsymbol{u}_{\max},\, \boldsymbol{u}_{\max})$ with per-axis limits $\boldsymbol{u}_{\max} = [0.95,\, 0.95,\, 0.8,\, 0.8]$ stored in the checkpoint. The change formulation retains the previous valve operating point near the direction-dependent dead zone (Section 2.1.2). Logged achieved transitions serve as desired increments because no independent cylinder-space reference was recorded.

The 512-wide residual MLP [He 16] uses SiLU activations, layer normalisation and dropout [Elfwing 18, Ba 16, Srivastava 14]. Adam minimises a smooth-L_1 loss on normalised targets, with saturated samples up-weighted [Kingma 15, Huber 64].

Figure 5.2 separates the three input groups and shows the four residual blocks explicitly. The network predicts a four-axis valve-command increment, which is accumulated and clipped before use.

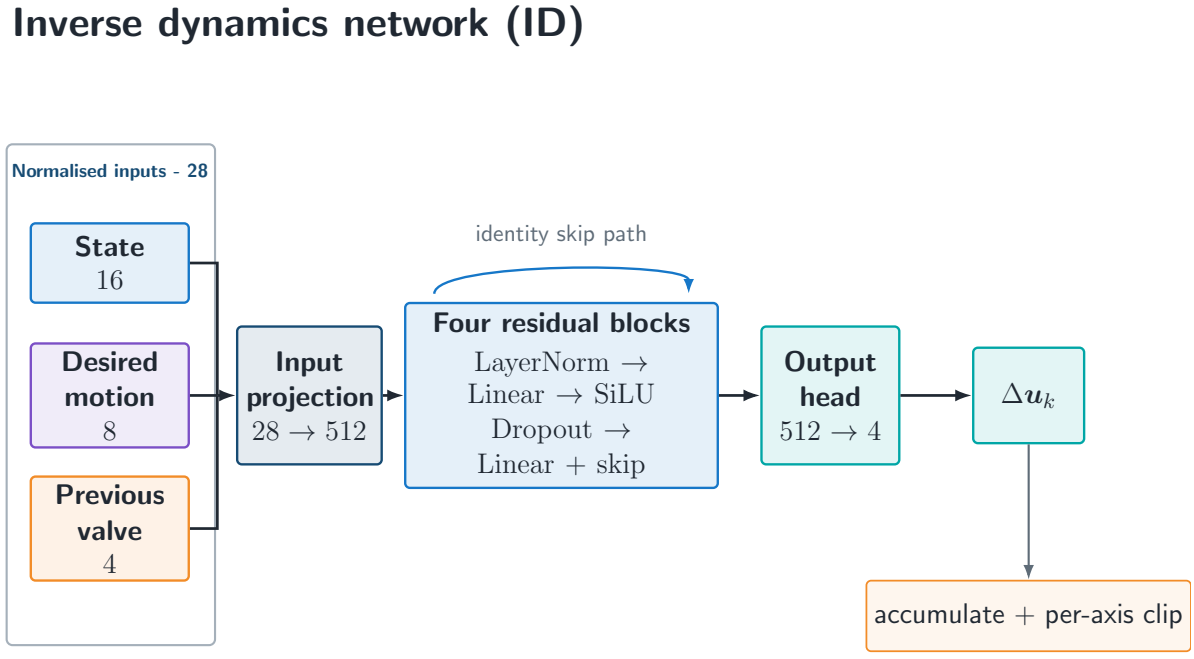


Figure 5.2: Inverse-model architecture. Current hydraulic state (16), desired motion (8) and the previous valve command (4) form the 28 inputs. A 512-wide, four-block residual MLP predicts $\Delta \boldsymbol{u}_k$, which is accumulated and clipped per axis to form the bounded valve command.

Per-valve validation R^2 is 0.9973–0.9996. This shows reproduction of logged commands, not plant authority: in closed-loop data, achieved transitions and logged commands are correlated (Section 2.4.2).

5.2.1 Paired Control Fine-Tuning

The composite is assessed by the *paired differential response*: the FD prediction under the ID command for the requested target minus the prediction under its command for a zero target,

$$\begin{aligned} \boldsymbol{u}_k^{\mathrm{tgt}} &= f_{\mathrm{ID}}(\cdot,\, \boldsymbol{c}_k,\, \boldsymbol{u}_{k-1}), \qquad \boldsymbol{u}_k^{0} = f_{\mathrm{ID}}(\cdot,\, \mathbf{0},\, \boldsymbol{u}_{k-1}), \\ \Delta\hat{\boldsymbol{x}}_k^{\mathrm{pair}} &= f_{\mathrm{FD}}\Big(\boldsymbol{u}_k^{\mathrm{tgt}}, \cdot\Big) - f_{\mathrm{FD}}\Big(\boldsymbol{u}_k^{0}, \cdot\Big). \end{aligned} \tag{5.3}$$

Before fine-tuning, the composite normalised control error was 1.00, equal to the zero-response baseline, and all regression slopes were below 0.004 (Table 5.1). The networks were fine-tuned jointly on $\mathcal{L}_{\mathrm{ctl}} = \|\Delta\hat{\boldsymbol{x}}^{\mathrm{pair}} - \Delta\boldsymbol{x}^{*}\|^2_{\mathrm{norm}} + \lambda_{\mathrm{ID}}\mathcal{L}_{\mathrm{ID}} + \lambda_{\mathrm{FD}}\mathcal{L}_{\mathrm{FD}}$, with the supervised terms retained. Target blocks were shuffled across valid states to break the logged state–target pairing. After 30 epochs, control error fell to 0.109; all slopes exceeded 0.8 and all correlations exceeded 0.92. This frozen checkpoint is the inverse model of the IF actuator path, and of the latent-conditioned adapter route of Section 5.7. It is not part of AF, whose command path contains no inverse model (Section 6.7.1).

Table 5.1: Effect of paired control fine-tuning on the composite differential response (5.3), per cylinder axis on validation data. Before fine-tuning the composite had essentially no control authority despite near-perfect supervised scores of both networks; the composite normalised control error fell from 1.0011 to 0.1086.

	Regression slope		Correlation	
Cylinder axis	before	after	before	after
1 (slew)	0.003	0.818	0.135	0.920
2 (boom I)	0.002	0.928	0.090	0.967
3 (boom II)	0.001	0.849	0.037	0.933
4 (head pivot)	0.004	0.912	0.249	0.958

5.2.2 Transfer of the Frozen Inverse Model

Because the adapter corrects a frozen ID, its transfer was measured on the deployment collection itself: all 118 frozen runs of Section 4.7, comprising 2 nominal, 40 hydraulic-only, 12 mechanical-only and 64 combined runs, of which 16 are held-out combined-test. These runs lie outside the ID's training distribution, which contains only Simscape data, and they are recorded in the plant the adaptive controller is closed around rather than in a separate collection. Two regimes were scored: a teacher-forced one-step transfer, in which the measured cylinder state is supplied at every tick, and a closed-loop transfer, in which the pair's own prediction is fed back while the recorded reference is supplied. Pressure and force are held at their training means, as the deployed feature contract requires.

Table 5.2: Transfer of the frozen inverse and forward pair across deployment domains, all 118 runs of the deployment collection. The one-step columns are teacher-forced; the saturation column is the fraction of pre-clip valve elements at the limit during closed-loop transfer. The ROM valve recorded on these same runs never saturates.

		one-step Δx error		closed-loop
Domain	runs	RMSE	worst run	valve saturation
combined	64	2.36 mm	5.72 mm	0.888
hydraulic	40	2.43 mm	4.96 mm	0.885
mechanical	12	2.81 mm	4.14 mm	0.919
nominal	2	3.36 mm	4.02 mm	0.929
all	118	—	—	0.891

Teacher-forced, the pair transfers well and evenly (Table 5.2). One-step extension error is millimetre-scale in every domain and nearly flat across them, from 2.36 mm on the combined runs to 3.36 mm on nominal, and the held-out combined-test group is no worse than the domains used in training, at 2.34 mm. This is the regime in which IF uses the pair, because the policy closes the loop at every tick and never asks it to predict its own future.

Closed-loop, it does not transfer at all, and the failure is uniform rather than occasional. Every one of the 118 runs leaves the recorded reference by more than one metre, after about 32 s on the combined and hydraulic domains and sooner on the harder ones. The mechanism is saturation: the pair's pre-clip valve command sits at its limit on 89.1 % of elements over the collection, between 88.5 and 92.9 % by domain, while the ROM valve recorded on those same runs never reaches its bound at all. For scale, the measured plant tracks the recorded cylinder reference at about 0.807 mm on the held-out group, so the closed-loop divergence is a failure of model transfer and not a property of the collection.

Two consequences follow and they point in different directions. The pair cannot be promoted as an unadapted autoregressive deployment path, because its own predicted state leaves the support its training data covered. Nothing in that result disturbs the one-step use IF makes of it. What it does disturb is adaptation: a bounded correction added to a command that is already resting on its stops has nothing left to move, which is the finding developed in Section 6.7.1.

5.3 Forward Dynamics Model

The FD network predicts the one-step cylinder response to a valve command,

$$\left[\Delta\hat{\boldsymbol{x}}_k,\ \Delta\hat{\dot{\boldsymbol{x}}}_k,\ \Delta\widehat{\Delta\boldsymbol{p}}_k\right] = f_{\mathrm{FD}}\left(\boldsymbol{u}_k,\ \boldsymbol{x}_k,\ \dot{\boldsymbol{x}}_k,\ \Delta\boldsymbol{p}_k,\ \boldsymbol{F}_k\right) \in \mathbb{R}^{12}, \tag{5.4}$$

using the same residual trunk as the ID network. Supervised pretraining on the closed-loop export, with an auxiliary recursive rollout loss [Nagabandi 18], reached grouped one-step R^2 near 0.999 for the extension increments.

Figure 5.3 shows the forward model separately because its input and output contracts differ from the inverse model despite the shared residual trunk.

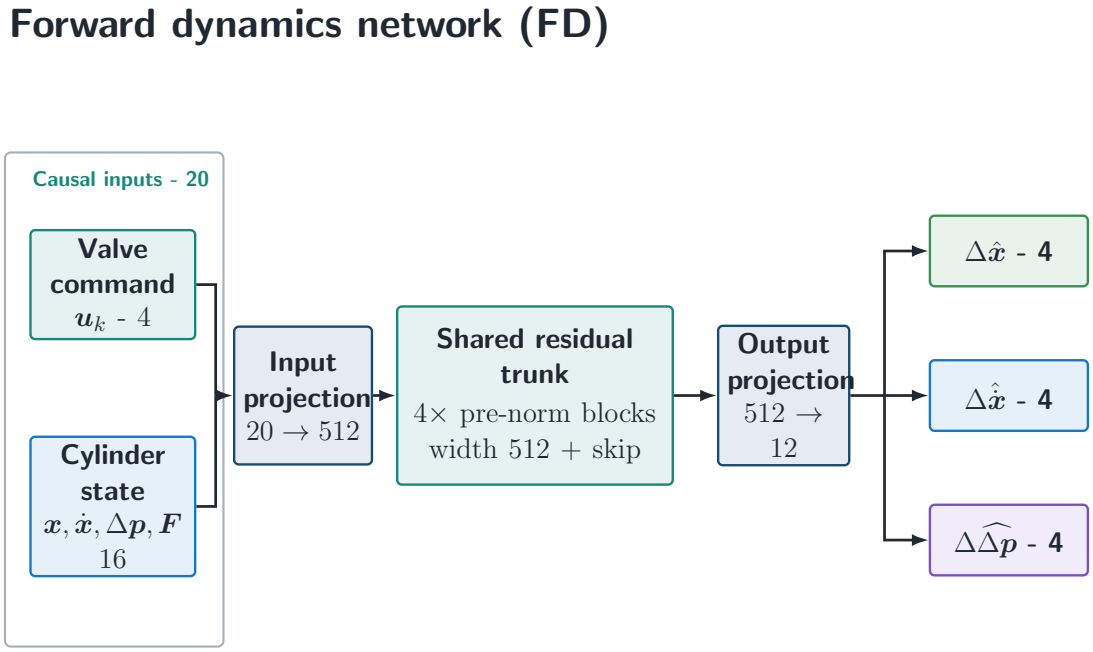


Figure 5.3: Forward-model architecture. The applied valve command and cylinder state condition a one-step hydraulic transition through a 512-wide, four-block residual MLP. Its 12 outputs are extension, velocity and pressure increments. Extension is the integrated controller output; velocity and pressure are diagnostic outputs.

5.3.1 Assessment of the Supervised Score

The supervised score did not establish valve sensitivity. At a 0.5 s horizon, zeroing the valve input changed the rollout-error ratio to only 0.94. The FD had mainly extrapolated the smooth closed-loop kinematics because command and state history were nearly redundant. Closed-loop ablation confirmed that the valve was not identifiable, so repair required new data.

The repair dataset applies rank-4 open-loop valve plateaus to four plant draws. Its trajectories are distinct, valve history predicts pressure with $R^2 \approx 0.80$–0.85, valve-regressor variance-inflation factors are 7–10, and relief saturation occupies 9–22 % of samples. A previous export with duplicated trajectories was rejected [Isermann 11].

Starting from the paired-control checkpoint, strict retraining minimised

$$\mathcal{L}_{\mathrm{FD}} = \mathcal{L}^{\mathrm{strict}}_{\mathrm{Huber}} + 0.25\,\mathcal{L}^{\mathrm{MAT}}_{\mathrm{Huber}} + 2.0\,\mathcal{L}^{(10)}_{\mathrm{roll}} + 10.0\,\mathcal{L}_{\mathrm{kin}}, \tag{5.5}$$

combining the one-step loss on the strict data, a replay term on the original closed-loop data, a 0.5 s recursive rollout loss, and a kinematic consistency term enforcing the trapezoidal identity

$$\Delta \boldsymbol{x}_k = T\,\dot{\boldsymbol{x}}_k + \tfrac{1}{2}\,T\,\Delta\dot{\boldsymbol{x}}_k, \tag{5.6}$$

which holds in the strict data with $R^2 = 0.994$–0.9997. A constant valve Jacobian was rejected because measured authority varies by a factor of 15–75 across pressure

quartiles and can change sign. This operating-point dependence motivates the ROM structure [Merritt 67].

Strict retraining reduces extension rollout RMSE from 75.2 mm to 12.4 mm at 0.5 s and from 157.9 mm to 27.3 mm at 1.0 s. The valve-zero error ratio rises from 0.94 to 1.81, and per-axis one-step extension R^2 improves from $[-5.7, 0.60]$ to $[0.76, 0.98]$. Zero-order hold remains competitive below 0.25 s, and velocity increments remain weak on two axes. Deployment therefore integrates only $\Delta\hat{\boldsymbol{x}}$ and retains velocity as a diagnostic output.

5.4 Reduced-Order Model

The R path uses one command-conditioned model instead of the ID–FD composition. Its structure is grey-box: physical equations propagate pressure, force, velocity and extension. Neural heads estimate valve realisation, load and bounded residuals [Karniadakis 21, Lutter 19]. Valve authority is pressure dependent by construction through the orifice-flow and continuity equations [Merritt 67]. A GRU [Cho 14] reads six samples (0.25 s) of cylinder state, previous predicted valve command and desired command. The measured valve is only an internal-head training target [Hwangbo 19].

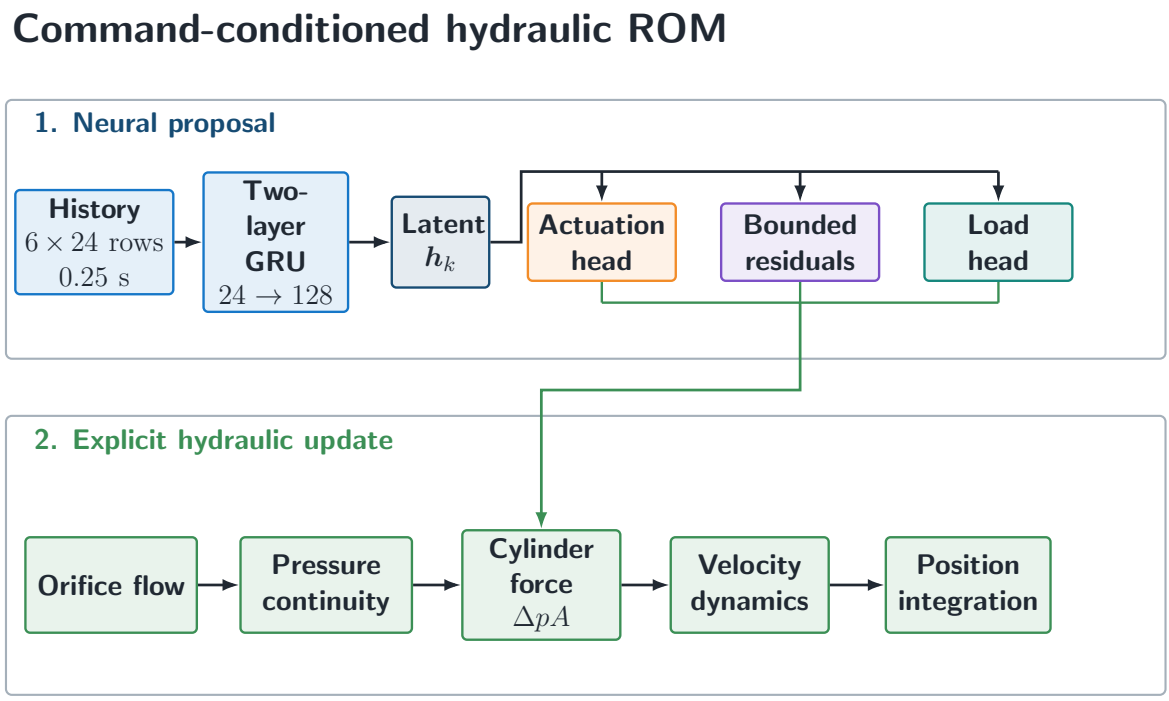


Figure 5.4: Command-conditioned ROM. A recurrent neural proposal encodes a six-step history. Its actuation, bounded-residual and load heads condition the explicit hydraulic update: orifice flow, pressure, force, velocity and position. The explicit chain constrains the predicted transition.

The actuation head uses a learnable smooth dead zone with per-cylinder width and asymmetric gains [Yang 26]. Its output $\hat{a}_i \in [-1, 1]$ drives the orifice-flow and continuity

update

$$q_i = \hat{a}_i\, k_{q,i} \sqrt{\tfrac{1}{2}\big(p_{s,i} - \operatorname{sgn}(\hat{a}_i)\,\Delta p_i\big)_+},$$
$$\Delta\hat{p}_{i,k+1} - \Delta p_{i,k} = \frac{4\beta_i}{V_i(x_i)}\big(q_i - A_i\dot{x}_i - k_{\ell,i}\,\Delta p_i\big)\,T \;+\; r_{p,i}, \tag{5.7}$$

where β_i, $p_{s,i}$, $k_{q,i}$, $k_{\ell,i}$, A_i and $V_i(x_i)$ are the effective bulk modulus, supply pressure, flow gain, leakage, piston area and chamber volume. Pressure then produces force and motion:

$$\hat{F}_i = \Delta\hat{p}_{i,k+1}A_i + r_{F,i}, \qquad \ddot{x}_i = \frac{\hat{F}_i - c_{v,i}\,\dot{x}_i - \ell_i(\boldsymbol{h}_k)}{m_i},$$
$$\dot{\hat{x}}_{i,k+1} = \dot{x}_{i,k} + \operatorname{sat}\big(T\,\ddot{x}_i + r_{v,i}\big), \qquad \hat{x}_{i,k+1} = x_{i,k} + \tfrac{T}{2}\big(\dot{x}_{i,k} + \dot{\hat{x}}_{i,k+1}\big) + r_{x,i}, \tag{5.8}$$

Here $c_{v,i}$ is viscous friction, m_i is effective modal mass and $\ell_i(\boldsymbol{h}_k)$ is the learned configuration-dependent load. Physical parameters are trained in log-space inside fixed clamps. Several settle on a clamp bound and are therefore fitting coefficients rather than measured constants: modal mass and viscous friction both reach their upper clamp of 10^7 (in kilograms and in newton-seconds per metre respectively), the piston area reaches its lower clamp of $1\,\mathrm{cm}^2$ against a $64\,\mathrm{cm}^2$ initialisation, and the supply pressure separates onto both clamps, with two cylinders at 1 MPa and two at 50 MPa. The same values appear in every checkpoint of this family. Pressure and velocity residuals are bounded to 10 % of their step limits, and the position residual to 1 cm, capping the magnitude each head may add per step.

The explicit chain and the neural heads contribute to the transition in very different proportions. With the residual heads inactive the explicit chain alone produces no net travel, its velocity increments falling six orders of magnitude below their nominal values: at the fitted parameters the friction term $c_{v,i}\dot{x}_i$ reaches 1.0 MN at 0.1 m/s, against a pressure force $\Delta\hat{p}_i A_i$ of about 10 kN at 100 MPa. Realised displacement is produced by the residual heads. The pressure equation nonetheless remains active and responsive to its own parameters: with the heads inactive, halving $k_{\ell,i}$ still moves Δp by 63 % of its range. Hydraulic parameters therefore reach the predicted motion indirectly, through the pressure state entering the recurrent encoder, rather than through the force balance. The model is grey-box in structure—the equations fix the functional form of the transition and keep it differentiable in named hydraulic quantities—rather than an identification of the machine's physical constants.

Training uses rollout horizons $H \in \{1, 5, 20, 50\}$ and scheduled sampling up to 75 % self-fed [Bengio 15]. Residual penalties retain the physical structure. Logged rod force has zero supervised weight because PhysX generates it at deployment and the export fails the piston-area consistency check.

With the true command, mean one-step R^2 is 0.998 for the internal valve, 0.64 for velocity and 0.97 for extension. Zeroing or shuffling the command reduces velocity R^2 below zero and extension to about 0.5; the extension Jacobian is diagonally dominant with condition number 2.14. At 5 s, terminal extension R^2 is 0.998, 0.998, 0.84 and 0.91 across the cylinders. Pressure-increment R^2 is only 0.06–0.09, so the internal pressure is a dynamic state, not a calibrated pressure estimate (Figure 5.5).

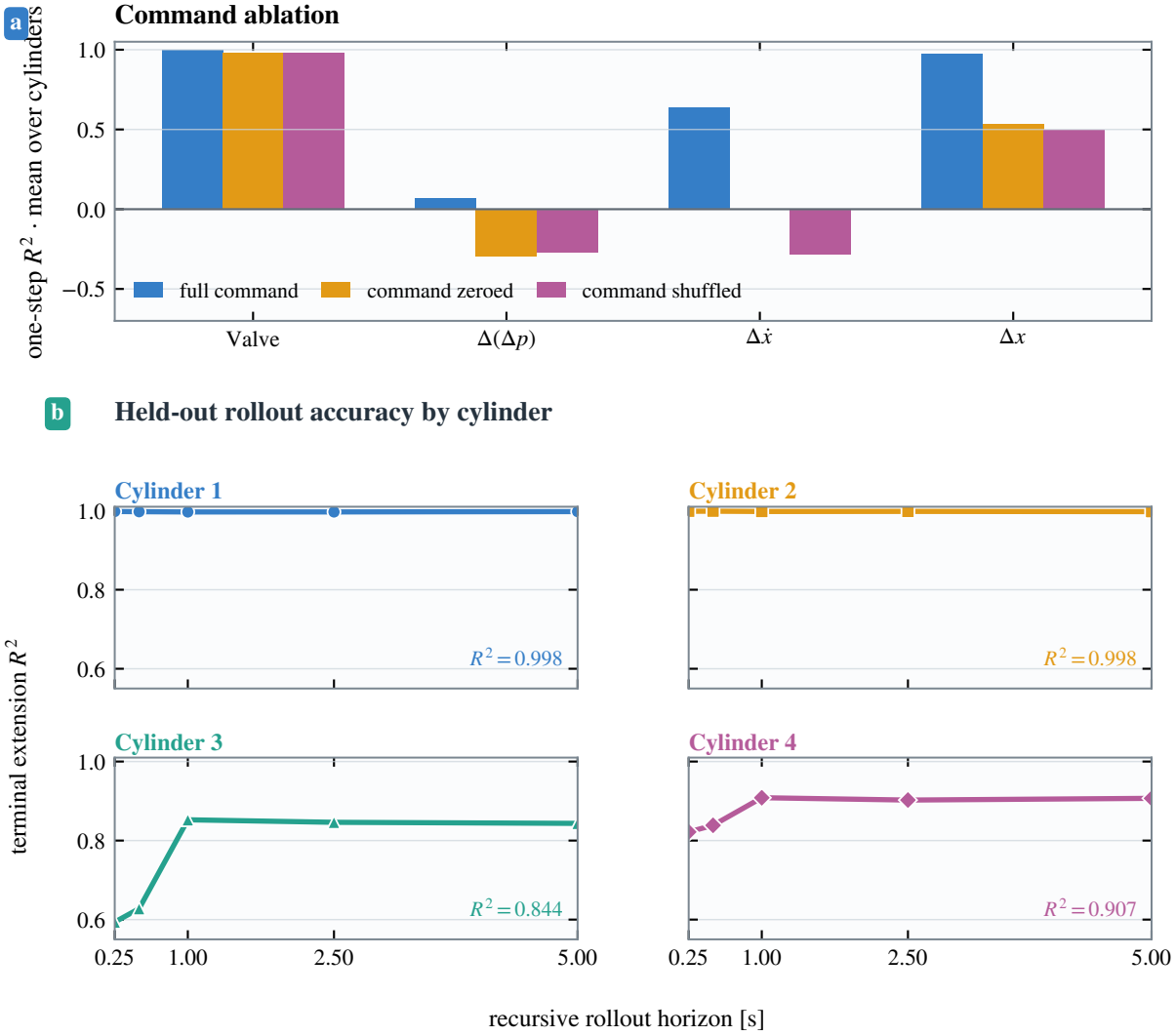


Figure 5.5: Offline validation of the command ROM on held-out runs. (a) One-step R^2 per output group, averaged over the four cylinders, with the full desired command, with the command zeroed, and with the command shuffled across the batch. The collapse of the velocity and extension scores without the true command demonstrates command authority. (b) Terminal extension R^2 of recursive rollouts up to 5 s, shown separately for each cylinder so coincident high-accuracy traces remain visible.

5.4.1 Closed-Loop Calibration Against the Deployment Plant

The Simscape-trained ROM has a systematic extension offset when driving PhysX. The promoted checkpoint adds a bounded, command-gated correction:

$$\hat{x}_{i,k+1} \leftarrow \hat{x}_{i,k+1} + \delta_i(\boldsymbol{h}_k, \boldsymbol{c}_k), \qquad |\delta_i| \le 3\,\mathrm{mm}, \qquad \delta_i\big|_{\boldsymbol{c}_k=\mathbf{0}} = 0, \tag{5.9}$$

It is trained on 4 084 IsaacLab transitions from five reachable reference shapes, with Simscape replay. All hydraulic and integration parameters remain frozen. The gate sets the correction to zero for a commanded hold.

The calibrated command ROM serves as the deployment plant in Section 3.4.1. Unlike the ID–FD path, it has seen PhysX data. It is frozen and has no latent input; adaptation modifies only the inverse path.

5.5 A Predictive Test of Plant Identity

The system-dynamics surrogate is a prediction instrument. It is not an actuator, and it is not an adaptation model: the only adaptation model in this thesis is the AAM of Section 5.6. Its purpose is to settle one question before any adaptive machinery is built, namely whether a compact description of which plant is present improves prediction of that plant at all. If it does not, nothing downstream can recover the difference.

It approximates the command-ROM–PhysX plant and is trained from scratch on the IsaacLab data of Section 4.7. At each 50 ms step it predicts the next 12-dimensional hydraulic-state increment from the observable state, joint context, consecutive commands and a six-tick causal belief, under a five-transition unroll; the matched control SD_0 is identical but has no latent input. A privileged encoder $\mathbf{z}_{\mathrm{SD}} = \mu(\mathbf{e})$ maps the six physical factors to two dimensions and is trained jointly with SD_z, so the latent is whatever serves prediction rather than a quantity fixed in advance.

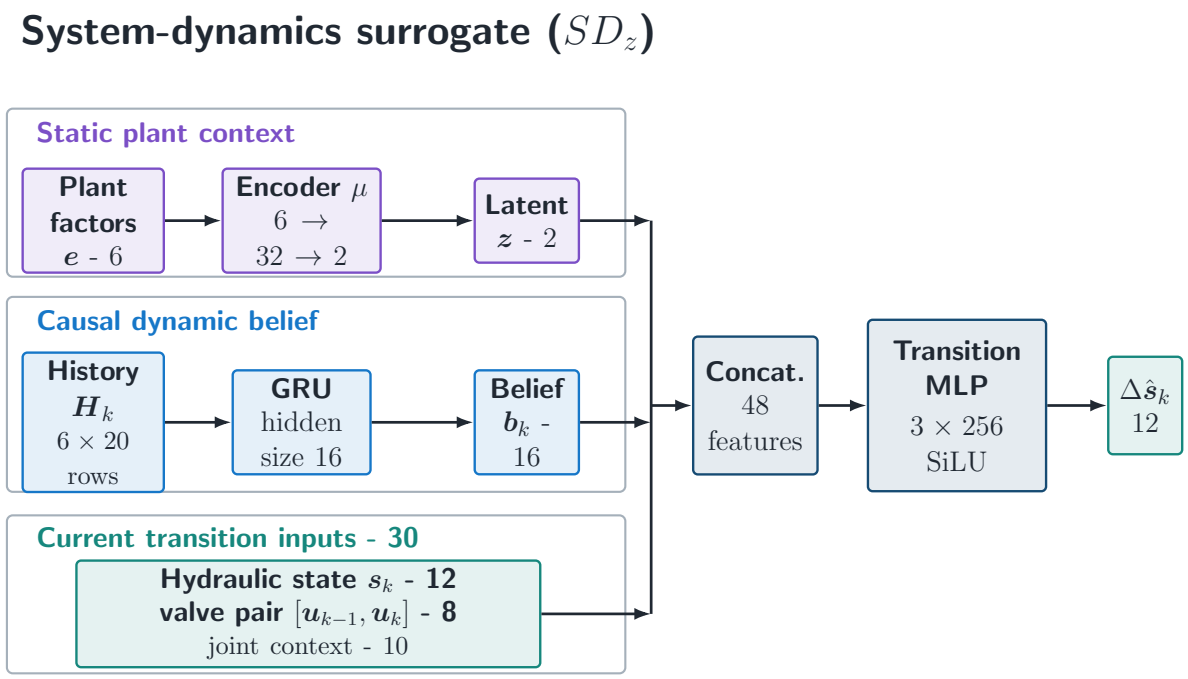


Figure 5.6: System-dynamics surrogate. The transition model combines a two-dimensional static plant context and a 16-dimensional causal dynamic belief with the hydraulic state, consecutive valve commands and joint context. A three-layer, 256-wide MLP predicts the transition; training uses a five-step unroll.

The answer is yes, but narrowly. Pooled rollout error falls by 7.20×10^5 on 20 validation runs (95 % interval $[-1.03 \times 10^6, -4.30 \times 10^5]$) and by 4.41×10^5 on 16 test runs ($[-7.61 \times 10^5, -1.74 \times 10^5]$). The per-channel decomposition places the whole of that gain in pressure: the extension and velocity intervals cross zero on both pools (Table 5.3), and those two are the channels the controller integrates.

Table 5.3: Per-channel decomposition of the paired latent advantage at a 20-tick horizon, pooled over 36 runs. Values are the mean paired difference between the latent-conditioned model and the no-latent control, with a 95 % bootstrap interval; a negative value favours the latent for the pooled state error, and a positive value favours it for the per-channel errors as signed here. The advantage is confined to the pressure channel.

Channel	mean paired difference	95 % interval	excludes zero
extension	5.2×10^{-5}	$[-3.6 \times 10^{-5},\ 1.4 \times 10^{-4}]$	no
velocity	-2.4×10^{-4}	$[-7.1 \times 10^{-4},\ 2.4 \times 10^{-4}]$	no
pressure	1.75×10^{5}	$[2.1 \times 10^{4},\ 3.3 \times 10^{5}]$	yes

Two limits follow and both are carried forward. The test compares separately trained models, so it establishes that the context is informative, not that an estimator can recover it or that a controller can use it; the within-model substitution of Section 7.5.5 isolates that. And the informative channel is pressure, which is the first appearance of the delivery problem Section 5.7.3 closes on. The encoder, SD_z, belief module, normalisers, input mask, split and rollout protocol are then frozen for downstream use.

5.6 Arm Adaptation Module

This section describes the encoder trained offline against a latent target on the frozen collection of Section 4.7. Its purpose here is evidentiary: it establishes that plant identity is recoverable from causal observable history at all, which is the third link in the chain of Chapter 7 and a precondition for anything built on top of it. The encoder actually deployed in AF is trained differently, under the reinforcement objective and without a latent target (Section 6.7.5); what follows should be read as a measurement of what the history contains, not as a description of the deployed component.

The AAM estimates $\mathbf{z}_{\mathrm{AAM}} = \mathrm{AAM}(\boldsymbol{H}_t)$ from deployment history. Each row contains five joint positions, five joint velocities, four cylinder pressure differences and four previous valve commands. A four-layer, width-64 causal convolution processes 20 rows (1.0 s, 360 values).

The pressure values are recursive actuator-model states, not measurements: they come from the command ROM in R and the frozen FD in AF. The factor vector, true latent, reference, future rows and hidden state are excluded. Using the previous valve command keeps each row causal. Validation selected the shortest competitive history from 5, 10, 20 and 40 ticks. Latent RMSE is 0.802 on validation and 0.734 on reserved test.

Inference runs at 20 Hz inside the 60 Hz control loop. The adapter is bypassed until 20 rows are available. A causal five-prediction rolling mean adds 250 ms lag. Results reported for this route use that online smoother, not the earlier non-causal full-run average. The smoother belongs to this encoder and not to AF, whose residual consumes its history window directly; the latency stated here therefore does not apply to the adaptive result of Chapter 7.

5.6.1 Information Content of the Estimated Latent

Linear decoding from the per-run mean estimate gives $R^2 = 0.956$ for seal condition, 0.448 for compliance, 0.114 for flow, 0.081 for supply pressure, 0.056 for centre-of-mass offset and -0.008 for mass. The latent therefore represents mainly seal condition. It is stable within a run but does not refine over time.

Low-dimensional adapters can lose information when factors act through different channels [Liu 25]. This differs from the rapid-motor- adaptation assumption that only task-relevant factor directions need to be retained [Kumar 21]: here the dominant seal-condition direction acts through pressure, which the controller does not integrate. Estimating named physical quantities would improve interpretation but requires an explicit plant model [Hu 26].

5.7 Latent-Conditioned Inverse-Model Adapter

The estimate has to reach the plant through something. The route taken first, and reported in full here, places the correction on the inverse model's output. It is the natural choice while the inverse model is in the command path, and it is the route the closed-loop evidence below finds insufficient; the redesign that follows from that finding is the subject of Section 6.7. Reporting it is not bookkeeping, because the reason it fails is what determines the shape of the replacement.

The frozen ID produces $\boldsymbol{u}_{\text{base}} = \text{ID}(\boldsymbol{s}, \boldsymbol{r})$. The adapter adds a bounded latent-conditioned residual:

$$\boldsymbol{u} = \text{clip}\Big(\boldsymbol{u}_{\text{base}} + g_\psi(\boldsymbol{s}, \boldsymbol{r}, \boldsymbol{u}_{\text{base}}, \mathbf{z}_{\text{AAM}})\Big). \tag{5.10}$$

Training minimises valve-command MSE at the exact ROM tick, with the residual bounded to 0.15. The ID, FD and policy are frozen during adapter training. The policy of this route is separately latent-conditioned, so disabling the residual restores the baseline actuator path but not the baseline actor, and the ablation is single-variable in the actuator only. That residual asymmetry is one of the things the redesign of Section 6.7 removes: under AF the base policy is frozen and unmodified, so disabling the residual recovers the baseline in full.

Figure 5.7 shows the causal convolutional stack and the bounded adapter that consumes its estimate. The upper part is the AAM; the lower part shows where adaptation enters the frozen inverse-model path.

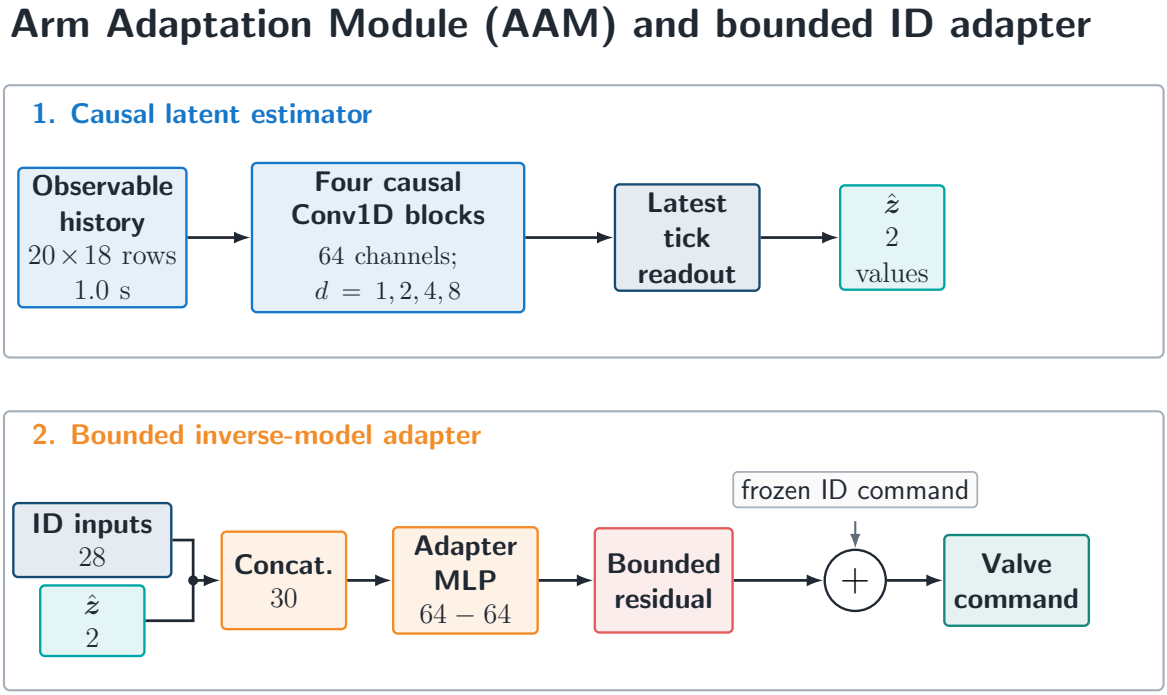


Figure 5.7: AAM and bounded ID adapter. Four causal convolutional blocks map a 20-step history of 18 observable features to the compact latent $\hat{z} \in \mathbb{R}^2$. The adapter combines this estimate with the ID inputs and adds a limited, bounded correction to the frozen valve command.

5.7.1 Open-Loop Evidence

The residual cap is 0.15, and the frozen ID already explains about 84 % of valve-command variance (Section 5.2.2). The adapter can therefore make only small corrections. Section 7.5.5 tests latent information without this ceiling.

Three matched adapter arms use a zeroed, estimated or privileged latent. Across five seeds, both latent arms improve validation loss over the zeroed arm in 5/5 cases. Mean paired improvements are 0.0037 for the privileged latent and 0.0118 for the AAM estimate. The effect is consistent but small.

Masking pressure and force does not strengthen the result: the privileged latent helps in only 2/5 seeds and variance increases by about one order of magnitude. The earlier single-seed improvement is not reproducible.

5.7.2 Closed-Loop Evidence and Its Bound

Closed-loop ablations compare the full stack with zeroed, shuffled, frozen and mean latents, plus a sham plant change. None consistently improves lateral error, cut-height error or impacts.

Valve saturation limits the test: cylinder 1 is saturated for about 92 % of the scored window, cylinder 2 for 67–82 %, and cylinder 4 for 50–57 %. Control-effort ratios before

and after the plant change are 1.005–1.009. A bounded additive correction has little authority under these conditions [Sofrony 24].

Reducing the lead cap lowers cylinder-1 saturation from 0.91 to 0.14 but triples cut-height error. The task is therefore close to the plant capability limit (Section 3.6.2).

5.7.3 Propagation of the Latent Advantage

The chain gate compares $SD_0(\boldsymbol{s}, \boldsymbol{u})$, $SD_z(\boldsymbol{s}, \boldsymbol{u}, \mathbf{z}_{\mathrm{SD}})$ and $SD_z(\boldsymbol{s}, \boldsymbol{u}, \mathbf{z}_{\mathrm{AAM}})$ under the frozen protocol with paired bootstrap intervals. Neither the true nor estimated latent improves extension or velocity over SD_0 with an interval excluding zero. The estimate is not worse than the true latent, so estimation is not the limiting stage. The measurable advantage remains in pressure (Table 5.3).

Section 7.5.5 finds the estimate comparable with the privileged latent within one model, confirming that the AAM is not the main loss. The missing link is a path from pressure information to the integrated extension command with sufficient actuator headroom.

That diagnosis is the pivot of this thesis, and it is worth stating as a conclusion rather than as a loose end. Every stage of the chain up to the command holds: the identity is present, an estimator recovers it causally, the estimate is as informative as a privileged one, and it improves a model that consumes it. What fails is the delivery. The correction is added to a command that is already resting against its own clip for most of the scored window, so the authority to act on the information does not exist at the point where the information arrives. A limit of that shape is not addressed by a better estimator or a larger latent, and Section 5.7.2 shows it is not addressed by reducing the demand either, since lowering the lead cap buys headroom at the cost of tripling cut-height error.

It is addressed by removing the accumulator that consumes the headroom, which is what AF does: the inverse model leaves the command path, the policy emits the valve command itself, and the residual acts on that command (Section 6.7.1). The result of that redesign is reported in Section 7.6.2; Chapter 8 returns to what the sequence implies for adaptation on saturating actuators generally.

5.8 Training and Model-Selection Protocol

Adaptation data are split 82/20/16 into training, validation and reserved test by complete run and factor configuration; windows do not cross run boundaries. The 16 test runs are not used for model or hyperparameter selection. SD_z and SD_0 are selected on validation rollout, the AAM on downstream rollout, and the adapter on validation command error. Table 5.4 records the selected checkpoints.

Table 5.4: Provenance of each model in the modelling chain. Frozen components are loaded unchanged and never receive gradients in this chapter.

Model	Status	Selected checkpoint
Control ID	frozen	paired-control, Section 5.2
Strict FD	frozen	strict-excitation, Section 5.3
Command ROM	frozen	PhysX-calibrated, Section 5.4.1
SD_0 (no latent)	scratch	epoch 58
SD_z + encoder	scratch	epoch 48, frozen after the gate
AAM	scratch	epoch 73, 1.0 s causal CNN
Latent adapter	scratch	epoch 116

The latent adapter of this chapter is trained and selected on recorded valve-command error. This avoids the objective mismatch of an earlier next-state-loss variant. Each component is evaluated on its stated training contract.

One component of the deployed stack is deliberately absent from Table 5.4. The adaptation residual of AF has no supervised checkpoint to record, because it is not trained in this chapter and not trained against a target: its encoder and head are optimised jointly under the reinforcement objective, and their provenance is a training run rather than an epoch number (Section 6.7.5).

With the models identified, audited and frozen, and with the one place the chain breaks located precisely, the next chapter builds the controllers that consume them. It assembles the reference and non-adaptive stacks from the models above unchanged, and then rebuilds the adaptive stack around the finding of Section 5.7.3, removing from its command path the model that leaves no room for a correction to act.

6. Control Framework and Simulation Integration

Chapter 5 left three actuator representations sitting on a bench, each with its own contract and its own offline evidence. This chapter puts a controller in front of them and turns them into three experiments.

Section 6.1 describes the classical controller, because it is where the competence comes from even though it never runs at deployment. Section 6.2 specifies the policy: what it sees, what it emits, what stops it from emitting something dangerous, and what it is paid for. Section 6.3 then explains how the first becomes the second, and reports a result about that transfer which changed the training protocol of this thesis. The individual stacks appear only after those three are fixed, because they differ in one place only.

6.1 The DLS Teacher and Its Two Roles

6.1.1 Scope and Role of the Controller

The classical controller occupies a narrower position in this architecture than its prominence suggests. There is no damped-least-squares layer underneath the policy at runtime. Nothing is added to the policy's output. If the policy emits zero, the arm does not move.

The DLS controller is nevertheless the reason the policies work at all, and it appears twice, both times at training or collection time and never at deployment (Figure 6.1). It is the *privileged teacher* whose behaviour the policies are distilled from (Section 6.3), and it is the closed-loop controller that drove the identification runs of Section 4.7. Calling it a nominal layer would misdescribe the deployed system and, worse, would make every reported result a property of a controller pair rather than of the policy.

6.1.2 Privileged Information

The teacher is *privileged* rather than better tuned: it reads two quantities that no deployable controller on this machine could obtain. The first is the exact terrain height directly under

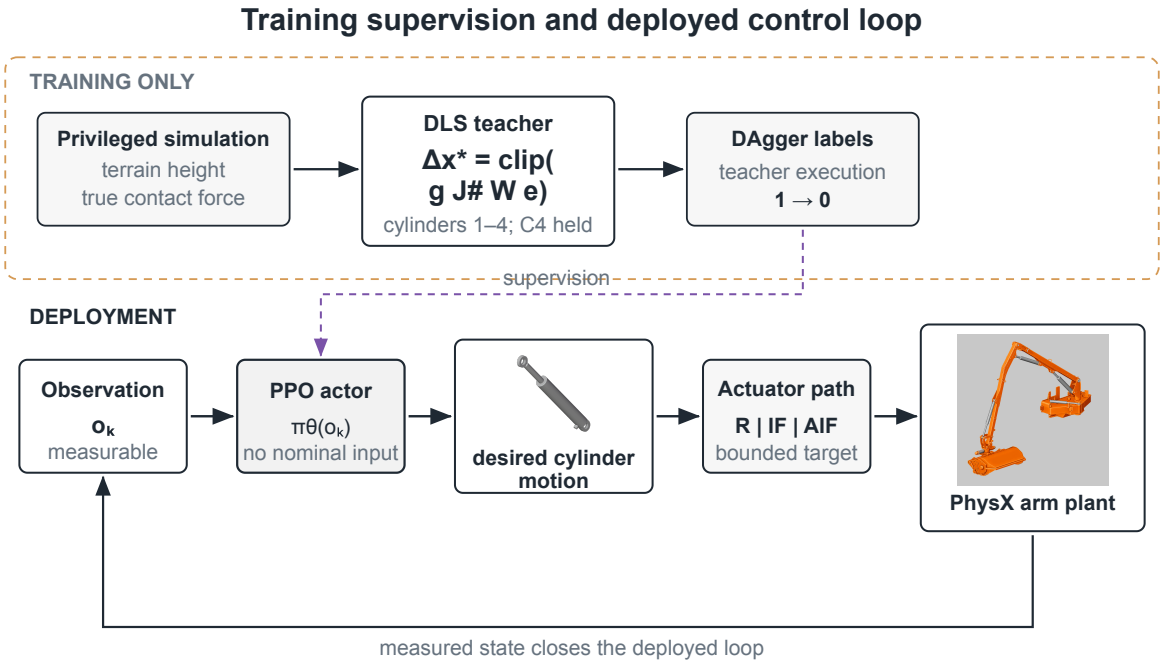


Figure 6.1: Training supervision and deployment. Simulator-only terrain and contact signals drive the DLS teacher, which supplies DAgger labels to the actor. At deployment only the lower feedback loop remains: measured observations pass through the PPO actor, the shared cylinder interface and the selected R, IF or AF actuator path before PhysX updates the arm. No nominal command is added to the policy output.

the cutter, queried from the simulator rather than inferred from a forward- looking scan. The second is the true cutter–ground contact force. Together these turn the hardest part of the task, knowing where the ground actually is, right now, underneath a 1.2 m flail head that is occluding it, into a lookup.

The teacher uses that privilege in a specific way. Rather than commanding a height, it runs a contact regulator: it maintains a vertical offset relative to the ground and adjusts that offset to hold the measured contact force near 250 N within a 100 N deadband, pressing down below the band, backing off proportionally above it, searching downward when contact is lost and retreating immediately on a force spike. The accumulated offset is clamped to $[-0.30, -0.05]$ m so the regulator can neither drive the head into the ground nor lift it clear.

The teacher's advantage is therefore not kinematic. Its inverse is the same DLS solution available to anything; what it has is contact sensing. That is the quantity the student must infer from a noisy forward scan and its own contact history, and it is a large part of why distillation is a nontrivial transfer rather than a formality.

6.1.3 From Task Error to Cylinder Command

The rest of the teacher is conventional and is stated for completeness. A goal point is maintained in the truck frame: laterally it tracks the verge centre line, vertically it follows the measured ground height plus the contact regulator's offset. The task error is the

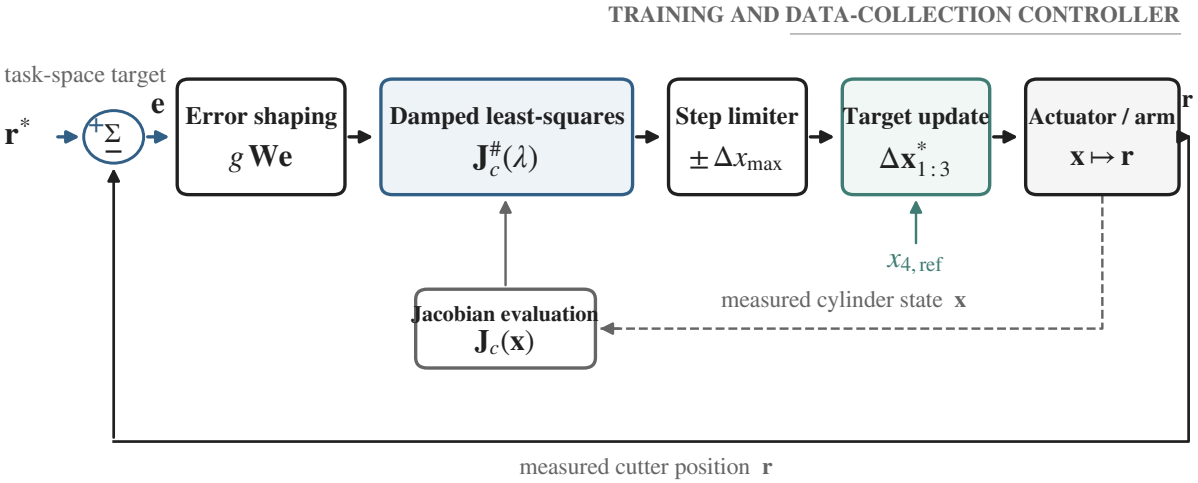


Figure 6.2: Feedback realization of the damped least-squares teacher of (6.1). The weighted task error is scaled by g, mapped to the first three cylinder coordinates by the measured damped inverse $\boldsymbol{J}_c^{\#}(\lambda)$, and clipped to $\pm\Delta x_{\max}$ per tick. Cylinder 4 is held at its head-down reference. Cutter position closes the task-space loop and the measured cylinder state updates $\boldsymbol{J}_c$ at every step. The loop is used only for teaching and data collection, never at deployment.

difference between that goal and the measured cutter position, with the lateral component weighted by a factor of three, because on this machine lateral error is what the acceptance gate measures and vertical error is largely handled by contact.

That error is mapped to cylinder space through the damped least-squares inverse of the measured cutter Jacobian $\boldsymbol{J}_c$ of Section 3.2.2,

$$\begin{aligned} \boldsymbol{J}_c^{\#}(\lambda) &= \boldsymbol{J}_c^{\mathsf{T}}\left(\boldsymbol{J}_c\boldsymbol{J}_c^{\mathsf{T}} + \lambda^2\boldsymbol{I}\right)^{-1}, \\ \Delta\boldsymbol{x}^*_{1:3} &= \mathrm{clip}\Big(g\,\boldsymbol{J}_c^{\#}(\lambda)\,\boldsymbol{W}\boldsymbol{e},\ \pm\Delta x_{\max}\Big), \quad g = 0.30, \quad \Delta x_{\max} = 3\,\mathrm{mm}. \end{aligned} \tag{6.1}$$

with $\boldsymbol{W}$ the lateral weighting and the per-step clamp bounding how far any cylinder may be asked to move in one tick. Only cylinders 1–3 are solved. Cylinder 4 is held at the reference extension that keeps the head down, for the reason given in Section 3.1. The gain g is small on purpose: this is a stiff, heavy mechanism in contact, and a resolved-rate controller that closes quickly on the task error simply excites it.

6.1.4 Execution Through the Student Interface

One implementation detail determines whether distillation converges to something usable. The teacher's output $\Delta\boldsymbol{x}^*$ is a desired cylinder target, and it is *not* written to PhysX directly. It is converted into the same normalized four-value action the student emits and pushed through the student's entire action chain and into the same plant (Section 6.2), so every label the student is asked to imitate is by construction an action the student can produce. A teacher writing targets directly to the plant would generate labels outside the student's reachable set, giving the imitation error a floor no amount of training could remove. That failure mode was measured here: when an earlier action-rate limit sat below the teacher's own structural bound, 0.43 % of labels became unreachable by construction and acted as exactly such a floor.

Table 6.1: The privileged DLS teacher, measured on the stage-6 verge task through the command-ROM plant (128 environments, 480 steps, all eleven path families, seed 17). It is competent, not perfect, and several acceptance thresholds sit deliberately just outside what it achieves.

Quantity	Teacher	Gate
Mean $\lvert e_{\text{cut}} \rvert$	0.0234 m	≤ 0.040 m
p95 $\lvert e_{\text{cut}} \rvert$	0.1524 m	—
Mean $\lvert e_{\text{lat}} \rvert$	0.1221 m	≤ 0.300 m
Ground-contact fraction	0.7807	≥ 0.700
p01 head alignment	0.9638	≥ 0.900
Minimum head alignment	0.8317	—
p95 tool acceleration	64.9 m/s^2	≤ 120 m/s^2

6.1.5 Teacher Performance

A teacher is not an oracle, and quoting its performance is necessary because distillation cannot exceed it and the gates of Section 3.9 were anchored to it. Table 6.1 reports the privileged teacher measured on the stage-6 verge task through the ROM plant, over 128 environments and 480 control steps.

Two readings follow. First, the teacher is a competent but visibly imperfect controller: it loses ground contact about a fifth of the time and its cut-height tail is six times its mean. Second, and this is why Section 3.9 re-derived several thresholds, a gate set above these numbers would be a gate that the ground-truth controller itself cannot pass, which carries no information about the policy.

6.2 Policy Formulation

6.2.1 Policy Definition

The deployed controller is a memoryless Gaussian multilayer perceptron. It maps a 66-value observation to four normalized actions, one desired motion command per hydraulic cylinder, and that is the entire control law. Actor and critic are separate networks with hidden widths 256–128–64 and ELU activations, over empirically normalized observations. This description is of the base policy, and it is shared by all three stacks: the actor width is the same 66 in each, and AF deploys this network frozen and unmodified rather than a widened variant of it. Two things differ for AF and are treated where they arise. Its four actions are interpreted as valve commands rather than as desired cylinder increments (Section 6.7.2), and the deployed controller is this policy plus an additive residual whose own encoder reads a history window carried alongside the observation, leaving the base actor's input unchanged (Section 6.7.3).

Three things this base policy is *not* must be stated, because each was a design option that was considered and rejected. It is not itself a residual: it has no nominal command to correct, and where a residual does appear in AF it is added around this policy rather than built into it, which is what keeps the frozen base recoverable exactly. It is not recurrent: temporal inference is placed in a separate encoder, offline in the module of Chapter 5 and

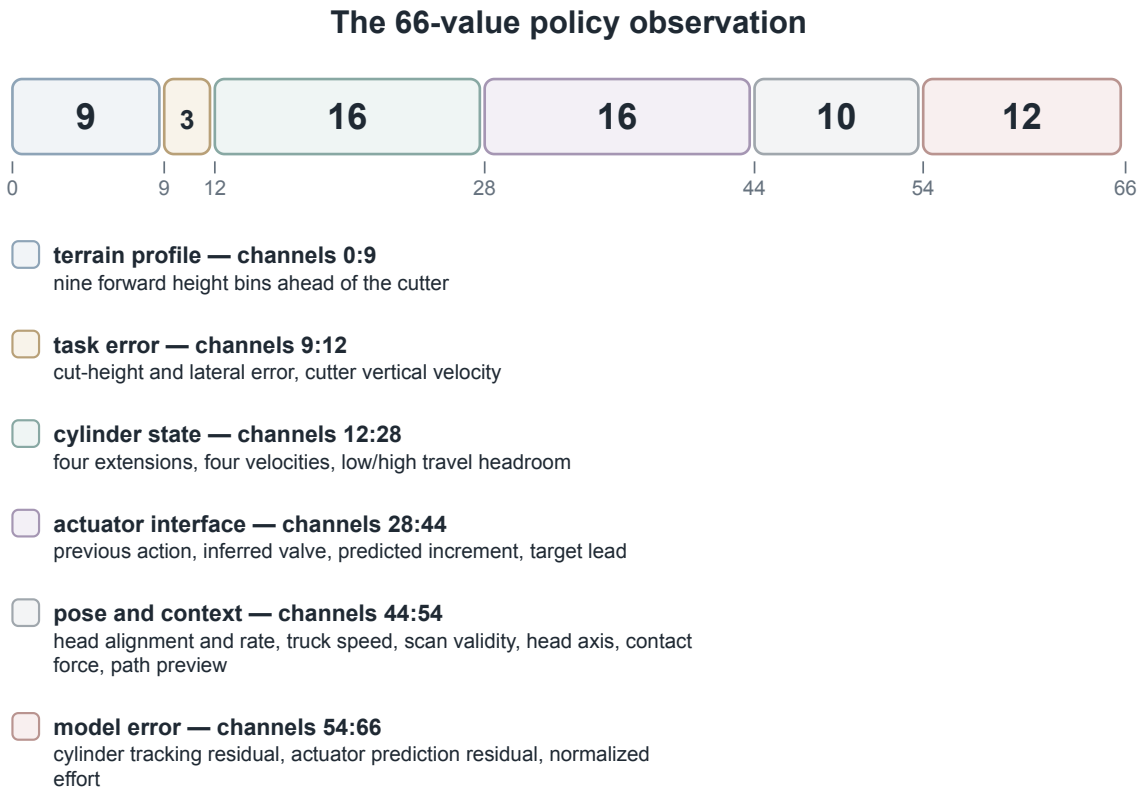


Figure 6.3: The 66-value policy observation, drawn to scale by group. The channels combine measurements, derived quantities and actuator-model estimates. No trajectory-family label, no physical factor vector and no true latent enters the actor. AF appends the two estimated latent channels to this vector, giving 68, and is reported as the different architecture that it is.

jointly trained with the residual in Section 6.7.5, and giving the policy its own memory would confound the two. And it does not command the vehicle: the truck's forward motion is scripted, so the policy cannot solve a tracking problem by driving differently.

6.2.2 The Observation Vector

Figure 6.3 lays out the 66 channels by group. The grouping tells the story of what this controller has to reason about: nine terrain-height samples ahead of the cutter, three task-error channels, sixteen channels of cylinder state and remaining travel, sixteen describing the actuator interface, ten of pose and context, and twelve of model error.

The last group deserves an argument, because twelve of sixty-six channels is a substantial fraction of the observation spent on something that is not the task. Those channels carry the actuator's standing error: the cylinder tracking residual, the actuator prediction residual, and the normalized cylinder effort. They exist as a direct consequence of the policy being memoryless. Forming an error requires differencing position across two steps and comparing against a prediction issued on the previous step, and a feedforward network can construct neither from a single observation, so without these channels the model error the policy is asked to compensate is not merely hard to see but absent from the input.

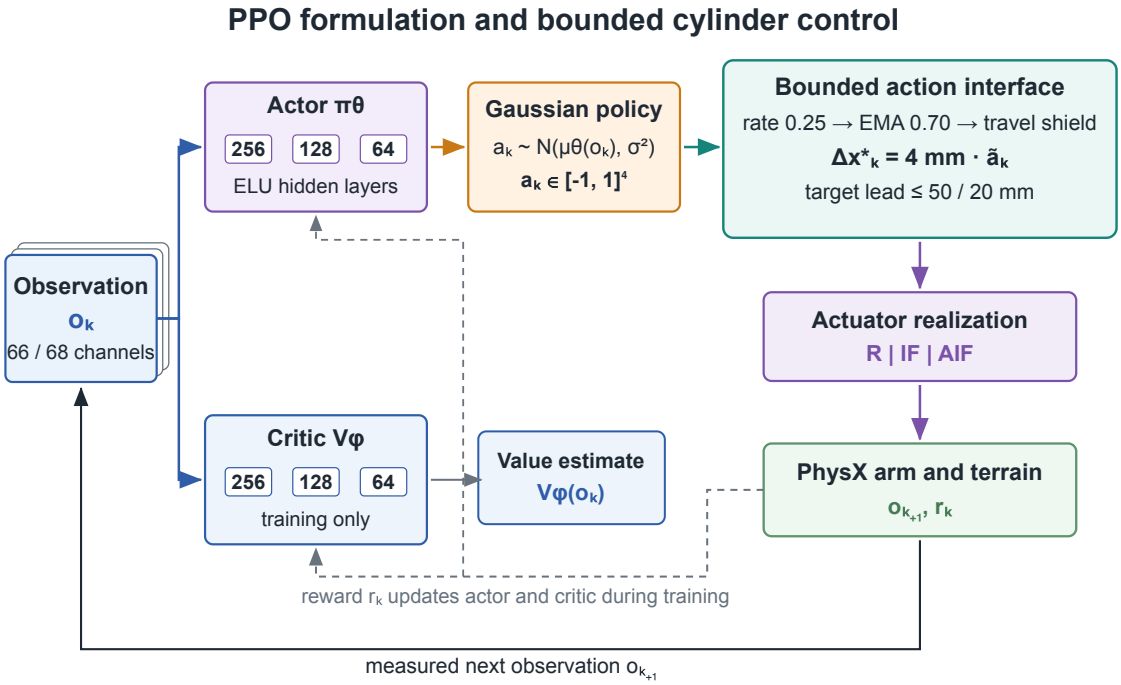


Figure 6.4: PPO actor–critic formulation and deployed control path. The actor maps the observation to a four-dimensional Gaussian action. Rate limiting, exponential filtering, the travel shield, command scaling and the target-lead cap bound that action before the selected R, IF or AF actuator path drives the PhysX arm. The critic and reward update are training-only; measured state closes the deployed loop.

This was measured rather than reasoned about: across ten training blocks the p01 head alignment sat at 0.79–0.80 and did not respond to changes in the feedforward source or the levelling gain, because the policy had no observable evidence of the error it was being asked to correct.

This group is also what makes the matched pair comparable. Under the ROM the tracking residual is near zero, because the ROM reaches what it is told. Under a learned ID–FD path it is the model's standing error. The same observation contract therefore gives each policy an honest view of its own actuator's shortcomings, which is what the comparison of Section 6.9.2 requires. The contract is shared by AF as well, but comparability there rests on the paired construction against its own frozen base rather than on the observation alone.

6.2.3 The Action and Its Bounds

The policy emits $\boldsymbol{a} \in \mathbb{R}^4$, sampled from its Gaussian at training time and taken at the mean for evaluation. Between that sample and the PhysX cylinder target sit five bounding stages. Figure 6.4 places them in the complete PPO actor–critic loop. The critic is used only during training; the actor and bounded action interface form the deployed controller. Each bounding stage can only *reduce* authority and cannot introduce motion the policy did not request.

The rate limit bounds $|\Delta\boldsymbol{a}|$ at 0.25 per control step and is held *constant* across every curriculum stage. It is not a curriculum knob, and the reason is the reachability argument

of Section 6.1: the teacher's own command stream has a structural per-step bound near 0.20, so a student limit below that makes part of the label set unreachable. The exponential filter ($\beta = 0.70$ at the final stage) smooths the command. The limit shield scales the command down, and only down, and only in the direction that consumes headroom, over the last 20 mm of cylinder travel, which is the soft replacement for the teacher's hard clamp into the learned range. The command scale converts the filtered action into 4 mm of cylinder travel per unit action. Finally the target integrator accumulates the increments in reference space, with the integrated target permitted to lead the measured extension by at most 18 mm.

This chain describes the cylinder-increment interpretation of the action, which is the one R and IF use. Under AF the same four-dimensional bounded action is read as a valve command and scaled to span exactly the admissible valve range, so that no further clamp binds at nominal, and the inverse-model anchor and the adapter residual of the earlier design are both held at zero. The bounding stages that act in cylinder space therefore do not have the same meaning on that stack, which is one of the reasons its results are read against its own frozen base rather than across the stacks (Section 6.9.2).

That last cap is an anti-windup device and it is load-bearing. Relaxing it from 18 mm to 50 mm halved the lateral error on the two families demanding a sustained standing offset (`lane_change_hold` from 0.4833 to 0.2310, `boom_sweep` from 0.3745 to 0.1602), showing that the clamp was bounding the standing effort those paths need, while the same relaxation on cylinder 4 drove head alignment from 0.9507 to 0.4636. The final task therefore resolves the trade per axis, and this is the configuration every reported R result was produced under: 50 mm on cylinders 1 to 3 and 20 mm on cylinder 4.

Two properties of this chain follow. It is what replaces the residual band, since exploration is bounded by how much motion a single action can command rather than by proximity to a baseline. And it fixes the plant's capability ceiling: four millimetres of cylinder travel per step is the entire authority budget, which produces the 0.240 m/s lateral rate limit of Section 3.6.2 from which the reference paths are derived.

6.2.4 The Reward Function

The reward of the final R task has ten bounded terms, listed in Table 6.2. Three are positive and describe the task. Seven are penalties and describe how the machine must behave while doing it. All are bounded, and all retain a live gradient beyond the acceptance boundary, so that a policy already inside a gate still has a direction to improve in.

It replaces an inherited formulation carrying 53 weighted reward fields. Isaac Lab's own locomotion task uses eleven. The reduction was not cosmetic. At 53 terms the optimum is not something a designer can reason about, and three separate terms turned out to be working against the task without being noticed for thirteen training campaigns. The three rules below were each adopted after a measured failure.

No Gaussian Shaping

The earlier reward priced tracking as $w \exp[-(e/s)^2]$, which is effectively flat beyond about $2s$. Measured on the lateral term, at an error of 0.50 m the gradient is -2.07 at $s = 0.30$ but -0.004 at $s = 0.15$: a factor of 520 weaker where the failing path families live. Every

shaping term is therefore linear or Huber in the error. Writing $x = |e|/s$, the two shapes used are

$$B(e; s, c) = \min(x,\ c), \qquad H(e; s, c) = \min\Big(c,\ \begin{cases} \frac{1}{2}x^2, & x \le 1 \\ x - \frac{1}{2}, & x > 1 \end{cases}\Big). \tag{6.2}$$

B has the same gradient at 0.5 m of error as at 0.05 m, so a policy far from the reference is still told which way to move; H buys quadratic precision near zero where cut height needs it, without a vanishing far field.

Bounded Terms

A term that can grow without limit can dominate the sum and destabilize the critic. Each contribution is clipped, which gives the reward a published budget: $6 + 12 + 4 = 22$ for the task terms against -24 for the safety terms. A non-finite value is replaced by -25, just below that worst case, so a numerical fault can never outscore legitimate behaviour.

Fixed Sign Convention

The three task terms are non-negative and the seven safety terms non-positive. This is what makes worst-case (CVaR) weighting safe to add later: scaling a non-positive term by a factor ≥ 1 can only lower the return, whereas up-weighting a *positive* reward for the worst environments pays the policy to be bad. Measured in the earlier stack, weighting the positive contact reward four-fold for the worst quarter of environments made a contact quality of 0.49 pay 7.84 against 2.00 for a quality of 0.50, and collapsed worst-family contact within a single training block.

Three terms of the earlier formulation are absent, and the reasons are results rather than preferences. The *action-magnitude* penalty on mean(a^2) was removed outright: a terrain action is a desired cylinder motion rather than a pose offset, so what the term actually charged for was the sustained offset that `lane_change_hold`, `offset_return` and `boom_sweep` structurally require. The policy correctly learned to return to neutral, which is the one thing "hold an offset" forbids. Removing it produced the first training block in a full session that did not degrade, improving the held-out score from 1.2058 to 0.3678. The *head barrier* was replaced, because in its earlier form the head group dwarfed lateral tracking and line following could never compete for the optimizer's attention. And the *clearance barrier* was disabled, since measured contact, cylinder headroom and cut-height error proved more reliable signals of the same condition.

One apparent inconsistency is deliberate. The tool-acceleration term saturates at 300 m/s^2 while the corresponding gate sits at 120 m/s^2, anchored to the measured physical floor of the stage-6 environment near 100 m/s^2. The reward scale is a saturation scale for a bounded penalty rather than a target, so the gate remains the binding criterion and the reward's role is only to keep a gradient against gross excursions. The action-rate term is held mild for a related reason: cylinder 4 owns head pitch, so a rate tax falls hardest on the fast corrections that keep the head down as boom attitude shifts, and under a heavier tax the optimizer chose head drift over paying it.

Every term is logged individually and by name during training. The earlier stack logged an aggregate and a partial subset, with the two largest lateral terms not logged at all, which is

Table 6.2: The ten reward terms of the final R task, with weights and shaping scales. B and H are the bounded shapes of Equation (6.2). The lateral and head terms each combine a coarse and a fine scale, so far-field correction and near-field precision are priced separately.

Term	Weight	Shape and purpose
Cut-height tracking	+6	H, scale 0.04 m, cap 2.5; the primary mowing-quality term
Lateral tracking	+12	B, coarse 1.50 m and fine 0.60 m mixed 0.6/0.4; the primary line-following term
Ground contact	+4	pays for the cutter being in contact
Head alignment	−12	B on the deficit below 0.98, fine scale 0.02 and coarse scale 1.00 mixed evenly
Contact loss	−3	charges directly for loss of contact
Action rate	−1	B on the mean absolute command change, scale 0.25; penalizes chatter
Tool acceleration	−1	B, scale 300 m/s^2; mechanical-wear proxy on the cutter
Contact force	−1	B on force above 30 kN, scale 150 kN
Cylinder headroom	−2	B on encroachment inside a 0.02 m margin, scale 0.10 m
Hard impact	−4	indicator on the impact event

part of why a reward term opposing the task survived thirteen campaigns. Reward weights on this system are measured trades rather than preferences, and several natural-looking choices are actively harmful. That is also why the reward is *not* the result criterion: promotion uses the eight physical gates of Section 3.9, and a policy may raise total reward while failing the thing the reward was written to encourage.

6.2.5 Optimization

Policies are optimized with clipped-surrogate PPO over generalized advantage estimation, in the form of Section 2.7. The maintained configuration uses clip parameter $\epsilon = 0.2$, discount $\gamma = 0.99$, GAE parameter $\lambda = 0.95$, five learning epochs over four minibatches per update, an adaptive learning-rate schedule targeting a KL divergence of 0.02 from a base rate of 3×10^{-4}, gradient-norm clipping at 1.0, and a clipped value loss with coefficient 1.0. The terrain task collects 32 steps per environment per update. At 512 environments that is 16 384 transitions per update, chosen because the inherited 128-step horizon exceeded two minutes per update on the available GPU without adding useful batch diversity.

The final fine-tuning blocks depart from that configuration because they run under the reference anchor of Section 6.4 rather than an adaptive learning-rate schedule: clip $\epsilon = 0.1$, discount $\gamma = 0.97$, three learning epochs, fixed learning rates of 10^{-5} for the actor and 10^{-4} for the critic, entropy coefficient 0.001, an exploration standard deviation fixed at 0.15 and excluded from the gradient, and frozen observation normalizers.

Three of those departures are structural rather than tuning. The discount is reduced because the episode is long in simulation steps but locally controlled, so an action should

be credited for correcting the terrain state it faces rather than for a return hundreds of steps later that the critic cannot resolve. The actor learning rate sits an order of magnitude below the critic's so that the actor is deliberately the slow variable and trust-region budget is not spent on steps driven by a value function that has not yet converged. And the adaptive schedule is dropped because the anchored formulation already constrains the update through an explicit divergence bound; two competing step-size controllers would make neither attributable.

The exploration settings changed for a reason specific to this architecture. A residual policy explores around a controller that already tracks, whereas here a zero action means the arm does not move at all, so the initial standard deviation was raised from 0.05 to 0.30 and the entropy coefficient with it, to let the policy discover the command directions before refining them. On the terrain task, where distillation supplies a competent starting point, both are reduced again and the standard deviation is *held fixed* rather than initialized, for a further reason taken up in Section 6.4. Exact values for any reported run must be read from that run's frozen manifest, because they changed during development.

6.3 Training Protocol and the Distillation Finding

6.3.1 Distillation by DAgger

Policies are initialized by imitation, not by reward. The privileged teacher of Section 6.1 is rolled out in the student's own environment, and the student is fitted to the teacher's raw action by regression, in the DAgger scheme of Ross et al. [Ross 11]: the teacher's probability of *executing* the action, as opposed to merely labelling it, is annealed from one to zero over the first 70 % of iterations, so the later part of training collects states the student itself visits rather than states the teacher would have visited. Labels are accumulated in a replay buffer of 262 144 transitions and refitted over three epochs per iteration in minibatches of 8 192.

Figure 6.5 shows a complete distillation run for the ROM stack, taken from the stored record. Panel (a) is the handover schedule. Panel (b) is the imitation error: the behaviour-cloning loss falls by more than an order of magnitude while the teacher is still driving, reaches its minimum around the handover, and then *rises* once the student is alone. That rise is not a failure. It is the distribution shift DAgger exists to expose: the student is now visiting states the teacher never chose, and the teacher's advice on those states is new information rather than a repetition of what has already been fitted. Panel (c) shows the corresponding closed-loop task metrics. They remain near the teacher's preflight values without matching them.

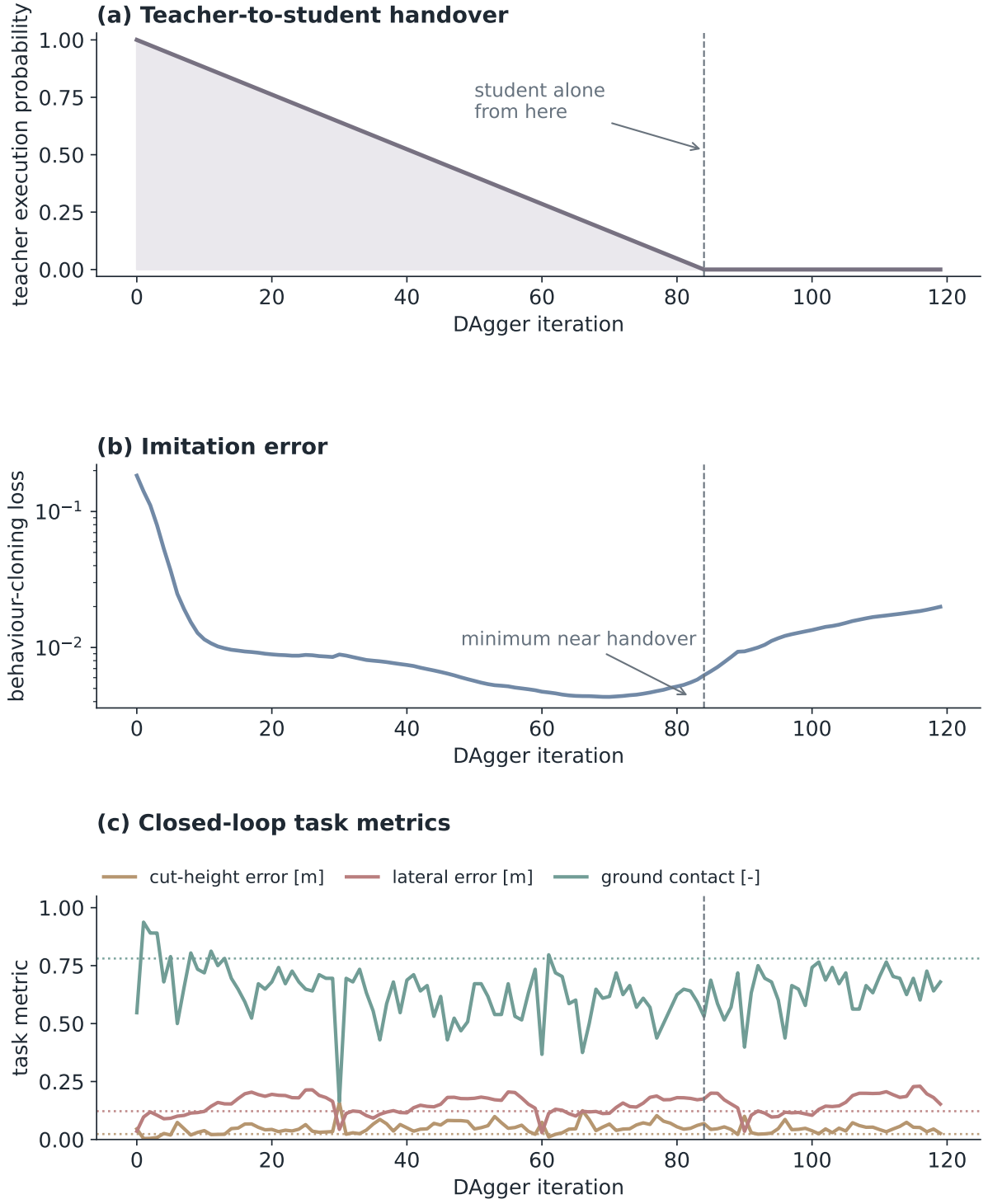


Figure 6.5: A measured distillation run for the R stack, from the stored record. (a) The teacher's execution probability is annealed to zero over the first 70 % of iterations. (b) Behaviour-cloning loss falls by over an order of magnitude under teacher execution, then rises once the student is alone, the distribution shift that DAgger is designed to surface, not a regression. (c) Cut-height error, lateral error and ground-contact fraction; dotted lines mark the teacher's preflight values from Table 6.1.

6.3.2 Objective Mismatch in Unconstrained Fine-Tuning

The natural next step is to treat the distilled actor as an initialization and improve it with PPO. That is what the residual literature does, it is what the plan for this thesis originally assumed, and in unconstrained form, on this task, it does not work. The finding is reported here in full because it is what motivates the anchored formulation of Section 6.4, which is how the final policies are trained.

PPO fine-tuning was run repeatedly from distilled checkpoints, and the outcome was consistent enough to be treated as a property of the problem rather than as bad luck. The critic was healthy throughout, with explained variance in the 0.85–0.95 band, so the failure is not a value-estimation failure. Total reward rose. And the primary tracking objective degraded, in the worst case by a factor of 12.9.

The mechanism follows from placing the reward and the gates side by side. The reward pays for ground contact and cut height as well as lateral tracking, and the optimizer discovers that it can buy the first two by spending the third. That trade increases total reward and moves the policy away from the gate that decides acceptance. The reward's optimum is simply not where the tracking gate is. Four earlier fine-tuning campaigns on a related configuration failed the same way in a different currency: every block degraded head alignment, with p01 falling from 0.9854 to the 0.48–0.78 range while safety terminations rose from 1.3 % to 3.6–7.7 %.

This is a claim about two functionals over the same policy space rather than about the optimizer. Training maximizes the expected return $J(\vartheta)$, while acceptance is decided by the vector of physical gates $\boldsymbol{G}(\vartheta) \preceq \boldsymbol{g}^\star$ of Section 3.9. Writing $\mathcal{F} = \{\vartheta : \boldsymbol{G}(\vartheta) \preceq \boldsymbol{g}^\star\}$ for the feasible set, the measured content of the finding is

$$\arg\max_{\vartheta} J(\vartheta) \notin \mathcal{F}, \qquad \text{while} \qquad \vartheta_{\text{distilled}} \in \mathcal{F}. \tag{6.3}$$

Distillation lands inside $\mathcal{F}$ because the teacher was constructed to satisfy the gates, not to maximize J. Unconstrained ascent on J from that point therefore leaves $\mathcal{F}$ by construction. This is the optimizer working correctly on the wrong functional, and it is why the remedy is a constraint on the ascent rather than a repair of the reward.

Two consequences follow, and both are protocol rather than interpretation. Unconstrained fine-tuning is inadmissible, and both the pre- and post-fine-tune checkpoints are reported in every case; the matching rules that this imposes on the three-stack comparison are stated in Section 6.9. And reward decomposition, critic explained variance, policy entropy and the gate metrics themselves are logged *during* training rather than only at the end, because the whole point is that total reward went up while the system got worse. A training curve reporting only the scalar reward would have shown a success.

The general form of the result applies beyond this machine. When a competent teacher exists and the acceptance criterion is not the reward, reward-based fine-tuning is not automatically an improvement step: PPO optimizes the written objective, that objective is a proxy, and the teacher was constructed to do the task. Distillation transfers the intent; PPO then optimizes the proxy, and here the two point in measurably different directions. Since that divergence grows with distance travelled from a policy where proxy and intent were measured to agree, the remedy is not to abandon reward optimization but to bound the distance explicitly, which is the subject of the next section.

6.4 Reference-Anchored PPO

Equation (6.3) says that reward ascent leaves the feasible set. It does not say that reward ascent is useless, only that it is unbounded. The construction of this section keeps PPO and constrains how far it may travel from a policy already known to satisfy the gates. Writing π_{ref} for a frozen reference policy and J for the expected return, each training block solves

$$\max_{\vartheta} J(\vartheta) \quad \text{s.t.} \quad \mathbb{E}_{\boldsymbol{o}\sim d_{\pi_\vartheta}}\Big[\mathrm{KL}\Big(\pi_{\text{ref}}(\cdot\,|\,\boldsymbol{o})\,\|\,\pi_\vartheta(\cdot\,|\,\boldsymbol{o})\Big)\Big] \le \delta_{\text{ref}}, \tag{6.4}$$

with the expectation taken over the states the policy itself visits.

6.4.1 Critic Warm-Up

A resumed checkpoint carries an actor ϑ and a critic ψ. After distillation the actor already behaves sensibly, but the critic has never been fitted to the long terrain return, so the first PPO updates would move a useful actor using a value estimate that is not yet meaningful. The actor is therefore frozen for 150 iterations while only the value loss is optimized, under the asserted invariant $\vartheta_{\text{after}} = \vartheta_{\text{before}}$. The observation normalizers are frozen at the same point: without that, the same raw observation is normalized differently from one iteration to the next, and the actor's behaviour changes while its weights do not.

6.4.2 The Reference Anchor

Let π_{ref} be a frozen copy of the accepted warm actor. It is queried only on states the live policy visits and never supplies an executed action, so PPO remains the sole controller of the plant. The penalty is the *forward* divergence, which for diagonal Gaussians is available in closed form,

$$D_{\text{ref}} = \mathrm{KL}\Big(\pi_{\text{ref}}\,\|\,\pi_\vartheta\Big) = \sum_i \left[\log\frac{\sigma_{\vartheta,i}}{\sigma_{\text{ref},i}} + \frac{\sigma_{\text{ref},i}^2 + (\mu_{\text{ref},i} - \mu_{\vartheta,i})^2}{2\sigma_{\vartheta,i}^2} - \frac{1}{2}\right], \tag{6.5}$$

at the cost of one additional forward pass and no sampling. Because both scales are fixed at $\sigma = 0.15$, this collapses to a pure quadratic on the drift of the commanded action,

$$D_{\text{ref}} = \sum_i \frac{(\mu_{\text{ref},i} - \mu_{\vartheta,i})^2}{2(0.15)^2}, \tag{6.6}$$

which is the second reason for fixing σ: with a free standard deviation, entropy collapse shrinks D_{ref} while the commanded action moves, so the constraint would report compliance during the drift it exists to prevent.

The direction is load-bearing. Forward KL is mass-covering: it charges π_ϑ wherever π_{ref} has density and π_ϑ does not, and therefore penalizes the *abandonment* of reference behaviour. The reverse direction is mode-seeking and would permit the policy to collapse onto a subset of reference behaviour at no cost, which is the observed failure, in which one competence was dropped to buy another. It follows that the term is a behaviour-cloning anchor evaluated on the policy's own state distribution, rather than the reverse-KL regularizer of soft reinforcement learning. Its lineage is demonstration-regularized policy gradient [Rajeswaran 18], not the Boltzmann-optimum family.

The actor objective is

$$\mathcal{L}(\vartheta) = \mathcal{L}_{\text{PPO}} + \mathcal{L}_V - \beta\,\mathcal{H}(\pi_\vartheta) + \lambda_{\text{ref}}\,\mathbb{E}_k\Big[\mathrm{KL}(\pi_{\text{ref}}\|\pi_\vartheta)\Big], \qquad \lambda_{\text{ref}} = 0.10. \tag{6.7}$$

6.4.3 Trust Regions and Rollback

A penalty is not a constraint: at a fixed coefficient, a sufficiently large advantage always buys its way past. Feasibility is therefore also verified on the update taken. Two bounds are imposed,

$$D_{\text{update}} = \mathrm{KL}(\pi_{\text{old}} \| \pi_\vartheta) \leq 0.01, \qquad D_{\text{ref}} \leq 0.05, \tag{6.8}$$

and they do different work. D_{update} is the classical step-size control that keeps the surrogate a valid local model of J. D_{ref} is the new object: because the anchor is *fixed* within a block rather than moving with the policy, it bounds cumulative drift. A sequence of updates each individually satisfying D_{update} can still walk arbitrarily far from feasibility, and that is the trajectory the earlier campaigns took, monotonically, with the reward rising the whole way. Only a constraint referred to a fixed feasible point can observe it.

Enforcement is in two stages. During minibatch optimization the epoch is early-stopped as soon as D_{update} exceeds its bound. After the update both divergences are recomputed over the full rollout. If either bound fails, or any quantity is non-finite, the actor and its optimizer state are restored *atomically*. On a divergence rejection the critic fit is retained when finite, since the bounds constrain the policy and not the value function, and rewinding valid critic fits computed on the same on-policy rollout was wasting most batches near the boundary. On a non-finite update both networks are restored. In the final block, five of one hundred logged updates were rejected, all of them on the reference bound and none on the update bound. That is, the drift caught was of the kind that ordinary PPO clipping does not see.

6.4.4 The Block Ratchet

The reference is not fixed forever. Within a block it is frozen. When a checkpoint is scored and passes the gates, it is promoted to reference,

$$\vartheta_{\text{ref}}^{(m+1)} = \vartheta^\star, \qquad \vartheta^\star \in \mathcal{F}, \quad \vartheta^\star \in B_{\delta_{\text{ref}}}\big(\vartheta_{\text{ref}}^{(m)}\big), \tag{6.9}$$

and the actor's optimizer moments are reset, because momentum accumulated relative to the old anchor is not meaningful relative to the new one. The critic's are retained: it is fitting the same return, and its calibration is expensive to rebuild.

This is the structural point of the construction. The policy is at all times within a bounded divergence of a checkpoint that has been verified against $\boldsymbol{G}$, and the anchor advances only through verified checkpoints. The acceptance criterion therefore enters the training loop instead of appearing only at the end. The optimizer still ascends the proxy. It simply can never travel far from the last point at which proxy and intent were measured to agree. In this construction distillation is no longer the endpoint: it is $\vartheta_{\text{ref}}^{(0)}$.

Algorithm 6.1 summarizes the procedure in five stages. The first two produce the initial verified anchor. Stages 3 and 4 repeat for each PPO block, and Stage 5 returns the last promoted checkpoint.

6.4.5 Theoretical Status

The components are standard and citable. The bound on D_{update} is conservative policy iteration and its trust-region descendants [Kakade 02, Schulman 15]; the atomic rollback

Algorithm 6.1: Five-stage reference-anchored PPO procedure. The teacher is used only in Stage 1. All PPO rollouts are controlled by π_ϑ alone, and the frozen reference moves only when a checkpoint passes every gate.

input : teacher π_T; gate test $\boldsymbol{G}(\vartheta) \preceq \boldsymbol{g}^\star$; M PPO blocks; anchored loss and divergence bounds from Equations (6.7) and (6.8)
output : last promoted actor $\vartheta^\star$ satisfying the gate test

`// Stage 1 - Distil a feasible actor`
$\mathcal{D} \leftarrow$ DAgger states labelled by teacher mean action $\boldsymbol{y}^T$
$\vartheta \leftarrow \arg\min_\vartheta \mathbb{E}_{(\boldsymbol{o},\boldsymbol{y}^T)\sim\mathcal{D}}\big[\|\mu_\vartheta(\boldsymbol{o}) - \boldsymbol{y}^T\|_2^2\big]$
remove π_T from the controller `// all later rollouts are teacher-free`
`// Stage 2 - Warm the critic and set the first anchor`
freeze ϑ and the observation normalizers
for $j \leftarrow 1$ **to** 150 **do**
  roll out π_ϑ and set $\psi \leftarrow \psi - \eta_{\text{critic}} \nabla_\psi \mathcal{L}_V(\psi)$
end
verify $\vartheta_{\text{after}} = \vartheta_{\text{before}}$
$\vartheta_{\text{ref}} \leftarrow \vartheta$; $\vartheta^\star \leftarrow \vartheta$ `// initial verified anchor`
for *block* $m \leftarrow 0$ **to** $M - 1$ **do**
  `// Stage 3 - Train inside the frozen reference bound`
  freeze ϑ_{ref} for the whole block
  for *each PPO iteration in block* m **do**
    collect an on-policy rollout with π_ϑ as sole controller
    compute and normalize GAE; save snapshot $\mathcal{S}$; set $\vartheta_{\text{old}} \leftarrow \vartheta$
    for 3 *epochs over the rollout minibatches* **do**
      if $\mathrm{KL}(\pi_{\text{old}} \| \pi_\vartheta) > \delta_{\text{upd}}$ **then** stop the update epochs
      update actor and critic using Equation (6.7)
    end
    compute $(D_{\text{upd}}, D_{\text{ref}})$ on the full rollout `// full rollout`
    if D_{upd} *or* D_{ref} *is non-finite* **then**
      restore the complete snapshot $\mathcal{S}$
    else if $D_{\text{upd}} > \delta_{\text{upd}}$ ***or*** $D_{\text{ref}} > \delta_{\text{ref}}$ **then**
      restore the actor and its optimizer moments from $\mathcal{S}$; retain the new critic fit
  end
  `// Stage 4 - Evaluate the block and promote a passing checkpoint`
  score every saved checkpoint on at least three held-out terrain meshes
  aggregate each gate to the worst mesh and, for tracking gates, the worst path family
  $\mathcal{C} \leftarrow \{\vartheta_c : \boldsymbol{G}(\vartheta_c) \preceq \boldsymbol{g}^\star\}$
  if $\mathcal{C} \neq \varnothing$ **then**
    choose $\vartheta^\star \in \mathcal{C}$ using the fixed checkpoint selection rule
    load $\vartheta \leftarrow \vartheta^\star$ and set $\vartheta_{\text{ref}} \leftarrow \vartheta^\star$
    reset the actor optimizer moments; retain the critic state
  else
    leave ϑ_{ref} unchanged; do not promote the block
  end
end
`// Stage 5 - Return the deployable policy`
return $\vartheta^\star = \vartheta_{\text{ref}}$

is that literature's backtracking line search reduced to a single trial point; the penalty in Equation (6.7) is a Lagrangian relaxation of the constrained problem (6.4) evaluated at a fixed multiplier, with the rollback supplying the feasibility enforcement a fixed multiplier cannot.

What does not follow is a certificate. Bounding the divergence from a verified policy does bound gate degradation in form: the chain rule of KL over trajectories, Pinsker's inequality and the boundedness of every gate metric give $|\mathbb{E}_{\pi_\vartheta}[g] - \mathbb{E}_{\pi_{\text{ref}}}[g]| \leq 2G_{\max}\sqrt{H\delta_{\text{ref}}/2}$ for a metric g with $\|g\|_\infty \leq G_{\max}$ over a horizon H, a bound available only because the reward of Table 6.2 is bounded term by term and one that an exponential term would destroy. At the production horizon of 2808 steps and $\delta_{\text{ref}} = 0.05$, however, the right-hand side exceeds eight, which for a metric bounded in $[0, 1]$ is vacuous. There is also a measure mismatch, since the constraint is taken under the live policy's state distribution while the chain rule requires the reference's.

The honest claim is therefore narrower. The construction is a principled restriction of reward ascent to a neighbourhood of a policy where proxy and acceptance criterion were measured to agree, assembled from standard trust-region and demonstration-regularized results, with feasibility re-established empirically at every block boundary. The gates, not the theory, certify each promotion.

6.5 R: Reference Actuator Baseline

R is the reference condition of this thesis. The policy emits desired cylinder motion, the frozen command ROM converts it to a cylinder position target, and PhysX closes the loop. There is no online adaptation anywhere in the stack. The truck is scripted over a 26 m strip at one constant speed drawn per episode from 2 to 7 km/h, the eleven path families of Section 3.6 are distributed across environments within a single run, and the terrain mesh is drawn per environment so that no policy can memorize one ground surface.

Its status is that of a reference condition rather than a competitor, as developed in Chapter 7: the command ROM is not a pure Simscape artefact but a Simscape-trained model with a bounded closed-loop calibration against IsaacLab and PhysX, and ROM-plus-PhysX *is* the deployment plant. R's actuator therefore coincides with the plant. Its tracking advantage over IF is consequently not evidence that a reduced order model approximates the actuator better than an identified ID/FD pair: one of the two defines the target.

The policy is produced by the three-layer construction of this chapter: distillation from the privileged DLS teacher to a feasible student, critic warm-up with the actor frozen, and reference-anchored PPO in blocks under Equations (6.7) and (6.8). The final block produced checkpoints 280 to 329. The last iterate was not taken, because PPO can raise mean return while damaging a single path family. Candidates were scored instead on held-out meshes against the eight gates, aggregated to the worst mesh and, for the tracking-critical gates, the worst path family.

The gate table for that selected checkpoint is deliberately not printed here, and the reason is a limitation rather than an omission. The numbers on record were measured on the superseded 16 m rollout, and the checkpoint's own selection record marks them provisional for exactly that reason; that checkpoint's own full-length six-mesh revalidation was never

run. Quoting a certified gate result for R on the strength of a shorter rollout would be the kind of claim this thesis is written to avoid, so R is described here as a reference *condition* whose acceptance evidence is pending, and Chapter 7 reports the revalidated figures for all three stacks under one protocol.

Three separate records of R acceptance exist in the project, and because they disagree it is worth stating plainly which one this thesis relies on. The pipeline's own documentation reports all eight gates passing across the 2 to 7 km/h range on three meshes, which is a selection-time record for a different checkpoint than the one described above. An earlier six-mesh evaluation of the rebuilt pipeline reports seven gates passing and *ground contact failing*, at 0.6486 against the 0.700 threshold and a margin of −0.0514, which was at the time the one genuine performance gap in the stack. Neither record certifies the checkpoint this chapter selects, and neither is the evidence used in Chapter 7: the closed-loop comparison there re-scores a single checkpoint per stack under one protocol, at one speed, over the full path, on the same meshes and the same eleven families. The earlier ground-contact failure is still the right expectation to carry into that chapter. It is measured on a worst-mesh, worst-family aggregation, which is the statistic that the pooled per-cell means of Table 7.3 do not report, and it identifies ground contact on the ROM plant as the gate most likely to bind, a prediction Chapter 7 is able to check rather than assume.

6.6 IF: Non-Adaptive Learned Baseline

IF subclasses the R task and changes exactly one thing. The policy still emits desired cylinder motion, but that command is now realised by the frozen inverse model of Section 5.2, which converts it to a valve command, followed by the frozen strict forward model of Section 5.3, which converts the valve command to a predicted extension increment. PhysX receives the same bounded cylinder target as before. Task, terrain families, meshes, observation contract, action chain, reward, gates and the PPO update are inherited unchanged, so the actuator representation is the only free variable between the two stacks.

The stack matters to the argument twice over. It is the honest control for R, whose actuator coincides with the deployment plant, and it is the stack in which the inverse model is carried in the command path and is adequate there. It is not the reference against which AF is measured: the two do not share the action contract, and the adaptive claim is made against that stack's own frozen base instead (Section 6.9.2).

Robust is used here in the sense of Section 2.6: the stack was trained across the randomised plant family and applies one fixed response to whichever member of that family it meets, performing no online estimation of plant characteristics and conditioning on no plant identity. It does not mean the policy is blind to plant variation. The twelve model-error channels of Section 6.2 carry the actuator's standing residual, and that residual is itself plant-dependent: under a given plant the actuator misses its commanded increment by a characteristic amount, and the policy acts on that miss. What the stack lacks is not information about the consequences of plant variation but any representation of *which* plant produced them.

Two properties of the composed path shape its behaviour, and both were established offline. The valve command is accumulated rather than issued absolutely and is clipped to

the per-axis limits $[0.95, 0.95, 0.80, 0.80]$, so the actuator path carries its own saturation boundary independently of the action chain. And only the predicted extension increment is integrated, since the forward model's velocity output failed validation on two axes (Section 5.3) and is carried as a diagnostic. This stack's standing model error is therefore visible to the policy through the twelve model-error channels of Section 6.2, which is what makes the shared observation contract fair across actuators rather than merely identical.

The promoted policy's run manifest records all eight gates passing across the 2 to 7 km/h speed range on three held-out meshes. That is the selection-time evidence on which the checkpoint was promoted, not a certified result: it was measured on three meshes rather than the six the final claim uses, and it is superseded by the single-protocol revalidation of Chapter 7, where the per-stack tracking numbers belong. One protocol point belongs here: the reported policy was obtained under the reference-anchored construction of Section 6.4 with a lower actor learning rate than R and a fixed exploration standard deviation of 0.05, which is read from the run manifest rather than assumed to match.

6.7 AF: The Adaptive Stack

AF places online adaptation on the forward model alone. The policy emits the valve command directly, the frozen strict forward model of Section 5.3 converts that command to a predicted extension increment, and PhysX receives the same bounded cylinder target as the other two stacks:

$$\pi_{\text{frozen}} + \text{history residual} \longrightarrow \boldsymbol{u} \longrightarrow f_{\text{FD}} \longrightarrow \text{PhysX}. \tag{6.10}$$

The inverse model is absent from the command path. This is the one structural difference from IF, and it is deliberate: the name of the stack records the two models that remain, AAM and FD. Task, terrain families, meshes, observation contract, reward, gates and the PPO update are inherited unchanged, so the actuator representation and the adaptation path are the only free variables against the baselines.

That the inverse model is genuinely out of the loop is a verified property rather than a claim about intent. Under the direct-valve configuration the adaptation-site instrumentation reports zero valve elements attributable to the inverse model and zero adapter applications at the inverse-model site across a complete run. The actuator key recorded inside the checkpoints still reads `id_aam_fd`; it is retained so that previously trained checkpoints remain loadable and is a legacy identifier, not a description of the runtime path. The reported counters are `valve_elements` $= 0$, `adapter_applied_env_ticks` $= 0$ and `online_aam_residual_scale` $= 0.0$; were the inverse model in the path, all three would be non-zero.

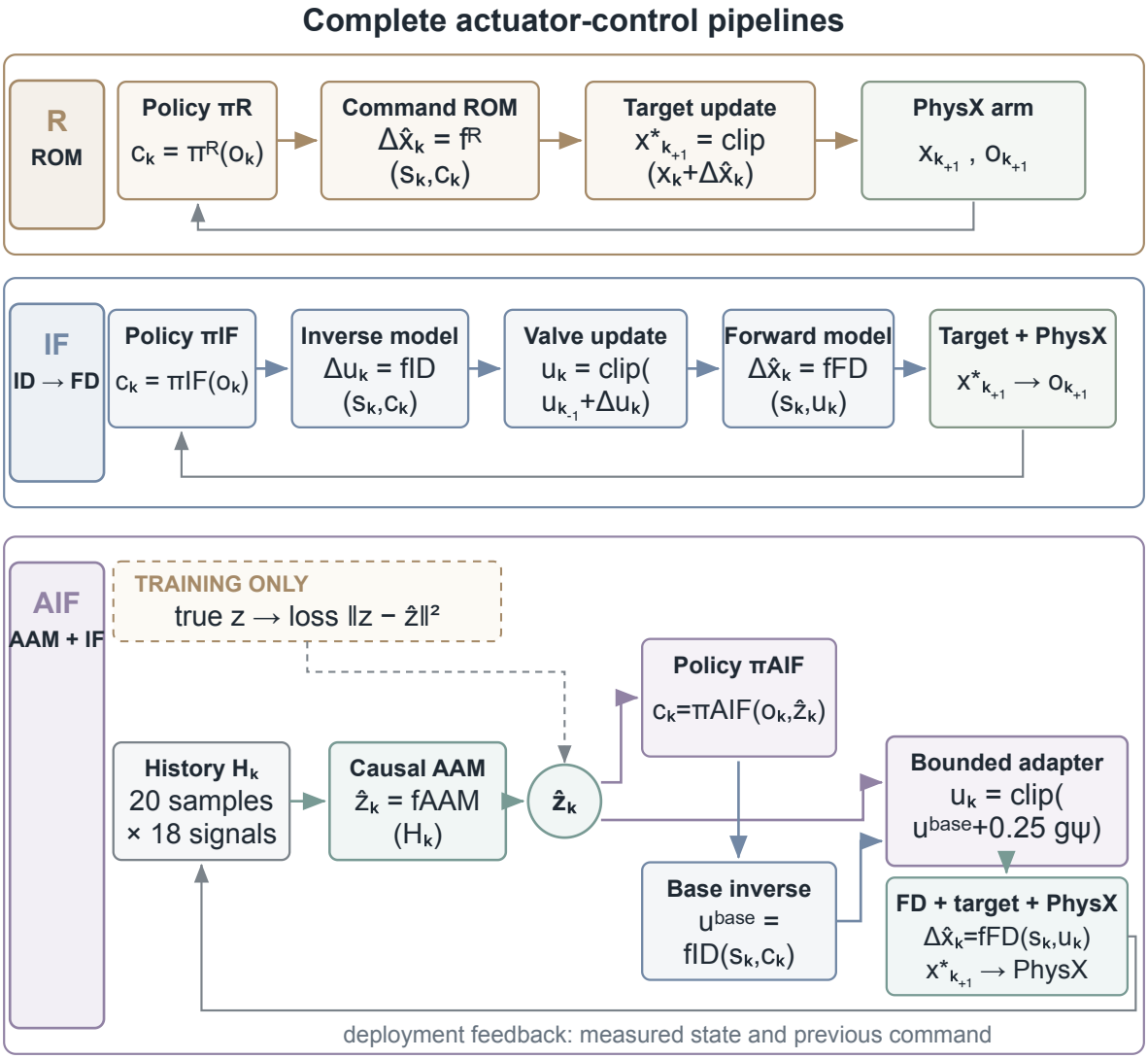


Figure 6.6: Complete R, IF and AF control pipelines. R maps policy commands through the command ROM. IF composes the inverse and strict forward models. AF adds a causal history encoder whose estimate $\hat{z}$ conditions both the actor and the bounded inverse-model adapter. The true latent is a simulation-only training target and is absent from deployment. All three paths produce the same bounded cylinder target for the PhysX arm.

6.7.1 Why the Inverse Model Is Omitted

The three stacks are not three attempts at one thing, and the inverse model's place in two of them follows from what each is for. The command ROM together with PhysX *is* the deployment plant (Section 5.4.1), and it is the only actuator representation calibrated against that plant, so R uses the plant itself and fixes the reference condition. The inverse and forward models are identified independently of it, from the high-fidelity Simscape data of Chapter 4, and neither has seen the deployment plant during training. IF therefore measures something specific: the cost of substituting an independently identified pair for the plant itself.

That question requires the inverse model. A policy that commands desired cylinder motion needs a map from that intent to a valve command, and supplying that map from identification data alone is exactly the substitution being priced. Composing the inverse model with the strict forward model is thus the right construction for the non-adaptive stack, and the R–IF gap is interpretable only because the composition is complete. The inverse model earns its place in IF on that reasoning and keeps it; nothing established below withdraws it.

The adaptive stack asks a different question, and the inverse model does not survive the change. It was not discarded on suspicion: it was adopted only after it had been shown to be trustworthy, and it was disqualified later for a reason that the trust analysis itself uncovered.

Three findings established the trust. Per-valve supervised validation reached $R^2 = 0.9973$–0.9996. Paired control fine-tuning (Section 5.2.1) repaired the one property the supervised score could not certify: the composite normalised control error fell from 1.0011, which is exactly the zero-response baseline, to 0.1086, with all per-axis regression slopes above 0.8 and all correlations above 0.92. The frozen checkpoint then transferred to the 118-run deployment collection, which lies outside its Simscape training distribution (Section 5.2.2), with teacher-forced one-step extension error of 2.36 mm on the combined domain against 3.36 mm on nominal, and a held-out group no worse than the domains used in training. That is the regime in which the model is actually used, because the policy closes the loop at every tick. On that evidence the inverse model was adequate for IF, and IF was built on it. IF remains in this thesis for exactly that reason: it is the stack in which the inverse model is used and is adequate, and it is reported as such.

The same transfer study recorded the failure mode that later disqualified the model for adaptation. Fed its own predictions instead of the measured state, the composite left the reference by more than a metre in *every one* of the 118 runs, and the mechanism was saturation: its pre-clip valve command sat at the limit on 89.1 % of elements, while the ROM valve recorded on the same runs never reached its bound at all. Instrumentation of the composed path in the closed loop measures the same thing directly, and in the terms that matter for adaptation rather than for rollout stability. Over the 33-cell panel the inverse model's command sat at its pre-clip limit on 79.9 % of valve elements, a figure consistent with the per-cylinder occupancies reported in Section 5.7.2; 34.5 % of the corrections the adapter computed were clipped away after being computed; and on 37.9 % of elements the correction could not move the command at all. A bounded residual cannot unsaturate an anchor that is already on its stops, and the consequence is not merely a small effect but an uninterpretable one: a null measured through such a path cannot distinguish a task that is not adaptation-limited from credit that was clipped before it could reach the metric.

The mechanism is visible in the model's own output contract. The inverse model predicts a valve-command *increment*, and the deployed command is the accumulated, clipped quantity $\boldsymbol{u}_k = \mathrm{clip}(\boldsymbol{u}_{k-1} + \Delta\boldsymbol{u}_k, -\boldsymbol{u}_{\max}, \boldsymbol{u}_{\max})$ with per-axis limits [0.95, 0.95, 0.80, 0.80]. Under sustained demand the accumulation walks the command onto its own clip and holds it there.

This is what removes the headroom that online adaptation requires, and it explains why the measured clipping above is structural rather than incidental. The quantity being adapted

and the quantity being saturated are the same, so the saturation is not an occasional excursion that a bounded residual can work around: it is the operating condition under precisely the sustained demand for which adaptation was introduced. A mechanism that needs the command to move cannot be demonstrated on a command that is at its stops nine times in ten.

The disqualification is therefore specific, and it is not a retraction of anything claimed for IF. The inverse model remains adequate wherever its command is consumed as issued, because saturation of a command that nothing further must be added to is a limit on authority rather than a defect of the map. It fails only where a second signal has to be added to that command and survive the same clip. The non-adaptive stack is the first case and the adaptive stack is the second, which is why one keeps the inverse model and the other cannot.

Repairing the inverse model rather than removing it was considered and rejected on cost. The repair would have required a new data collection, and the scale of that collection is known from the precedent in this same pipeline: the forward model suffered an analogous identifiability failure, and it was repaired only by collecting a purpose-built rank-4 open-loop valve-plateau dataset over four plant draws (Section 5.3.1). Two further limitations pointed the same way. The inverse model is trained on Simscape data alone, and deployment already substitutes the training-mean pressure because supplying the measured pressure raises extension error from 1.311 mm to 1.907 mm. A model that must be fed a counterfactual input to remain accurate is a poor foundation for a mechanism whose entire purpose is to react to the true plant.

The decision follows from the direct-valve formulation. Once the policy emits the valve command itself, the inverse model has no remaining function: its sole role in IF was to convert desired cylinder motion into a valve command, and a policy that outputs the valve command performs that conversion inside its own learned mapping. The saturation is not thereby abolished, since the command is still bounded, but it is no longer produced by an accumulator outside the policy's control, and the residual acts on the same quantity the policy itself has learned to keep within range. The inverse model was therefore removed rather than repaired, and no new data collection was required.

6.7.2 Whether the Forward Model Alone Is Sufficient

Removing one of the two learned actuator models raises the question of whether the remaining one carries the path. Sufficiency here is a specific claim: the forward model must convert a commanded valve position into a one-step extension increment with genuine valve sensitivity, because that increment is the only quantity the controller integrates.

The evidence for that claim is the strict retraining of Section 5.3.1, and it is the reason the retraining was necessary in the first place. Before it, the supervised score was misleading: at a 0.5 s horizon, zeroing the valve input changed the rollout-error ratio to only 0.94, meaning the network had extrapolated smooth closed-loop kinematics rather than learned the valve's effect. After strict retraining on the open-loop plateau data the valve-zero error ratio rises to 1.81, extension rollout RMSE falls from 75.2 mm to 12.4 mm at 0.5 s and from 157.9 mm to 27.3 mm at 1.0 s, and per-axis one-step extension R^2 improves from $[-5.7, 0.60]$ to $[0.76, 0.98]$. The kinematic consistency term holds in the strict data with $R^2 = 0.994$–0.9997.

Sufficiency is therefore established on the channel that is used and is not claimed beyond it. Three qualifications are carried forward explicitly. Only the extension increment is integrated; the velocity output remains weak on two axes and is retained as a diagnostic. Zero-order hold remains competitive below 0.25 s, so the forward model earns its place at the horizons the controller actually spans rather than uniformly. And the valve authority the model represents is strongly operating-point dependent, varying by a factor of 15–75 across pressure quartiles and capable of changing sign, which is why a constant valve Jacobian was rejected and why the reduced-order structure is retained elsewhere in the pipeline. Within those limits the forward model supplies what the stack requires, and the composite no longer depends on a second learned model whose saturation was its dominant failure mode.

6.7.3 The Residual Controller

Adaptation enters as an additive, bounded residual on the valve command:

$$\boldsymbol{u} = \mathrm{clip}\Big(\boldsymbol{u}_0 + \alpha \tanh\Big(g_\psi(\boldsymbol{o},\, \phi(\boldsymbol{H}_t))\Big),\, -1,\, 1\Big), \qquad \boldsymbol{u}_0 = \pi_{\mathrm{frozen}}(\boldsymbol{o}), \tag{6.11}$$

where ϕ is a temporal-convolution encoder over a window of (state, action) pairs and g_ψ is a small multilayer perceptron producing a per-cylinder correction. Only ϕ and g_ψ are trained: 53 836 parameters against a 59 588-parameter frozen base.

Three construction choices make the comparison in Chapter 7 a single-variable one, and each was verified rather than assumed. The final layer of g_ψ is zero-initialised and $\tanh(0) = 0$, so at construction the residual is exactly zero and the wrapped policy reproduces the frozen base bit for bit, at a measured maximum absolute action difference of 0.0. Every arm of an ablation therefore begins as the identical policy, and any end-state difference is attributable to the residual having been learned rather than to the task having been relearned. The correction is bounded by $\alpha \tanh(\cdot)$ regardless of what the head produces: with head weights drawn from $\mathcal{N}(0,\, 50)$ the output saturates at exactly α and remains finite, so a bad update degrades accuracy instead of emitting a non-finite action, which matters because a non-finite action written into the history buffer would poison the entire window. And the residual scale can be set to zero at evaluation time without touching any other part of the stack, in which case the stack reduces to the frozen base exactly.

This last property is a strict improvement on the arrangement it replaces. In the earlier latent-conditioned design the actor was itself a 68-input network, so disabling the residual restored the baseline actuator path but not the baseline actor, and the ablation was never quite single-variable. Under AF the base actor is the distilled policy unmodified, the residual carries the whole of the context dependence, and disabling it recovers the baseline in full.

The history window is carried in the observation vector rather than in a buffer held inside the module. PPO calls the actor many times per update epoch, and a module-held buffer would be advanced by the optimisation loop and desynchronise from the simulation; carrying it in the observation means the rollout buffer holds exactly the history under which each action was taken. At evaluation the opposite convention is correct, since the policy is called once per step. The causal convention is fixed throughout: the estimate at step t uses $\boldsymbol{x}_{t-k:t-1}$ and $\boldsymbol{a}_{t-k:t-1}$ and does not include step t.

6.7.4 Why the Base Policy Is Frozen

Freezing the base is structural rather than a tuning preference, and the alternative was closed by measurement before the residual route was adopted.

Under a trajectory-regularised fine-tune with a reference-KL anchor the policy is stable but cannot acquire a new input dependence, because the anchor's reference is context-blind by construction and any context-dependent behaviour therefore increases KL to that reference and is penalised. A gradient multiplier of 1000 applied to the context columns alone moved action sensitivity from 0.001 % to 0.004 % of action scale over 400 iterations. Masking the anchor at $\boldsymbol{z} = \mathbf{0}$, so that context dependence costs nothing in KL, was tried on a criterion fixed in advance and also failed, reaching 0.005 % of action scale with context weights four orders of magnitude below the base columns. The anchor is not the blocker. Without the anchor, PPO is free to walk away from the distilled behaviour and does, trading line tracking for ground contact because that raises its objective.

A residual head has no anchor. It is new capacity whose only function is to depend on the latent, so nothing opposes that dependence. Measured on the same stack, the trained residual's action responds to its history at 3.35 % of action scale, roughly 840 times the anchored route. The residual formulation is adopted on that measurement rather than on preference.

6.7.5 How the Adaptation Encoder Is Trained

The encoder ϕ and the head g_ψ are trained together under the reinforcement objective, not fitted offline to an auxiliary target and handed to a controller afterwards. This is the concurrent scheme of Ji et al. [Ji 22], and its consequence is that the latent is control-relevant by construction rather than by hope: the only gradient reaching ϕ is the one that improves the task.

The alternative was tried in this project and failed in a way worth recording, because it is the failure the concurrent scheme is designed to avoid. An encoder frozen after being fitted offline to a dynamics objective carried about 0.4 % plant identity into the controller and scored 7.1 % worse than simply supplying the mean context. Feeding a policy a near-noise estimate is worse than feeding it a constant. A latent that is accurate against a dynamics target is not thereby useful to a controller, and the offline route provides no mechanism that would make it so.

This changes the status of the adaptation module relative to the earlier design, and the change is one of kind rather than degree. The module is no longer a standalone model with its own supervised validation score that can be quoted independently of the controller; it has no meaning apart from the policy it was trained with. Its quality is therefore reported behaviourally in Chapter 7, through the sensitivity of the action to the history and through the closed-loop contrast against the frozen base, and not as a latent reconstruction error.

6.7.6 Consistency With the Plant Dynamics

Two consistency requirements follow from the system model of Chapter 3, and both are met by construction rather than by observation.

The first is causality. Each history row carries observable quantities only: joint positions, joint velocities, cylinder pressure differences and the previous valve command. The privileged factor vector, the true latent, the reference, future rows and any hidden simulator state are excluded, and the estimate at step t is formed without step t. Nothing enters the deployed path that would be unavailable on the physical machine.

The second concerns what the pressure entries represent, and it is a qualification rather than a guarantee. Under AF the pressure differences supplied to the encoder are recursive states of the frozen forward model, not measurements of the plant. The encoder therefore observes the plant through the forward model's representation of it, and its estimate is consistent with the true dynamics only to the extent that the forward model is. Section 6.7.2 bounds that extent: strong on the integrated extension channel, weak on velocity for two axes, and operating-point dependent in valve authority. This is a real limit on the interpretation of the latent and is carried into the discussion of Chapter 7 rather than resolved here.

What the stack does not claim is a physically named estimate. The residual is trained to improve the task, and the quantity ϕ extracts is whatever serves that objective; it is not constrained to correspond to seal condition, compliance, supply pressure or any other named factor, and it is not reported as though it were. Recovering named physical quantities would require an explicit plant model and a different estimator, which is noted as future work in Chapter 8.

6.7.7 What This Section Claims

The stack is complete and its interfaces are verified. The residual authority α is a genuine tuned hyperparameter and is reported as one rather than presented as a natural constant. The deployed value is $\alpha = 0.5$. At $\alpha = 0.15$ the residual is not merely less effective but actively worse than the base it corrects, at 10.041 mm cut-height error against the base's 7.964 mm. A bounded correction needs enough authority to matter, and below that threshold it contributes noise rather than adaptation; no value of α should therefore be read as incidental. Whether the adaptation path contributes anything to closed-loop task performance is a separate question, settled by the paired comparison of Section 7.6.2 and not asserted here.

6.8 IsaacLab Integration and Timing

Four rates run concurrently and Table 3.4 lists them: PhysX integrates at 120 Hz, the policy acts at 60 Hz under a decimation of two, the learned actuator models advance on a 20 Hz grid of 0.05 s model ticks, and the AAM updates on that same grid. No single one of these is "the control frequency", and reporting a scalar rate for this system would misdescribe every one of the three stacks.

The relationship between the 60 Hz and 20 Hz layers is where integration errors hide, so it is fixed by construction rather than by interpolation. A learned actuator model is evaluated on every third control step and its predicted increment is held across the two intervening steps. The action presented to the model is the action of the tick on which it fires, not an average, and the target integrator accumulates in reference space so that a held prediction cannot re-anchor to a sagging measured position. Chapter 3 records why

that reference-space integration is mandatory: integrating against the measured extension turns a static hold into a slow ratchet.

Order within a tick is fixed and asserted at run time. The history encoder consumes the window ending at the previous tick and produces its estimate before the actuator path runs, ahead of the inverse model in IF and of the residual correction in AF, so no quantity computed at the current tick can influence its own estimate. History buffers are per-environment and cleared on episode boundaries, which matters because the vectorised environments reset asynchronously and a shared buffer would leak one episode into the next. Both properties are checked rather than assumed: a runtime check replays a trace and asserts bit-exact agreement between the online estimate and an offline causal recomputation, and truncating the future of the input leaves the prefix of the output unchanged. Checkpoint loading then compares the stored normalisation statistics, feature contract and history length of every learned component against what the environment expects, and aborts on a mismatch rather than degrading silently.

6.9 Constraints and Fair-Comparison Rules

6.9.1 Limits and Safeguards

Authority is bounded at four independent places, and the redundancy is deliberate because each bound protects against a different failure. The action chain of Section 6.2 limits the rate of change of the command, fades authority as cylinder travel headroom is consumed, and caps how far the integrated target may lead the measured extension. The actuator path clips the valve command to its per-axis limits. The environment terminates an episode on the safety conditions of Section 3.9 rather than letting it accumulate reward in a state an operator would stop. And every learned component falls back to a defined behaviour on a non-finite output.

Preflight runs before any training or evaluation and refuses to start on a violated precondition. The check that does the most work asserts that the commanded reference stays inside the measured actuator authority ceiling of Section 3.6.2: the cylinders move the cutter laterally at most 0.240 m/s, so a path family demanding more is infeasible by construction and no reward, critic or optimiser change could recover it. Terrain is derived from that ceiling, and the preflight assertion keeps the two from drifting apart.

6.9.2 Matching Rules and What Each Comparison Carries

This thesis does not present a three-way ranking of the stacks, and the reason is structural rather than presentational. Two distinct comparisons are made, each with its own matching rule, and each carries one claim. Conflating them would produce a table whose rows are not commensurable.

The Matched Pair: R Against IF

The representation claim rests on R and IF, which agree on every recorded training field and differ only in the actuator path. Held identical across the pair are the verge task and its eleven path families, the terrain generator and the held-out mesh set, the reward function of Table 6.2, the action chain and its authority budget, the eight acceptance gates

and their aggregation to the worst mesh and worst path family, episode length, actor and critic capacity, and the three-layer training construction of distillation, critic warm-up and reference-anchored PPO. That pair is a controlled single-variable comparison and is the only place an across-stack number is read as such.

One departure is declared. R's actuator is the command ROM, and the command ROM plus PhysX *is* the deployment plant (Section 5.4.1). Its tracking advantage over IF is therefore not evidence that a reduced-order model approximates the actuator better than an identified pair, because one of the two defines the target. R is a reference condition, not a competitor.

Why AF Is Not Ranked Against IF

AF does not share the action contract of the other two. Under the direct-valve construction of Section 6.7 the policy emits the valve command itself, where R and IF both emit a bounded desired cylinder increment that a downstream actuator path realises. The quantity the policy chooses is not the same quantity in the two cases, so the action chain that makes the other comparison fair does not extend to this one. This is a declared property of the implementation rather than an observation about results, and it is the reason no row of any table ranks AF against IF on task error. The adaptive claim is not weakened by this, because it was never a claim about beating the non-adaptive stack.

The Paired Comparison: AF Against Its Own Frozen Base

The adaptation claim is made within the adaptive stack, against the frozen base policy that stack was built from, and the matching there is exact rather than approximate. The arms share the frozen base policy, the residual architecture, the trainable capacity of 53 836 parameters, the training budget and the seed. Because the head of g_ψ is zero-initialised and $\tanh(0) = 0$, every arm begins as bit-for-bit the same policy, verified at a maximum absolute action difference of 0.0. The single manipulated variable is whether the residual may see the (state, action) history it is conditioned on.

That construction gives the comparison a property the across-stack contrasts cannot have: the arms are paired environment by environment, so scenario variation is differenced out rather than averaged over, and any separation is attributable to online sensing rather than to added parameters, a different optimiser trajectory or a different checkpoint. The result is reported on this basis in Section 7.6.2.

Fine-Tuning Budgets Must Be Matched or Reported

Comparing a distilled baseline against a separately optimised adaptive policy and attributing the difference to adaptation is not permitted, for the reason established in Section 6.3.2: reward ascent moves a policy off the acceptance criterion, so an unmatched budget is itself a treatment. The paired construction above satisfies this rule by design rather than by bookkeeping. The base policy is frozen and identical across the arms, no arm receives a fine-tuning budget the others do not, and the only optimisation that occurs is of the residual, which every arm trains under the same budget and seed. The rule remains stated because it constrains any future stack added to this comparison, not because the present one is at risk of violating it.

7. Experimental Evaluation and Discussion

7.1 Experimental Protocol

This chapter evaluates the three control stacks assembled in Chapter 6 and the learned actuator models they act through. It answers the last two research questions of Section 1.4:

RQ5 How does the choice of actuator representation affect tracking, command behaviour, latency and robustness under plant variation?

RQ6 Where plant variation survives that robustness, is it identifiable from recent observations, and does online adaptation by latent identification improve closed-loop performance?

The two differ in kind. RQ5 asks what a design choice changes and is answered by direct measurement on the matched pair R (R) and IF (IF), which agree on every recorded training field and differ only in the actuator path. RQ6 asks whether a mechanism pays, could fail in ways worth declaring in advance, and is therefore carried by predeclared hypotheses and by within-checkpoint substitutions rather than by the position of AF (AF) in a table. The short labels R, IF and AF of Section 1.4 are used interchangeably with the full names. RQ6 also decomposes into a chain of dependent questions, each worth testing only if the one before it holds: is plant identity present in the data at all; can an estimator recover it; does the estimate follow the plant when the plant moves; does it improve a model that consumes it; does that improvement reach the controller; and if it does not, what stops it? The first four links hold and the fifth does not, which is why the chapter spends its last experiments on the sixth.

7.1.1 Test Conditions and Experimental Matrix

Every closed-loop experiment scores exactly three rows – R, IF and AF – under one common condition: three held-out terrain meshes, the eleven trajectory families of the

Table 7.1: The experimental matrix. The closed-loop experiments fall into two groups that answer different questions: a matched pair contrasting actuator representations, and a within-stack contrast isolating online adaptation. Model-level experiments use the open-loop excitation collection. The replication unit differs between groups and is stated for each.

Experiment	§	Arms	Unit
Closed loop, matched pair			
Task performance	7.2	R, IF	33 (mesh, family) cells
Chain latency by stack	7.3.2	R, IF	4 cylinders × 16 envs
Commanded step	7.3.3	R, IF	16 envs
Control effort	7.3.5	R, IF	one mesh
Randomisation sweep	7.4	R, IF× five scales	11 families per point
Closed loop, within AF			
Mid-episode load change	7.6.2	base, control, adaptive	528 paired envs
Training budget	7.6.5	400 against 1200 iterations	528 paired envs
Model level			
Identifiability envelope	7.5.1	four excitation tiers	36 held-out runs
Latent content	7.5.3	online, constant, shuffled	five training seeds
Frozen-inverse sufficiency	7.5.4	latent, no latent	five training seeds
Predictive utility	7.5.5	privileged, estimated, mean, shuffled	36 held-out runs
Actuation dead time	7.3.1	four excitation tiers	24 runs

library, a pinned truck speed of 4 km/h, a 26 m scored window, 32 environments and a fixed evaluation seed. Model-level experiments use the open-loop excitation collection `rom_openloop_v1` of Section 7.5.1. Everything scored here is held out: the meshes are unseen, the families are scored individually rather than pooled, and the plant is drawn from sixteen reserved factor configurations whose keys are disjoint from training and validation by construction (Section 4.7), so a scored run is a plant the stack has never seen and not merely an episode it has not seen. How much of that factor draw reaches each stack's plant differs, and is audited in Section 7.4.1. Table 7.1 lists every experiment with its arms and its replication unit.

7.1.2 Metrics, Statistics and Fairness

Closed-loop performance is scored on mean absolute lateral error and mean absolute cut-height error, with ground contact, hard impacts, head alignment and tool acceleration reported alongside. The two errors are never pooled: they differ by an order of magnitude in scale and are traded against one another by some arms. Model-level experiments share one estimand, free-running accumulated normalised state error over a five-tick horizon

with no teacher forcing, because a one-step error flatters every arm equally and hides the compounding that separates them.

Metrics and seeds are fixed before each run. Comparisons are paired, report the sign count beside the mean, and take their intervals from a paired bootstrap. Selection uses the validation split; reserved test runs are scored once.

The unit of replication differs by experiment, and getting it wrong would inflate the evidence. Model-level experiments replicate over training seeds. The closed-loop evaluation is deterministic — five evaluation seeds returned bit-identical numbers to nine decimal places — so its unit is the (mesh, family) cell, and metrics existing only per mesh are marked $n = 3$.

Several controls run alongside the experiments, and each was added because something it guards against had already gone wrong. Input ablation guards against a model scoring well without command authority, and caught a forward model at one-step $R^2 \approx 0.999$ that was insensitive to the valve. Open-loop excitation guards against command and state being made redundant by feedback, and established that the valve is not identifiable from controller-driven data at all. A determinism check guards against reporting one observation several times: five evaluation seeds agree bit-exactly, so the seed is not a unit of replication. Full-path re-scoring guards against a selection window certifying part of an episode, the default covering 8.9 m of 26 m. Retrained baselines guard against comparing numbers across collections, whose normalisers differ. Per-family resolution guards against a worst-family aggregate hiding a threshold artefact. The matched-pair design guards against crediting the architecture for what the training run produced, and the capacity-matched control against crediting sensing for what extra parameters buy. One control remains outstanding: a sham arm, in which the flip schedule fires but the load never moves, would separate adaptation from improved tracking, and it has not been repeated under the current objective.

7.1.3 What the Three Stacks Can and Cannot Be Compared For

R and IF agree on every recorded manifest field — seed, terrain seed, environments, stage, learning rate, entropy coefficient, frozen normaliser and warm-up — and differ only in the actuator path. A difference between them is attributable to the actuator representation, and that pair answers RQ5.

AF is not a third column in the tables that follow. It does not share the action contract: its policy emits the valve command itself where the other two emit a bounded cylinder increment (Section 6.9.2). The cross-stack protocol was also run on an earlier version of it, before the inverse model left the command path, so those numbers describe a controller that is no longer deployed and are withdrawn rather than reported. Re-running that protocol on the rebuilt stack is outstanding.

The adaptation claim is made within AF instead, against its own frozen base and a capacity-matched control, paired environment by environment (Section 7.6.2). That contrast is tighter than any across-stack ranking, because the arms differ in one thing and start as the same policy.

Table 7.2: Candidate adaptive checkpoints re-scored over the full 26 m path, worst-family values on one held-out mesh. The incumbent is among the weakest of the seven. Only `headroom_priced_v1`, whose reward prices cylinder headroom explicitly, clears the ground-contact gate.

Run and iteration	contact	cut height	lateral
`headroom_priced_v1`/299	**0.799**	**0.017**	0.326
`contact_focus_scale025`/625	0.503	0.054	0.092
`boom_focus_scale025_v2`/610	0.403	0.060	0.089
`contact_focus_scale025`/600	0.393	0.061	0.073
`stage31_promotable_v2`/375	0.360	0.068	0.482
`train_final`/599	0.346	0.073	0.107
`speed_range_2_7_contact_safe_v2`/700	0.344	0.076	0.540
gate	≥ 0.700	≤ 0.040	≤ 0.300

7.1.4 Checkpoint Selection and the Scoring Horizon

Every closed-loop result names the checkpoint it was measured on and the window it was scored over, because both were found to change conclusions during this work. Checkpoints are those declared in Table 7.2; none was reselected after seeing a result.

The scoring window matters as much. The default selection window covers 8.9 m of a 26 m path, so a policy can be certified on a third of the trajectory it will actually drive. All results reported here are re-scored over the full path.

7.2 Closed-Loop Task Performance

This is the unperturbed comparison of the matched pair, and it answers RQ5 on tracking. Both stacks are scored on three held-out meshes and eleven trajectory families under identical conditions: pinned truck speed of 4 km/h, a 26 m scored window, 32 environments and the same evaluation seed. The unit of analysis is the (mesh, family) cell, giving 33 paired conditions per stack.

Figure 7.1 shows the three families the stacks track best. The commanded line is followed closely by both, and the visible differences are oscillations about the reference rather than departures from it, at one to two metres wavelength and 26 to 28 mm mean absolute error. The aggregate means are not dominated by this typical case: families such as the double S-curve carry errors an order of magnitude larger, so the stacks differ in the *tail* rather than in typical behaviour. The worst excursion over all cells is roughly 0.25 m for IF against about 2 m for R.

No adaptive stack appears here. The cross-stack protocol was run on the earlier architecture, before the inverse model left the command path, so those numbers describe a controller that is no longer deployed and are withdrawn rather than reported (Section 7.1.3).

One entry is larger than any tracking contrast. IF scores 0.2247 on head alignment, including -0.855 on one mesh: the collapse under unconstrained fine-tuning of Section 6.3.2, measured over the full path. It is the clearest evidence in the chapter that reward ascent

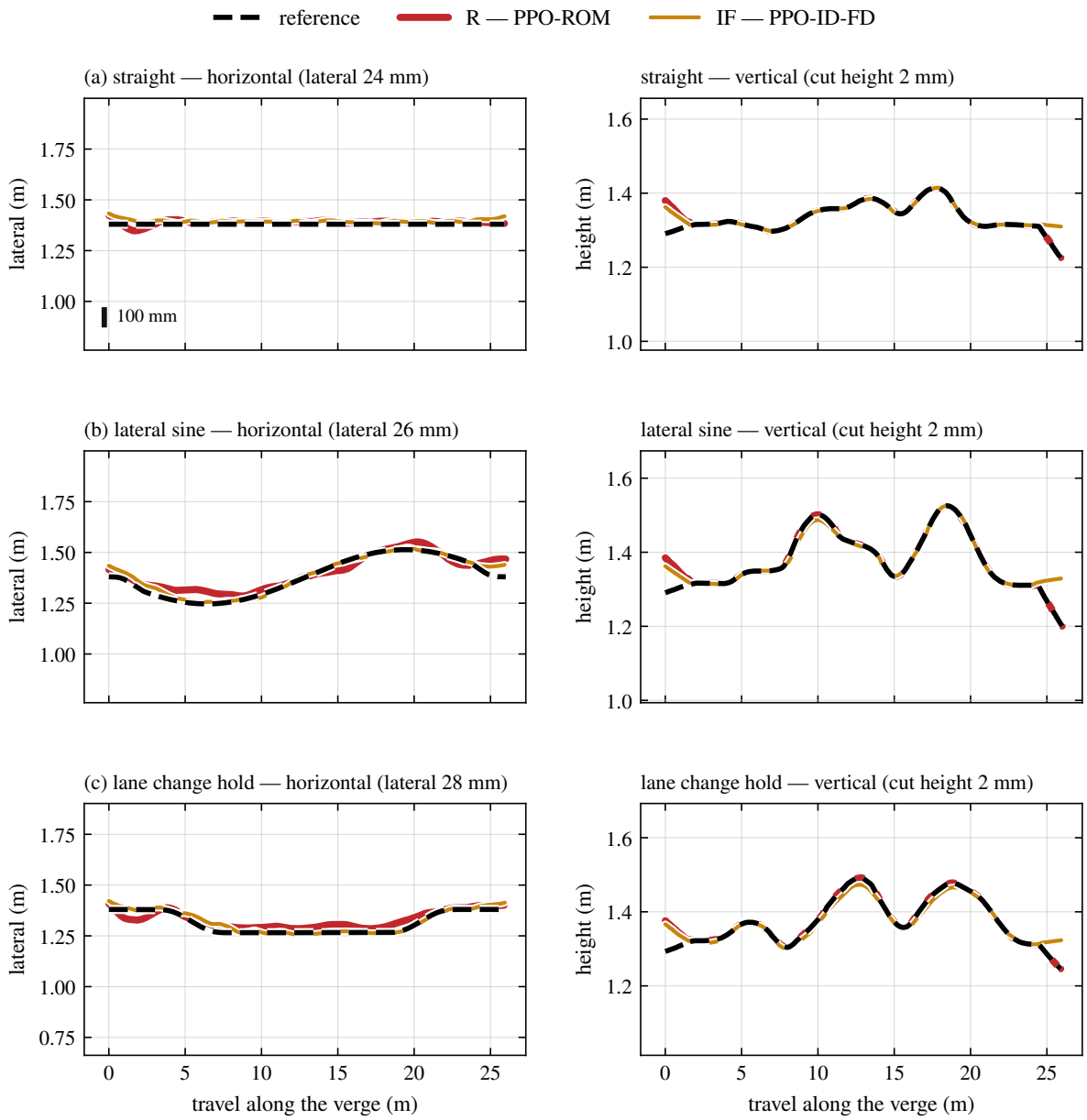


Figure 7.1: Cutter path against the reference on held-out mesh 1337, for the three well-tracked trajectory families that are also structurally different from one another: a flat line, a smooth periodic sweep and a step-and-hold, at 24, 26 and 28 mm mean absolute lateral error averaged over the matched pair. The Gaussian slalom scores marginally better than the lane change but is omitted because its reference is visually a sine and its panel would merely repeat the one above it; the lane change additionally exposes a transient that neither periodic family contains. The left column is the horizontal plane and the right column the vertical plane over the same drive. The vertical reference is not recorded directly and is recovered as $z_{\mathrm{ref}} = z_{\mathrm{cutter}} - e_{\mathrm{cut}}$, which makes both columns like-for-like reference-versus-actual comparisons. In the vertical panels the reference is largely hidden beneath both traces, which is what a 2 mm mean cut-height error looks like when drawn at scale: the cutter follows the terrain undulation closely on every family, and the two stacks separate only in the final metre. AF is not drawn, for the reason given in Section 7.1.3. The environment-to-family assignment is deterministic and was verified by re-aggregating per-environment errors into per-family means and checking them against the run's own gate report.

Table 7.3: Closed-loop performance over 33 paired (mesh, family) cells. Lateral and cut-height error are the mean over families rather than the worst family, which is a maximum dominated by a single hard trajectory. Impacts, head alignment and tool acceleration exist only per mesh and are therefore $n = 3$. R and IF are the matched pair and differ only in the actuator. AF is absent by design: its command path was rebuilt after these runs were scored, so no cross-stack number here describes the deployed adaptive stack (Section 7.1.3).

	R	IF
lateral error, mean (m)	0.1026	**0.0703**
lateral error, median (m)	0.0392	**0.0281**
lateral error, p95 (m)	0.4664	**0.2665**
cut-height error, mean (m)	0.0084	**0.0063**
cut-height error, median (m)	**0.0013**	0.0031
ground contact	**0.9623**	0.9529
hard impacts	0.0947	**0.0905**
head alignment	**0.7438**	0.2247
tool acceleration p95 (m/s^2)	39.90	**39.02**

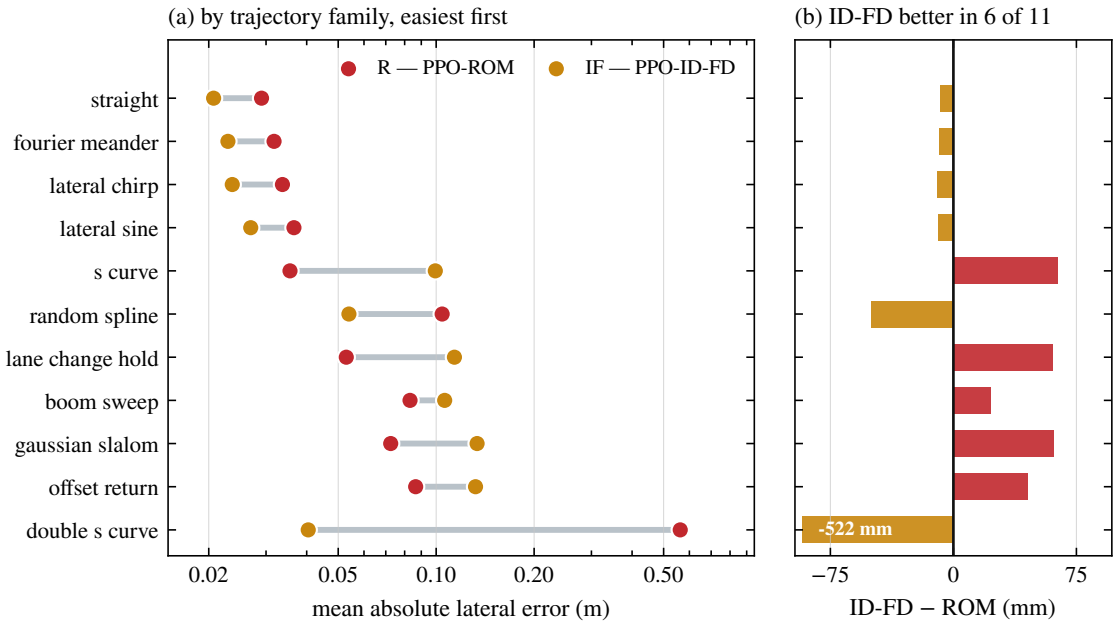


Figure 7.2: Lateral error by trajectory family, which is what a pooled distribution hides. (a) Each row is one family, ordered easiest first, with the bar spanning both stacks; the suite covers more than an order of magnitude, and the two hardest families are what carry the pooled means in Table 7.3. (b) The same data as the paired matched-pair contrast, family by family. Neither stack dominates: IF is better on six of eleven families and worse on five, and ten of the eleven differences are tens of millimetres in one direction or the other. The eleventh is the `double_s_curve`, where IF is ahead by 522 mm; that single family is what separates the pooled means, and its bar runs to the edge of the panel carrying its own value.

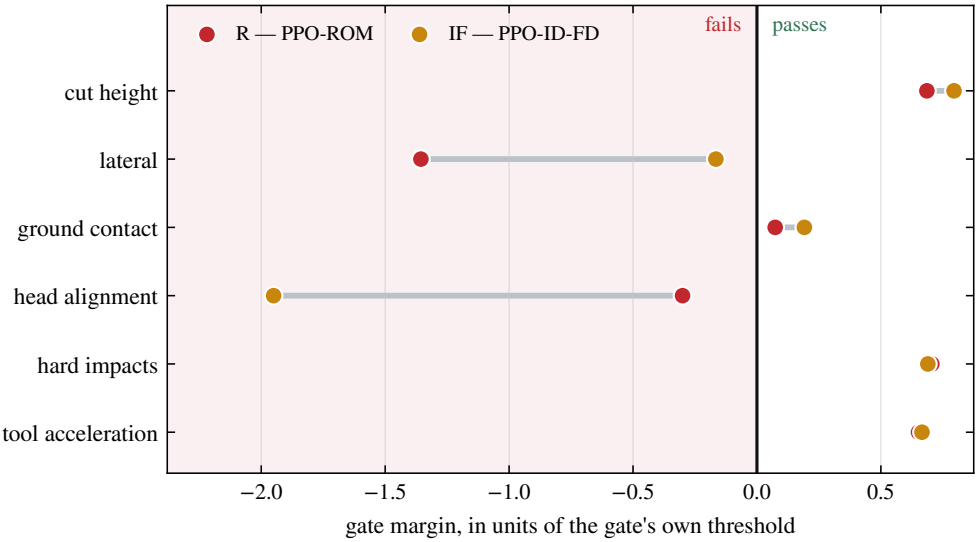


Figure 7.3: Gate margins on worst-mesh statistics, in units of each gate's own threshold, so the six are directly comparable; the shaded half-plane is failure. Two of the eight gates, cylinder headroom and the safety envelope, are not scored per mesh in these reports and are omitted. Two results stand out. The lateral gate is failed by *both* stacks and most severely by R, so it reflects the difficulty of the worst trajectory family rather than a deficiency of either controller. And head alignment is failed badly by IF, which is the fine-tuning collapse of Section 7.2 seen against the acceptance contract.

and the acceptance criterion can move in opposite directions, and it is why the reference-anchored construction exists. The impact and tool-acceleration columns aggregate per mesh rather than per cell, so those entries are sign indications rather than calibrated bounds.

Figure 7.3 places these numbers against the eight-gate contract. On worst-mesh, worst-family statistics the lateral gate is failed by both stacks, most severely by R: a threshold no configuration satisfies is a statement about the hardest trajectory family, not a discriminator between architectures. Figure 7.4 resolves it per family.

IF holds the gate in 32 of 33 cells at a median of 0.0281 m, an order of magnitude inside the threshold, and R in 30 of 33 at 0.0392 m. Each exceeds it on one family, and not the same one: the 0.300 m figure was anchored before the trajectory library was widened, so it now selects whichever family the widening made hardest for a given stack. Both clear the task comfortably in the ordinary case, and the gate discriminates between trajectories rather than between controllers.

Three of the eight gates therefore do not separate the pair; the remaining five do, head alignment most sharply, and the comparative statements of this chapter rest on those. Only the lateral and ground-contact gates aggregate to the worst family, so the per-family reading applies to those two alone.

Three questions follow. Why the representations produce different controllers is answered in Section 7.3; whether the performance survives unseen terrain and plant in Section 7.4; and whether an estimated latent reaches the task from Section 7.5 onwards, ending in Section 7.6.2.

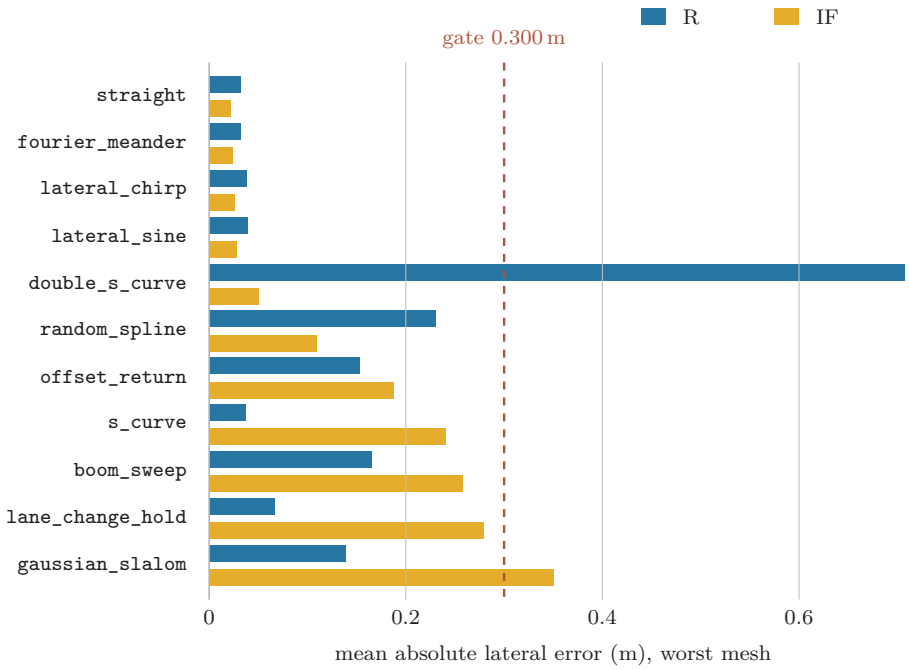


Figure 7.4: Mean absolute lateral error per trajectory family on the worst of the three held-out meshes, against the 0.300 m gate. Each stack exceeds the gate on exactly one family, and on a different family in each case, which is what a threshold measuring the hardest trajectory rather than a controller deficiency looks like. Families are ordered by IF. AF is not shown, for the reason given in Section 7.1.3.

7.3 Actuator Representation and Dynamic Response

Section 7.2 established that the two actuator representations do not produce the same controller. This section measures what physically differs between them, and it is the section that answers RQ5 on latency and command behaviour: how long a command takes to reach the cylinder, how quickly each stack answers a change in what it is asked to do, and what each spends in actuation to track at all.

Latency enters the system in two quite different places, and separating them comes first. The hydraulics impose a transport delay that no controller can remove; the control stack then adds its own, and only the second is an architectural choice. Section 7.3.1 measures the first on open-loop excitation, where it can be isolated cleanly. Section 7.3.2 asks whether the two stacks differ in how quickly they drive the plant, and resolves the difference into its hydraulic and mechanical halves. Section 7.3.3 asks the question that matters for the task – how long each takes to answer a change in the commanded line – and finds the answer dominated by the control side rather than the hydraulic one. Section 7.3.5 then prices the tracking of Section 7.2 in commanded actuation and in command reversals, which is what wears the machine.

7.3.1 Dead Time in the Plant

Delay is measured here rather than assumed, and it is worth separating from an earlier negative result. Attempts to identify the valve's dynamic *model* from the available datasets did not succeed, but that is a parameter-fitting outcome and says nothing about whether

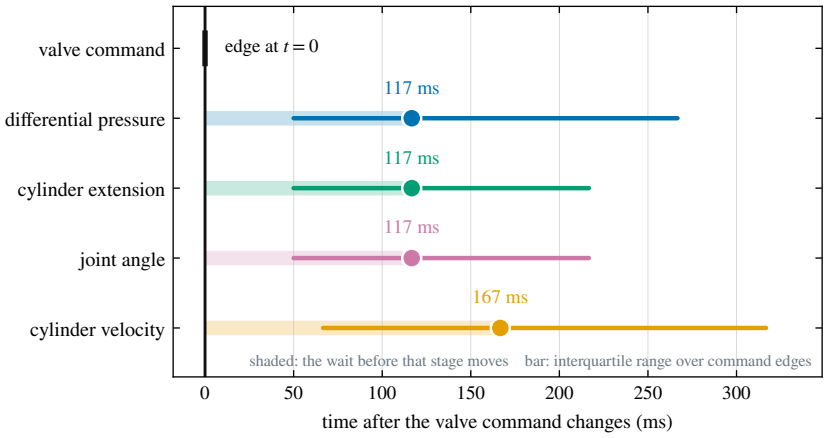


Figure 7.5: How a command propagates through the actuation chain, at excitation amplitude 0.15. The valve command changes at $t = 0$; each row below sits at the delay measured before that stage begins to move, with the shaded lead-in showing the wait, the dot the median onset and the bar the interquartile range over command edges. Pressure, cylinder extension and joint angle all begin at 117 ms and cylinder velocity at 167 ms. That extension and joint angle coincide is the control: the linkage between them is static and must contribute no lag. Each edge is aligned on the command's own onset rather than its steepest sample, since aligning on the slope places the origin after the cause has already acted and the response then appears to precede it.

the command-to-response *latency* is observable. It is, and the open-loop collection of Section 7.5.1 is close to an ideal instrument for it: the excitation is a train of zero-mean square waves with known edge times, which is what one would design for a step-response measurement.

The estimator locates each command edge and records the first sample at which the response leaves the pre-edge noise floor by four standard deviations. A cross-correlation peak was rejected because it conflates dead time with rise time. Figure 7.5 draws the measured onsets rather than the averaged responses themselves: cylinder and joint velocity are differentiated quantities whose edge-averages leave a transient at the command instant that reads as a response preceding its own cause, so a trace of them would misstate the very quantity the figure is about.

The dominant term is the command-to-pressure lag of 117 ms, stable across three of the four amplitude tiers. The kinematic control is exact: the command-to-extension and command-to-joint onsets agree to within 0.0 control steps at every amplitude, so the linkage contributes nothing and the estimator is not manufacturing lag.

The medians do not compose – the first two links sum to 16 control steps while the direct measurement gives 7 to 11 – because the per-edge distributions are wide and the median of a sum is not the sum of medians. Each link is supported individually; the chain decomposition is not.

This matters for the architecture rather than merely describing the plant. A dead time of 117 ms is 2.3 model ticks at the 20 Hz estimator rate and roughly a tenth of the AAM's one-second observation window. A controller reading only the instantaneous state cannot compensate for it, because the information required is not present in a single slice; this

Table 7.4: Transport latency along the actuation chain, median over 24 runs at 60 Hz control, in milliseconds. The last two rows are the control: the two onsets must coincide, and they do at every amplitude. Amplitude-invariance is what distinguishes a transport delay from a threshold effect.

	excitation amplitude			
Link	0.05	0.15	0.35	0.70
command $\rightarrow \Delta p$	117	117	117	150
$\Delta p \rightarrow \dot{\mathbf{s}}$	150	150	133	133
command $\rightarrow \dot{\mathbf{s}}$	183	167	133	117
command $\rightarrow \mathbf{s}$	117	117	100	100
command $\rightarrow \mathbf{q}$	117	117	100	100
linkage contribution	0	0	0	0

Table 7.5: Actuation-chain latency during a closed-loop drive, median cross-correlation lag over four cylinders and sixteen environments. The last row is the control: cylinder and joint are coupled by a static linkage, so it must read the resolution floor of one control step, and it does so on every stack. R and IF are the matched pair and differ only in the actuator. AF is absent: its command path was rebuilt after these runs were scored (Section 7.1.3).

Link	R	IF
command $\rightarrow$ pressure	450.0	**166.7**
pressure $\rightarrow$ cyl. velocity	**0.0**	166.7
command $\rightarrow$ cyl. velocity	283.3	283.3
cyl. $\rightarrow$ joint velocity	16.7	16.7

All entries in milliseconds.

is the standard argument for history-based estimation [Ljung 99], and it holds here on measured rather than assumed grounds.

7.3.2 The Chain Compared Across the Three Stacks

The measurement above characterises one plant on open-loop excitation. The three control stacks reach that plant through different actuator paths, and whether those paths differ in latency is a question about the architectures rather than about the hydraulics. It is answered by recording the same chain during a closed-loop drive on each stack.

One methodological difference must be stated before the numbers, because it prevents a false comparison. A closed-loop command varies continuously and has no step edges, so onset detection cannot be used and the estimator is instead a cross-correlation lag, which conflates dead time with rise time. The values in Table 7.5 are therefore valid for comparing stacks, where the bias is identical across the three, and are *not* comparable with the open-loop dead times of Table 7.4. The two tables do not measure the same quantity.

Three readings follow. The adaptive stack reaches cylinder motion in less than half the time of the other two, 133 ms against 283 ms, and the difference is in the hydraulic half of

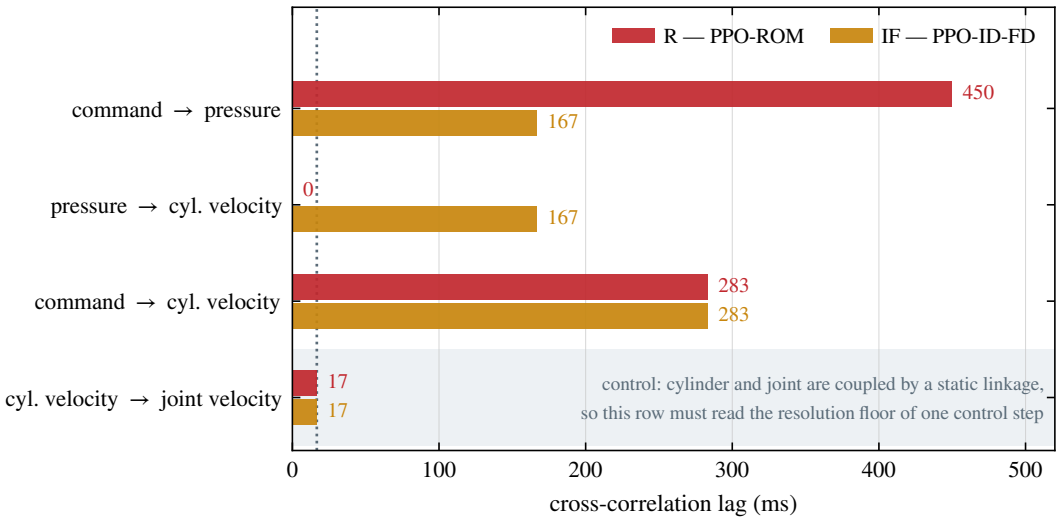


Figure 7.6: The same comparison drawn, matched pair only. The mechanical link at the bottom is the control: cylinder and joint are coupled by a static linkage, so that row must read the resolution floor of one control step, and it does so on both stacks. The hydraulic links above it separate the architectures. AF is absent for the reason given in Section 7.1.3.

the chain rather than the mechanical one. R records no lag at all from pressure to cylinder velocity, which is a structural fact rather than a fast plant: the command reduced-order model predicts pressure and velocity together from one internal state, so there is no transport between them to measure, and its long apparent command-to-pressure lag of 450 ms should be read in the same light, since the valve signal on that path is produced inside the model rather than applied to it. The two stacks that drive the plant through the inverse and forward models, by contrast, show a genuine two-stage propagation with comparable lag in each stage.

The mechanical end is identical everywhere, at one control step, which is the resolution floor: the architectures differ in how they drive the hydraulics and not in how motion propagates through the linkage.

7.3.3 Step Response to a Commanded Change

The dead time measured so far belongs to the hydraulics and is the same for every stack. What differs is how each control stack copes with it, and that cannot be recovered from an ordinary drive: reference and achieved path are coherent only below about 0.3 Hz, where phase reflects the controller's own lag dynamics rather than transport, so no delay extracted there is a dead time (Section 7.4.1). The delay is therefore injected rather than inferred. A step of 100 mm is added to the *commanded* line partway through the scored window, and each stack is scored on the same mesh, seed and speed. The step is applied at the single point where the tracking line is computed, so it reaches the policy's own preview of the path as well as the error term: a step visible to the metric but not to the controller would measure only the metric's discontinuity.

The result is that **the hydraulics are not what makes these stacks slow**. Every stack begins to answer the step three to five times later than the 117 ms the plant needs, and none

Table 7.6: Response to a 100 mm step in the commanded line, median over sixteen environments. The rise time is 10 to 90% of the step and settling is the first time the error stays within 20 mm for half a second. The underlying trajectory is subtracted before measurement, so the only content left is the response to the step itself.

Stack	dead time	rise	settled	overshoot
R	**333**	**617**	2633	**57%**
IF	350	717	**2183**	73%
plant dead time, open loop	117	—	—	—
command rate limit	—	417	—	—

All times in milliseconds.

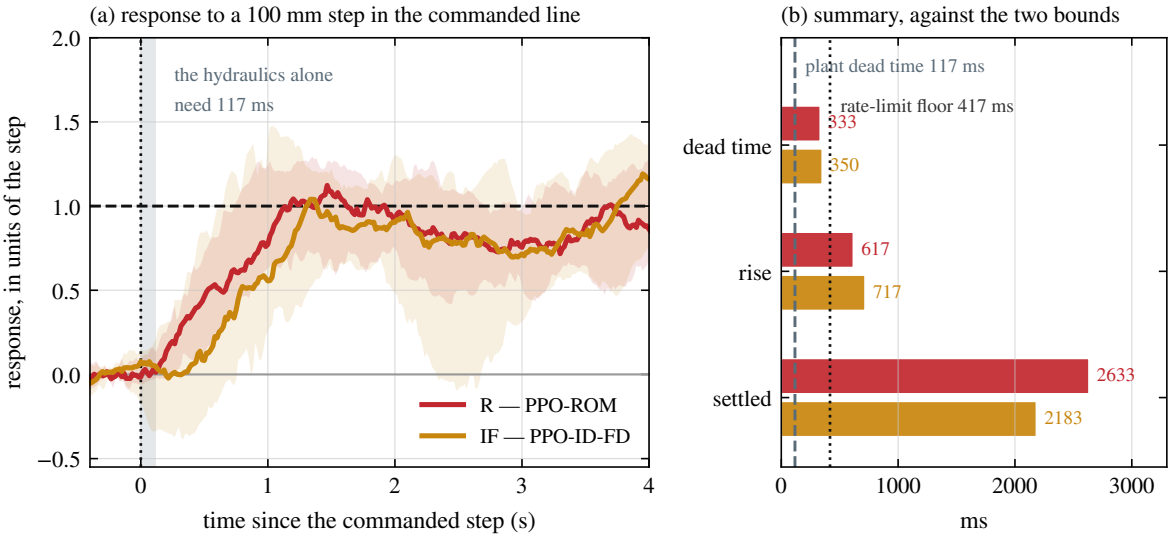


Figure 7.7: (a) Median response of each stack of the matched pair to the commanded step, in units of the step, with the interquartile band over environments; the shaded strip at the origin is the 117 ms the hydraulics alone require. (b) The same summarised, against two references: the plant's dead time and the 417 ms floor the command rate limit imposes on a 100 mm step. Both bounds are small beside the settling times, which is the point of the panel.

settles inside two seconds. Part of that is a hard bound rather than a control deficiency: at a command rate limit of 0.004 m per control step the fastest possible completion of a 100 mm step is 417 ms, which already exceeds the hydraulic dead time by a factor of three. The remainder is control-side, and settling times of 2.2 to 2.6 s are four to five times that bound.

Between the two stacks the ordering is modest and not uniform. IF settles soonest, at 2.18 s against 2.63 s, while R begins soonest, at 333 ms against 350 ms, and overshoots least, at 57 % against 73 %. Neither ordering is large beside the control-side bound above, and that is the point: the differences the actuator representation makes to the chain do not translate into a proportionate difference in how quickly the tool answers a change in command.

The dead-time column compares when a response becomes *detectable* rather than pure

transport, because it is measured against each stack's own pre-step band and the stacks oscillate differently during normal tracking; the rise and settling columns carry the more trustworthy comparison.

7.3.4 The Latency Budget of the Closed Loop

The three results are easy to misread in isolation, and together they say something the chapter needs later. The plant costs 117 ms from valve command to differential pressure, and that figure is amplitude-invariant with an exact kinematic control, so it is a property of the hydraulics rather than of any controller. The stacks differ in how quickly they traverse that chain, and the adaptive stack traverses it fastest, reaching cylinder motion in 133 ms against 283 ms for both others. Yet when the commanded line moves, the adaptive stack is the *slowest* of the three to respond and to settle.

A short path from command to actuator therefore does not produce a quick response at the tool, and the reason is that the dominant term is neither of the two. A 100 mm step cannot be completed faster than 417 ms at the command rate limit, which already exceeds the hydraulic dead time threefold, and the stacks settle in 2.2 to 3.2 s, five to eight times that bound. The actuation chain is not the bottleneck; the commanded rate and the policy's own dynamics are.

This bears directly on the adaptation question. An estimator that updates every 50 ms and a plant that answers in 117 ms are both fast relative to a loop that needs seconds to move the tool to a new line. Whatever the latent contributes, it contributes it to a controller whose response is set elsewhere, which is one candidate explanation for the gap between the model-level effect reported in Section 7.5.4 and the closely matched closed-loop arms of Section 7.2. It is taken up as an explanation once the model-level effect itself has been established, and it is resolved in Section 5.7.3: the correction had no authority left at the point where it was applied.

7.3.5 Control Effort and Command Chatter

Tracking error ranks outcomes but not costs, and two stacks with comparable error are not equivalent if one commands twice the actuation to get there. Three measures are taken from the same drives, chosen so that none restates another: the root-mean-square valve command, its rate of change, and the number of times per second the command reverses the direction of motion. The last is deliberately scale-free, because a stack can command little in total and still chatter, and chatter is what wears the machine.

The measure separates two failure modes that the error column alone conflates. R uses the *least* command of the three and tracks worst, which on its own would read as a conservative controller; its reversal rate of 40 per second, four times either other stack, says instead that its actuation is largely spent fighting itself. That also gives a mechanical account of its tool-acceleration and impact figures in Section 7.2, which otherwise appear as separate observations.

The two stacks driving the plant through the inverse and forward models behave in the opposite way: they command more, reverse a quarter as often, and convert the additional effort into tracking. Between them the adaptive stack commands slightly less and reverses

Table 7.7: Control effort and the tracking it buys, on held-out mesh 1337. Effort read without the error beside it would rank R best, which is the opposite of the truth. R and IF are the matched pair and differ only in the actuator. AF is absent: its command path was rebuilt after these runs were scored (Section 7.1.3).

Stack	RMS command	rate RMS	reversals/s	lateral error
R	0.586	7.55	40.0	120.1
IF	0.827	10.68	**10.6**	**26.8**

Lateral error in millimetres; command in normalised units.

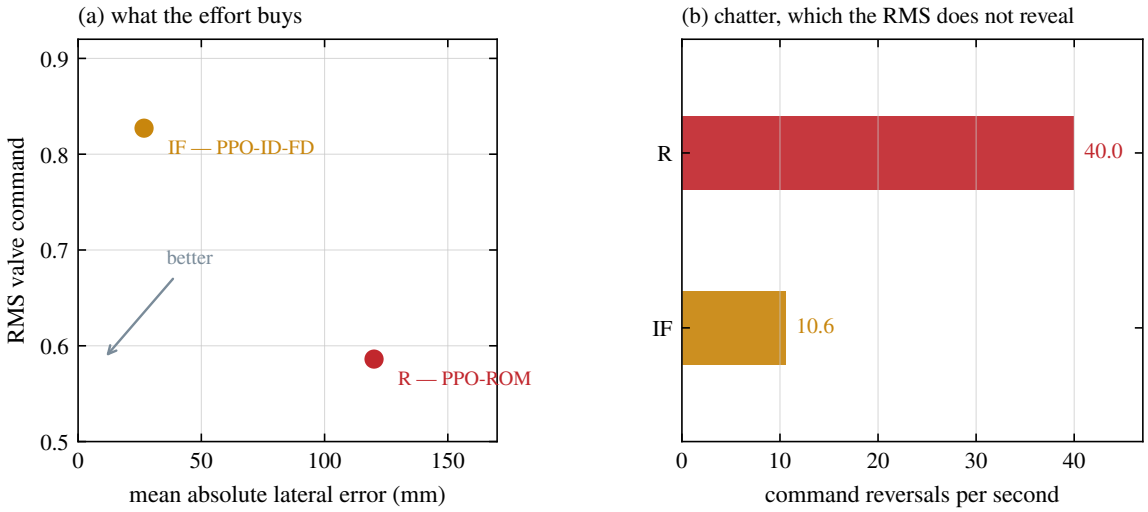


Figure 7.8: (a) Effort against the error it buys, matched pair; the lower left is better. R spends the least and achieves the least. (b) Command reversals per second, which the root-mean-square does not reveal: R reverses close to four times as often as IF. AF is absent for the reason given in Section 7.1.3.

slightly less often than IF while tracking about twice as poorly, so on this evidence it is smoother rather than cheaper.

One limitation bounds what this table can be used for. It compares three distinct policies, so a difference in effort cannot be attributed to any single component of the architecture. In particular it does not test the explanation that the estimated latent disturbs the policy through its variation, since that claim concerns different latents supplied to *one* policy rather than three distinct ones. Measuring effort across the arms of Section 7.6.2, which do share a policy, is the direct test and remains outstanding.

7.4 Generalisation and Robustness

Every closed-loop number in this chapter is already measured on held-out material: unseen terrain meshes, eleven trajectory families scored individually, and plant draws from sixteen reserved factor configurations whose keys are disjoint from training by construction (Section 4.7). Generalisation across simulator, plant draw, terrain and trajectory is therefore demonstrated before this section begins, and demonstrated without any online adaptation.

Table 7.8: The randomised factor space, as applied in the deployment collection. Hydraulic factors are offsets from nominal; `mass_scale` multiplies tool mass and `com_z` displaces its centre of mass in metres. The domains are drawn separately so that hydraulic and mechanical variation can be attributed, and the combined domain draws both. Only the command reduced-order model consumes the four hydraulic factors; on the inverse/forward plant they are inert (Table 7.9).

Factor	nominal	hydraulic	mechanical	combined
`flow`	0	$[-0.25, 0.25]$	0	$[-0.25, 0.245]$
`compliance`	0	$[-0.25, 0.10]$	0	$[-0.25, 0.100]$
`seal`	0	$[0, 1]$	0	$[0, 1]$
`pressure`	0	$[-0.15, 0.15]$	0	$[-0.15, 0.146]$
`mass_scale`	1	1	$[0.883, 1.420]$	$[0.80, 1.50]$
`com_z` (m)	0	0	$[-0.091, 0.050]$	$[-0.10, 0.05]$

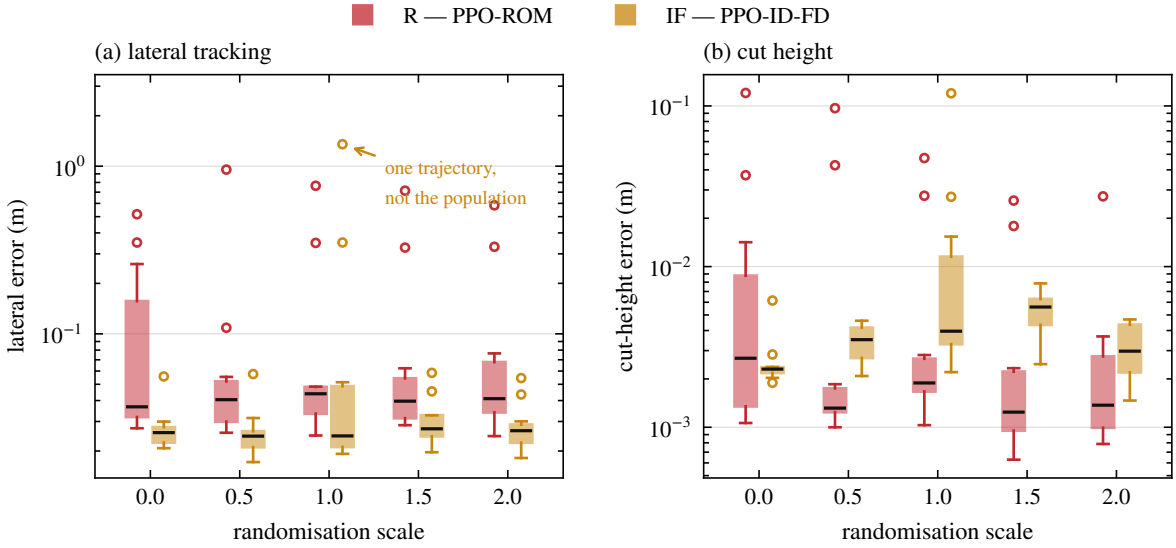


Figure 7.9: Per-trajectory performance at each randomisation scale, matched pair, over the eleven trajectory families of one held-out mesh. Scale 0 is the unrandomised nominal draw. Boxes give the interquartile range and the line the median; points are families outside the whiskers. The medians are flat across the sweep for both stacks. How much of the requested stress reached the plant is examined in Section 7.4.1, and the flatness above scale 1 must be read against it. IF's excursion at scale 1 is one trajectory and not a shift of the population, which a plot of means would have reported as a breakdown.

This section asks the narrower question the sweep was built for: how far the plant can be pushed before performance moves. The factor space is swept at five multiples of the nominal draw, from 0 to 2, over the ranges in Table 7.8; the first point applies no randomisation and is the nominal plant.

The curves are flat. Neither stack's tracking moves across the sweep, on any metric, at any scale. A flat curve admits two readings — the controllers absorbed the stress, or the stress never arrived — and the next subsection establishes which, because the answer determines what the sweep is evidence for.

Table 7.9: Where each deployment factor acts. The inverse/forward plant takes no factor input, so on the two stacks that use it the hydraulic factors are inert and reach only the privileged encoder used by the oracle arm. This was established by inspection of the runtime and is checked empirically by the reachability control described in the text.

Factor	R (command ROM)	IF, AF (ID/FD)
flow, compliance, seal, pressure	acts, clamped to training range	inert
tool mass, centre-of-mass height	acts, unclamped	acts, unclamped

Table 7.10: Requested against delivered stress over the sixteen held-out factor rows. "Delivered" is the applied hydraulic deviation from nominal as a fraction of the requested one – a ratio in factor space, not a measure of authority – so 1.000 means the sweep point reached the plant intact. The last column is the separate quantity that does scale realised motion, the reduced-order model's response gain. Above scale 1 the hydraulic half of the sweep is progressively pinned to the training envelope, while tool mass and centre-of-mass height are never clamped.

Requested scale	rows with a pinned factor	delivered	min. response gain
0.5	0/16	1.000	0.671
1.0	0/16	1.000	0.472
1.5	13/16	0.911	0.402
2.0	16/16	0.778	0.376

7.4.1 How Much of the Requested Stress Reached the Plant

A flat curve across a randomisation sweep can mean the controller absorbed the stress or that the stress never arrived. Two properties of the perturbation path decide which, and neither was accounted for when the sweep was designed.

Which factors reach which plant.

The six factors do not all act, and what acts differs by stack (Table 7.9). The command reduced-order model takes the four hydraulic factors as an explicit input, so they act on R. The inverse/forward plant does not: its step consumes the valve command, the cylinder state, the differential pressure and the load, and takes no factor argument at all. On the stacks built from it the hydraulic factors reach the privileged encoder and the logs and nothing else, and what varies across a held-out draw is tool mass and centre-of-mass height, applied through PhysX without a clamp.

How much of the rest survives clamping.

Even where the factors do act they are bounded by the plant. The reduced-order model pins every hydraulic factor back into the interval it was identified on and then bounds the realised actuator response gain to $[0.35, 1.50]$. Table 7.10 evaluates that over the sixteen held-out rows the sweep visits. At requested scale 1.5, thirteen of sixteen rows have at least one factor pinned; at scale 2 all sixteen do, and 77.8 % of the requested deviation arrives. The label "twice every deviation" is therefore accurate for the mechanical factors and wrong for the hydraulic ones.

The response gain is the quantity that concerns authority, since it scales the realised cylinder motion directly. Within the clamped ranges the worst case the six factors can compose is 0.376, so the factor path cannot scale the reduced-order model's realised motion below about 38 % of nominal, a reduction of at most 62 %. On the inverse/forward plant the same factors scale nothing at all.

The consequence for reading Figure 7.9 is direct. The flat curves are not evidence that the stacks are robust to hydraulic variation, because for two of them the sweep did not vary the hydraulics, and for the third it varied them inside the envelope the model was identified on. What the sweep does establish is insensitivity to tool mass and centre-of-mass height over the range tested, which is a narrower claim than the one the sweep was built to make, and it is the reason the adaptation experiment perturbs mass rather than hydraulics.

7.4.2 What a Robustness Boundary Would Require

The sweep above does not locate a robustness boundary, and extending it would not either: Table 7.10 shows the upper scales delivering less of what they request rather than more. Finding the point at which command authority becomes insufficient needs a different stress axis, one that reaches every stack and means the same thing on each. The deployment factors are not such an axis, for the two reasons above.

The axis that satisfies the requirement scales the actuator's realised motion directly, immediately before it updates the physics target:

$$\Delta x_k^{\text{applied}} = \alpha \, \Delta x_k^{\text{actuator}}, \qquad \alpha \in (0, 1], \tag{7.1}$$

where $\Delta x^{\text{actuator}}$ comes from the command reduced-order model on R and from the inverse/forward pair on the others. It bypasses the clamp, it is applied at the same point in every stack, and it is named for what it is: actuator-authority degradation at the deployment interface, not seal, not flow, and not any hydraulic parameter.

That axis was subsequently implemented, and it is the α of the plant perturbation used in the adaptation experiment of Section 7.6.2, applied there together with a change of load. The boundary search itself, sweeping α downwards on each stack until the acceptance gates fail, is specified but not run, and it is the most direct extension of this chapter.

7.5 Identifiability of Plant Variation

Sections 7.2 to 7.4 answer RQ5 and leave RQ6 standing. Both representations support closed-loop control on held-out terrain, plants and trajectories, and neither degrades when the plant is moved as far as this work can move it – all without any online adaptation. That is the setting in which adaptation has to earn its place, and the first question is not whether an estimator is accurate but whether there is anything in the data for it to recover. This section measures that, and then asks what kind of quantity the recovered latent actually is.

7.5.1 Excitation Design and the Identifiability Envelope

What the data makes identifiable determines what any estimator can recover, so it is measured before any estimator is judged, and the headline is that excitation gathered under a task controller does not separate plants at all.

Table 7.11: What each excitation tier makes identifiable, and why the higher tiers do not. Decode R^2 is held out over 36 reserved runs; drift is the across-plant spread of cylinder extension at matched ticks, expressed as a fraction of the stroke the probe itself produces. Plant identity survives the first two tiers and is gone by the third, while command saturation never exceeds ten per cent.

Amplitude	seal R^2	mean R^2	drift / stroke	command sat.
0.05	0.703	0.109	0.18	0.11%
0.15	0.505	0.099	0.28	0.35%
0.35	−0.018	−0.040	0.39	3.26%
0.70	0.043	−0.021	0.50	9.98%

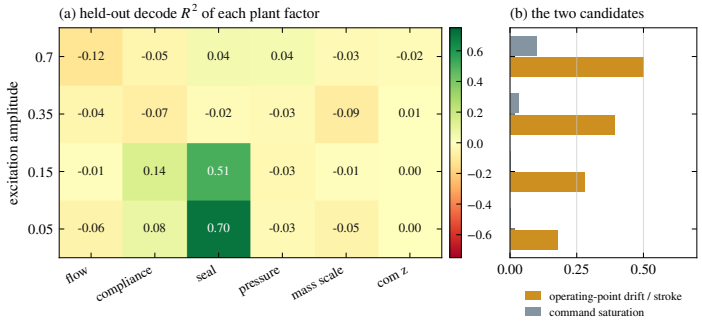


Figure 7.10: The identifiability envelope. (a) Held-out decode R^2 for each of the six plant factors at each excitation level. Only the seal condition is recoverable, and only in the two lowest tiers. (b) The two candidate explanations, on the same amplitude rows: operating-point drift, as a fraction of the stroke the probe produces, rises steadily with amplitude, while command saturation stays low throughout and cannot account for the collapse.

The reason is structural: a task controller computes its command from the measured state, so command and response are nearly redundant and little is left for a plant descriptor to explain. The input must be rich enough to keep the regressor well conditioned [Ljung 99, Schoukens 19]. The collection used here therefore removes the controller and applies open-loop zero-mean square-wave excitation at the cylinder-command interface, since the command reduced-order model accepts cylinder extensions rather than valve signals (Section 5.4). It comprises 118 plant draws of 11 040 steps, split 82/20/16 by run, with a Hankel matrix at full rank 40 of 40 and condition number 17.8. Four amplitude tiers are swept; Table 7.11 decodes the six true factors from the per-tier mean estimate on held-out runs, with seal condition as the positive control.

The obvious explanation for the collapse, actuator saturation, is wrong: saturation reaches only 9.98% of steps even at the largest amplitude, which cannot account for a decode falling from $R^2 = 0.703$ to zero. Operating-point drift does account for it. Every run executes an identical schedule, so at a matched tick the only difference between runs is the plant, and the across-run spread of cylinder extension grows from 1.5 mm at the start to 54.2 mm at the end, half the stroke the probe itself produces. Once runs are that far apart, a one-second window cannot separate a response that differs because the seal is leaking from one that differs because the cylinder sits five centimetres lower.

Two consequences constrain later sections. All adaptation experiments here run below amplitude 0.35, inside the band where plant identity is present at all. And because the plan ramps amplitude monotonically across the episode, one tier per quarter, amplitude and episode time are the same axis: this collection supports statements about amplitude, not about elapsed time. A pilot analysis that read decode quality as decaying over the episode was reading the amplitude sweep.

7.5.2 Closed-Loop Data Against Excitation Data

The envelope above is a statement about data rather than about estimators, and the sharpest demonstration of that is an architecture that changes sign when the data changes. The latent-conditioned state predictor of Section 7.5.4 lowers free-running error by a median of 22.3 % on the open-loop collection. On the earlier collection gathered under a task controller, the same architecture, the same objective and the same selection criterion put the latent *behind* its no-latent baseline: the no-latent predictor scored 0.17278, the end-to-end latent 0.17596 and the privileged latent 0.18515, so plant conditioning appeared worthless or worse.

Nothing about the model changed between those two results. What changed was whether the data contained an answer, exactly as the envelope above predicts: under a task controller the command is computed from the state, so command and response are close to redundant and the plant has little left to explain. Two consequences follow for this chapter. Reporting only the closed-loop collection would have supported a confident negative conclusion about the architecture that the open-loop data contradicts, which is why every model-level experiment here uses `rom_openloop_v1`. And an identifiability check of this kind is cheap, requires no estimator, and belongs before rather than after the decision to build one.

7.5.3 Information Content of the Adaptive Latent

That the latent helps does not establish why. Either the estimator recognises which plant it is on, or it tracks a time-varying operating condition and its value lies in the estimate moving. A substitution experiment separates them, changing nothing but the contents of the latent slot. One trained model is scored three ways: *online*, read from history at every tick as in deployment; *constant*, replaced by that run's own mean, which preserves plant identity exactly and destroys all variation; and *shuffled*, replaced by another run's mean, preserving the distribution while making the identity wrong.

The decomposition is tight across seeds (Figure 7.11b). Removing the variation costs 13.6%, per-seed range 11.9 to 14.3%; removing the identity on top of that costs a further 0.43%, range 0.32 to 0.84%. The second is consistently signed across all five seeds and so is real, but roughly thirty times smaller. Essentially all of the latent's value lies in the estimate moving, almost none in its being correct about which plant this is. The thesis therefore does not claim the module identifies the plant; it claims the module tracks a time-varying operating condition, and that the tracking earns the performance. That is the ordinary form of such an estimator: a fixed one-second causal window carrying no state between ticks re-estimates rather than accumulates, as recursive estimation with a forgetting factor does when the parameters are themselves time-varying [Åström 95, Ljung 99].

Table 7.12: Where the latent's value lies, five seeds, one trained model each, scored on the reserved validation runs. Freezing the estimate at its correct per-episode value costs 13.6%; making that frozen value wrong costs a further 0.43%. Both effects have the same sign in every seed.

Latent supplied to the state model	mean error	vs. online	seeds
online estimate $\mathbf{z}_{\mathrm{AAM}}(t)$	0.018842	—	—
per-episode constant, correct identity	0.021402	+13.6%	5/5
per-episode constant, wrong identity	0.021492	+14.1%	5/5

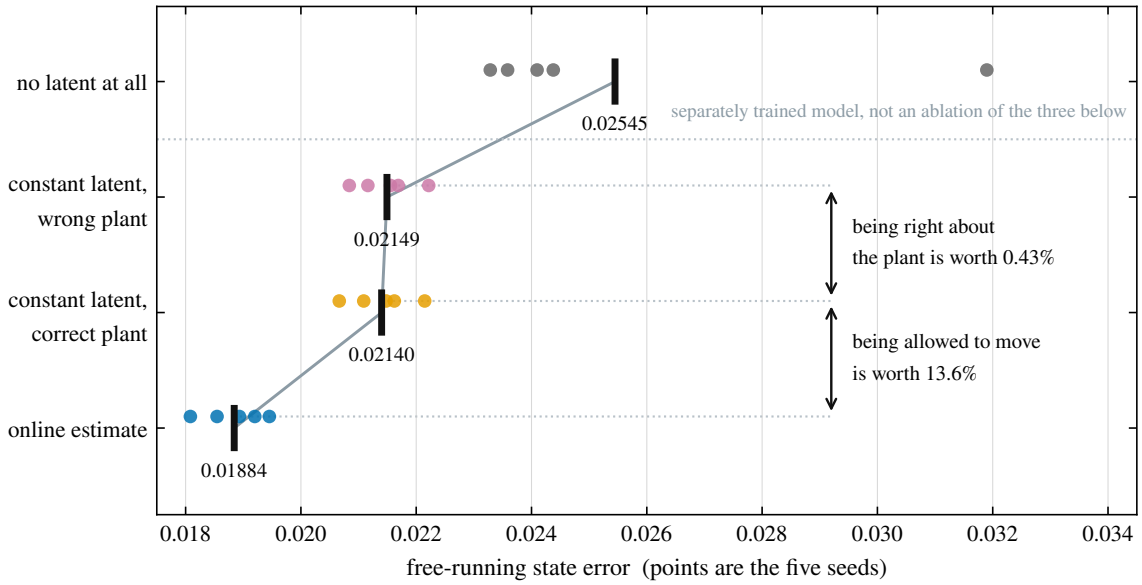


Figure 7.11: The model-level result as a single sequence, worst at the top. Each point is one of the five seeds and the heavy bar is the mean. The *gaps* are the claim: making the constant latent correct about the plant rather than wrong is worth 0.43%, while allowing that same estimate to vary in time is worth 13.6% – a factor of thirty-two. The topmost row is a separately trained no-latent model rather than an ablation of the three below it, so the step down to it is not attributable to the latent alone and is divided off.

Two boundaries apply. This analysis concerns the estimator trained on open-loop excitation, a different model on different data from the deployed one of Section 5.6.1: that one is static within a run, this one varies within an episode by roughly nine times the spread between plants, and excitation decides which behaviour appears. And everything here is measured on a dynamics model rather than in the control loop; only Section 7.6.2 carries the claim into the task.

7.5.4 The Model-Level Gain and the Frozen Inverse Model

The inverse model is trained on randomised Simscape data and deployed against the IsaacLab reduced-order plant, and the architecture of Chapter 6 depends on it staying frozen. The question is whether that is defensible. The direct measurement of inverse-model transfer error is reported in Section 5.2.2; what is tested here is the weaker but

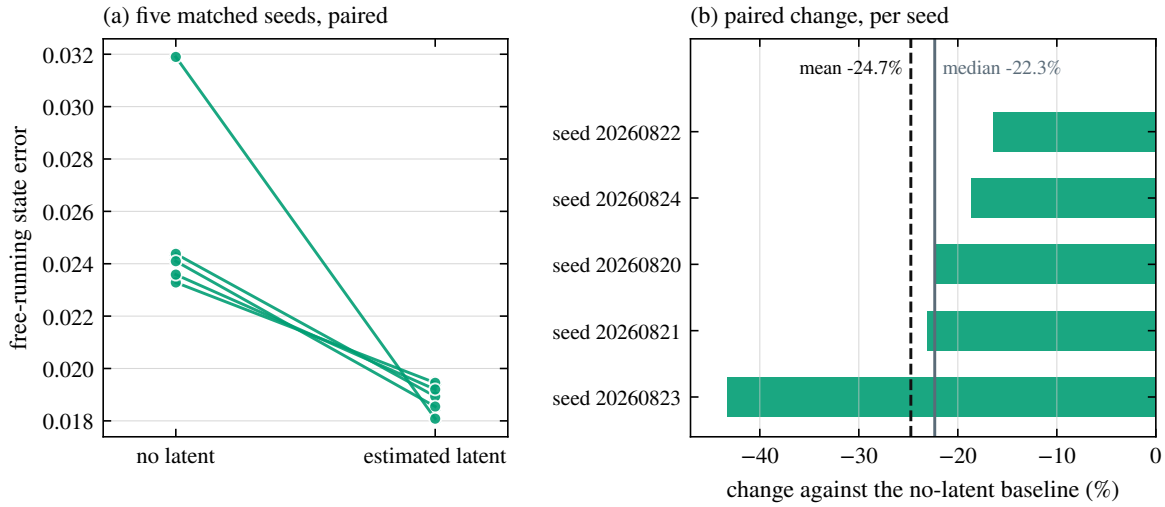


Figure 7.12: Latent conditioning against a matched no-latent baseline, five seeds. (a) Each line is one seed. No pair crosses, which is the result; the mean is not. (b) The same data as paired changes, with the mean dashed and the median solid. Seed 20260823 carries the mean because its baseline is an outlier, so the median −22.3 % is the figure to quote.

Table 7.13: Sufficiency of the frozen inverse model, five matched seeds. The estimated latent lowers free-running state error in every seed. Seed 20260823 has an anomalously poor baseline and inflates the mean, so the median of −22.3% is the representative figure.

Seed	no latent	estimated latent	change
20260820	0.02438	0.01893	−22.3%
20260821	0.02410	0.01855	−23.0%
20260822	0.02328	0.01945	−16.5%
20260823	0.03190	0.01808	−43.3%
20260824	0.02359	0.01920	−18.6%
mean	0.02545	0.01884	−24.7%

more consequential proposition that whatever residual gap remains is learnable *without touching the inverse model at all*.

The test removes the inverse model from the loop entirely. A state predictor consumes the current state, the applied command and an estimated latent inferred end to end from $[\mathbf{q}, \dot{\mathbf{q}}, \Delta p, \mathbf{u}]$, and predicts the next state. The baseline is the same predictor with no latent input, retrained on the identical data, split and objective, at five matched seeds. Figure 7.12 reports the pairs.

The estimated latent improves on the no-latent baseline in 5 of 5 seeds, with a mean paired change of −24.7% and a paired bootstrap 95% confidence interval of [−34.2%, −18.2%] that excludes zero (Figure 7.11a). One qualification belongs with the number rather than in a footnote: seed 20260823 produced an unusually poor baseline at 0.03190 against roughly 0.024 elsewhere, and its −43.3% is as much a statement about that baseline as about the latent. The median change of −22.3% is the figure to quote, and the sign, reproduced in every seed of a paired design, is what supports it.

Table 7.14: Free-running rollout error under four latents, pooled over 36 held-out runs. Lower is better. The estimate tracks the privileged latent at every horizon, and both are clearly ahead of an uninformative or a wrong one.

Horizon	privileged $\mathbf{z}_{\mathrm{SD}}$	estimated $\mathbf{z}_{\mathrm{AAM}}$	mean	shuffled
1 tick	2.961×10^5	2.966×10^5	3.388×10^5	3.499×10^5
10 (0.5 s)	1.820×10^6	1.826×10^6	1.952×10^6	1.998×10^6
20 (1.0 s)	2.961×10^6	2.977×10^6	3.196×10^6	3.256×10^6

That the same comparison reverses on controller-driven data is what makes the result interpretable, and it is reported with the identifiability evidence in Section 7.5.2.

The claim is therefore supported, with its scope stated plainly. The experiment does not measure inverse-model error, and it is not evidence that the inverse model is accurate in any absolute sense. It shows that the residual mismatch between the Simscape training distribution and the deployment plant is recoverable by a latent-conditioned state model, which is what licenses the simpler architecture in which the inverse model is never fine-tuned online.

7.5.5 Estimated Against Privileged Latents

Section 7.5.3 established that the latent's value is temporal and Section 7.5.4 that it lowers free-running error. Neither asked the question a controller cares about: is the *estimated* latent as useful as the true one, and is it better than no plant information at all? That comparison is made here, on the frozen system-dynamics surrogate, by supplying four different latents to one network and changing nothing else.

The four arms are the privileged latent read from the true factors, the causal AAM estimate, an in-distribution mean carrying no plant information, and a shuffled latent with the right distribution and the wrong plant. All four are scored on the same 36 held-out runs, on free-running rollouts at one, ten and twenty ticks, with the model integrating its own predictions.

Table 7.14 gives the ordering, and the paired statistics give its standing. Against the privileged latent the estimate is indistinguishable: the paired difference at twenty ticks is $+1.60 \times 10^4$ with a 95 % interval of $[-1.35 \times 10^4, +4.62 \times 10^4]$, and the estimate is ahead in 18 of 36 runs, which is exactly chance. Against the two uninformative controls it resolves: -2.19×10^5 with the interval excluding zero and 33 of 36 runs against the mean latent, and -2.79×10^5 with 29 of 36 against the shuffled one. On the reserved test split the estimate is marginally ahead of the privileged latent, 2.837×10^6 against 2.850×10^6.

Two things follow. The AAM recovers essentially all of the information the privileged encoder carries, so the estimator is not the limiting component of the adaptation chain. And the advantage resolves on extension and velocity, not only on pressure: against the mean latent the extension difference is -1.87×10^{-4} with 32 of 36 runs and the velocity difference -5.14×10^{-4} with 31 of 36, both intervals excluding zero. This is a different comparison from the one in Section 5.5, which contrasted a latent-conditioned model with a separately trained no-latent model and found the advantage confined to pressure. Substituting latents *within* one model isolates the latent itself, and by that measure the channels the controller integrates do respond.

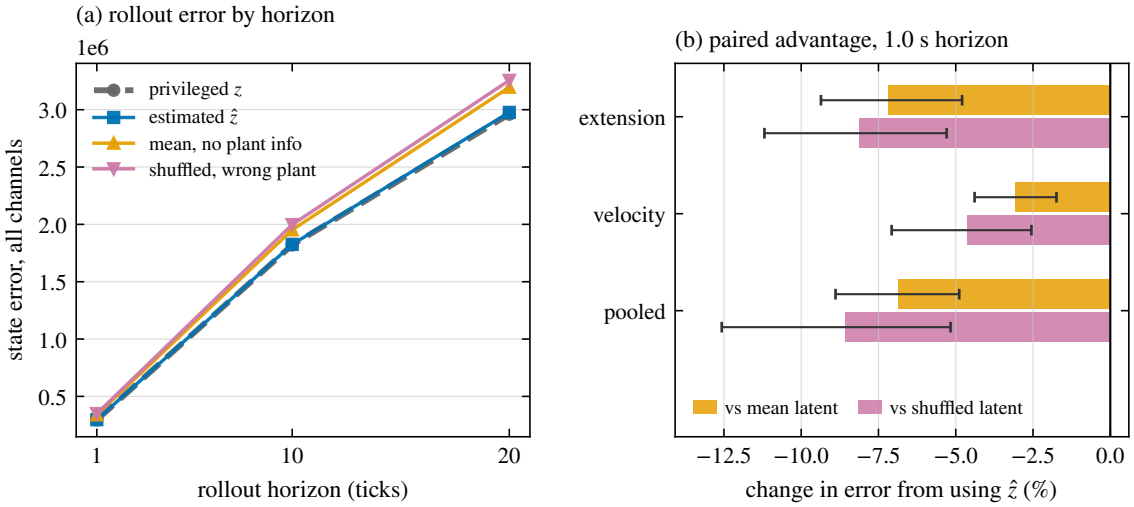


Figure 7.13: Predictive utility of the estimated latent, 36 held-out runs. (a) Free-running rollout error against horizon under the four latents. The estimate tracks the privileged latent at every horizon while both stay clear of an uninformative or a wrong one. (b) Paired advantage of the estimate at the 1.0 s horizon, per channel, as a percentage of the control arm's own error, with 95 % paired bootstrap intervals. Extension and velocity are the channels the deployed controller integrates, and both resolve.

7.6 Online Adaptation

Plant identity is present in the data when the excitation is right, and what the latent carries is a moving operating condition rather than a fixed label. This section asks what that buys. It proceeds outward from the model to the machine: first whether the latent improves a model that consumes it and whether the estimate is as good as the truth, then whether the estimate follows the plant when the plant moves, then whether either matters to the closed loop under a plant change and under an oracle that removes estimator error altogether.

7.6.1 The Adaptation Experiment

The preceding sections establish that plant identity is present, recoverable and useful to a model. This section asks the question the thesis is named for: whether it changes what the machine does. The experiment is deliberately narrow, because a broad one would not be attributable.

Three arms are compared. All three share the frozen base policy, the residual architecture, the 53 836 trainable parameters, the training budget, the reward and the seed. Because the head of g_ψ is zero-initialised, all three begin as bit-for-bit the same policy. One thing is manipulated: whether the residual can see the (state, action) history it is conditioned on. The *adaptive* arm sees it; the *control* arm has the same capacity and the same optimiser but its history window is zeroed; the *base* arm is the frozen policy with no residual at all.

The plant is built so that seeing the history is necessary rather than merely possible. The cutter mass flips between two well-separated modes mid-episode, the flip instant is jittered per environment so it cannot be anticipated on a timer, and the direction is balanced across environments so it cannot be assumed. A policy that cannot sense the change

Table 7.15: The three arms under a mid-episode change of load, 528 paired environments across three held-out meshes. The control arm differs from the adaptive arm only in that its history window is zeroed; the frozen base, the architecture, the 53 836 trainable parameters, the budget, the reward and the seed are shared. Percentages are median paired improvements over the frozen base.

	cut-height error		lateral error	
Arm	mean	over base	mean	over base
base (frozen policy)	7.964 mm	—	61.155 mm	—
control (history zeroed)	5.380 mm	—	61.521 mm	—
adaptive (history live)	**3.772** mm	+120.1 %	**35.371** mm	+78.2 %

must commit to one compromise response, and that response is wrong in one half of every episode. Scoring uses 528 paired environments across three held-out meshes, with bootstrap intervals on the paired difference.

7.6.2 What Control and Adaptation Each Contribute

The result is clearest on lateral error, and it is clean because the control arm settles the attribution by itself. Table 7.15 gives the pre-registered comparison: 528 paired environments across three held-out meshes, under the plant the arms were trained on.

The adaptive arm improves both scored errors against the policy it wraps: cut height by 120.1 % and lateral by 78.2 %, with paired differences of +3.626 [+3.082, +4.503] and +22.533 [+20.802, +23.990] millimetres. Against the capacity-matched control it improves both again, by 56.9 % and 80.4 %, with paired differences of +1.836 [+1.553, +2.155] and +25.266 [+23.378, +26.367]. All four intervals exclude zero.

The control arm is what makes this attributable, and the two metrics divide the credit differently. On *lateral* error, adding the residual's capacity with nothing to sense changes nothing at all: 61.155 mm becomes 61.521 mm, which is no movement. Letting the same residual see the history takes it to 35.371 mm. The arms differ in one thing only, so that improvement belongs to the history input rather than to parameters, optimisation or seed. On *cut-height* error capacity already does much of the work, 7.964 mm to 5.380 mm, and the history improves it again to 3.772 mm. Both stages help; on that metric the larger share is capacity, and saying so costs nothing because the lateral result carries the claim.

A separate run on one held-out mesh, used for the traces in this section and in Section 7.6.4, reproduces the same structure on 176 environments: the control arm sits within 0.02 % of the base on lateral error and wins in 53 per cent of environments, which is chance, while the adaptive arm halves it and wins in 80 per cent against the control, at a median paired ratio of 1.74 with interval [1.58, 1.86]. The pattern is not an artefact of one plant draw.

7.6.3 Where the Advantage Comes From

The history input is doing the work, and it is worth being precise about what kind of work, because two mechanisms would both produce the table above.

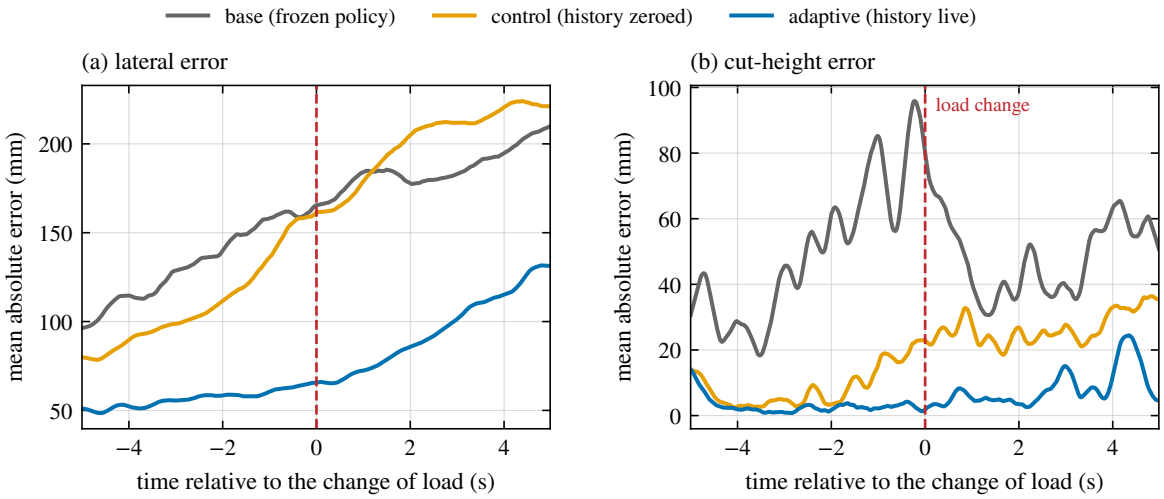


Figure 7.14: The three arms through the change of load, from the single-mesh run: the 151 of 176 environments with a full 5 s window either side of their own flip instant, each aligned on that instant. Curves are the mean over environments, the estimand of Table 7.15. The adaptive arm holds both errors below the other two throughout. (a) The frozen base and the control arm track each other on lateral error, which is the separation the table reports: capacity alone does not move that metric. (b) On cut height the control arm sits well below the base, which is the same table's other reading.

The first is response to the change itself: the residual senses that the load has moved and corrects for it. The second is continuous conditioning: the residual reads the operating condition it is in at every tick and commands accordingly, whether or not anything has just changed. Both are online adaptation in the sense this thesis uses, since both condition the controller on runtime history, but they predict different things.

Two measurements separate them. Across the matched windows the adaptive arm's advantage over the control is 3.09 before the change and 3.24 after it on cut height, which is flat. And splitting by the load the arm is currently carrying, the advantage is larger at nominal load than at the raised one, 2.81 against 1.40 on cut height. Neither is the signature of a transient correction following an event. Both are what continuous conditioning looks like: a controller that is better because it knows where it is, not because something just happened.

This is the more useful of the two mechanisms for a machine of this class. The plant of a verge mower is not usually characterised by discrete events, and a controller that must wait for a detectable transient before improving would spend most of an episode no better than its baseline. What the evidence supports is a residual that is better throughout, and the strength of the lateral result follows from exactly that.

What remains open is narrower and is named here rather than in a footnote. A sham condition, in which the flip schedule fires but the load never moves, would separate the two mechanisms directly rather than by inference. It has not been run under this objective, and Section 7.6.6 records it as outstanding.

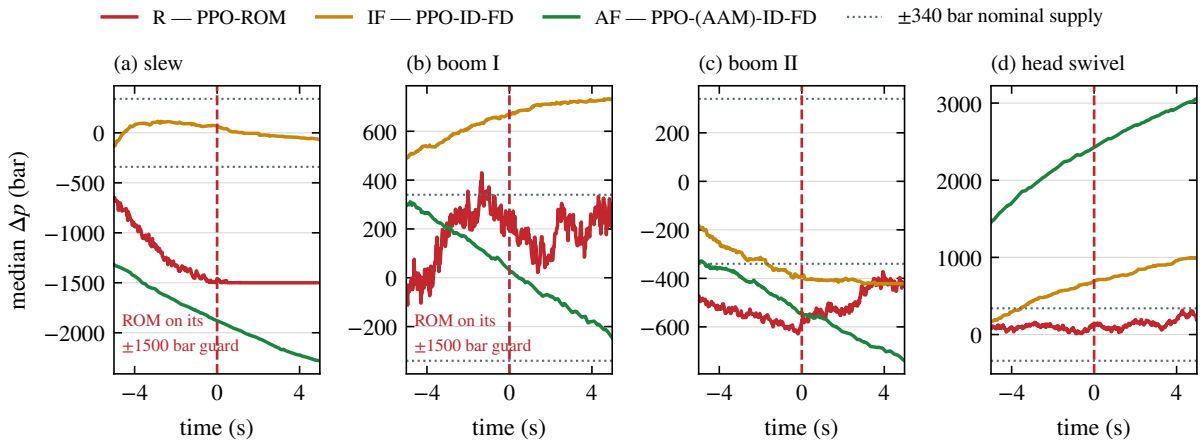


Figure 7.15: Median differential pressure per cylinder through the change of load, 151 environments aligned on their own flip instant. The dotted lines are ± the 340 bar nominal supply. The quantity is each stack's own actuator-model state, internal to the command ROM for R and the forward model's recursive state for the other two, so the panels show what each controller believes about the hydraulics rather than a common measurement, and each panel therefore carries its own scale. The slew and boom I panels mark where R sits on its 1500 bar numerical guard.

7.6.4 The Pressure Channel Across the Three Stacks

The latent carries mainly seal condition (Section 7.5.3), and seal condition acts through pressure, so the pressure the actuator model carries is where any conditioning should first be visible. Figure 7.15 shows it per cylinder for all three stacks, on the same plant and aligned on the same event.

Three properties of these traces bear on the rest of the chapter, and the first two are limitations of the signal rather than results.

The pressure state is not a calibrated pressure on any stack. Against the model's own nominal supply of 340 bar the median differential pressure sits well outside it in magnitude on most cylinders, in both directions, and the peaks reach several times it. On R the excursions are clipped by a numerical guard at 1500 bar, and that clip binds on the slew and the first boom cylinder and not on the other two, so those panels show a flattened trace where the guard is active. On the inverse and forward path no such clip is applied, so the two stacks are not bounded in the same way and their absolute levels are not directly comparable. What is comparable, and what the figure is for, is the *shape* of the response either side of the change.

Read that way the traces support the reading of Section 7.6.3. The pressure state moves at the change of load on every stack, so the information the residual would need is present in the channel it reads. The adaptive arm's advantage, however, does not appear as a transient following that movement: it is already present before it. The channel carries the operating condition continuously, and the residual uses it that way.

7.6.5 Sensitivity to the Training Budget

The reported arms are trained for 400 iterations, which was the protocol budget for every arm from the start. Resuming the same adaptive arm to 1200 iterations still passes both criteria, at +85.5 % cut height and +39.1 % lateral against the base, so the result is not an artefact of stopping at a fortunate point. Held-out performance is nonetheless lower at 1200 than at 400, which is ordinary overfitting to the training plants, and the shorter budget is what is reported because it is what was declared.

7.6.6 What This Experiment Does Not Yet Establish

Two limitations bound the claim, and both are matters of evidence not yet collected rather than of interpretation.

The result is demonstrated on one seed. Under the same objective a second seed trades the two errors against each other rather than improving both, at −7.4 % cut height and +24.9 % lateral, and three further seeds were in training when this was written. The pre-registered criterion is that the median of five seeds passes. Until that table exists the defensible claim is that the mechanism was diagnosed and repaired and the repair demonstrated on a seed, not that online adaptation improves both errors in general.

The falsification control is also outstanding. A sham condition, in which the flip schedule fires but the mass never changes, separates adaptation from a policy that has simply learned to track better. Under the earlier objective the sham showed +29.3 %, meaning a substantial part of the measured gain there was improved tracking rather than response to the plant. That control has not been repeated under the objective reported here, and until it is, the share of the gain attributable to adaptation specifically is bounded above rather than measured.

7.7 Integrated Discussion

RQ5: What the Actuator Representation Changes

How does the choice of actuator representation affect tracking, command behaviour, latency and robustness under plant variation?

Substantially, and not uniformly. On the matched pair, driving the plant through the frozen inverse and forward models rather than the command reduced-order model lowers mean lateral error from 0.1026 m to 0.0703 m and cut height from 8.4 to 6.3 mm (Section 7.2). The command behaviour behind that is the more informative result: R spends the *least* actuation and tracks worst, reversing direction 40 times a second against 10.6 for IF, so its effort is largely spent fighting itself (Section 7.3.5). Representation buys tracking by buying smoothness, not effort, and that also explains its tool-acceleration and impact figures.

Latency separates the representations in the hydraulic half of the chain but not in the task. A shorter path from command to cylinder does not produce a quicker response at the tool, because the response is set by the command rate limit and the policy's own dynamics rather than by the actuation chain (Section 7.3.3). This is the most transferable finding of the strand: a faster actuator path is not by itself a design goal on a machine of this class.

Robustness does not separate them, and both are robust in the strong sense. Every closed-loop figure here is measured on held-out meshes, eleven trajectory families scored individually, and plants drawn from sixteen reserved factor configurations disjoint from training. Underneath, the frozen inverse model was identified on Simscape alone and transfers to all 118 deployment runs with millimetre-scale one-step error, flat across domains (Section 5.2.2). Generalisation across simulator, plant draw, terrain and trajectory is established for both, without online adaptation.

Robustness *beyond* that population is a weaker claim than the sweep suggests. Tracking does not move across it, but on the inverse/forward plant the sweep varied no hydraulics at all, and on R the upper half is pinned to the training envelope and cannot remove more than 62 % of actuator authority (Section 7.4.1). No boundary was located within a bounded instrument; that is not the same as insensitivity, and locating it is left to Section 7.4.2.

RQ6: Whether Online Adaptation Pays

Where plant variation survives that robustness, is it identifiable from recent observations, and does online adaptation improve closed-loop performance?

It is identifiable, but only from data collected to make it so, and it pays where the correction has authority to act.

On identifiability, the same architecture, objective and selection criterion put the latent *behind* its no-latent baseline on task-controller data and ahead of it by a median 22.3 % on open-loop excitation data (Section 7.5.2). Nothing about the model changed; what changed was whether the data contained an answer. That check is cheap, needs no estimator, and belongs before the decision to build one.

On whether it pays, the first attempt said no, and the reason was delivery rather than estimation: the correction was added to a command already resting on its own clip, so it was clipped away before reaching the plant (Section 5.7.3). Removing the inverse model from the command path changes the answer. Against its own frozen base the adaptive arm improves cut height by 120.1 % and lateral by 78.2 %, every paired interval excluding zero (Section 7.6.2). The capacity-matched control divides the credit: capacity alone buys cut-height accuracy but leaves lateral error at baseline, so the lateral gain belongs to the history input.

The mechanism is continuous conditioning rather than event response. The advantage is present before the load changes and does not widen after it (Section 7.6.3). For a machine that settles over seconds this is the more useful of the two: a controller that must wait for a detectable transient would spend most of an episode no better than its baseline.

Relation to the Research Gap

Chapter 2 positioned this work against history-based adaptation methods that infer a plant code and condition a policy on it. Two conditions those arguments leave implicit are made measurable here: whether the data identifies the plant at all (Section 7.5.1), and whether the correction has authority to act (Section 7.4.1). Both are cheap and both precede estimator training.

A third follows from the results together. Rapid motor adaptation is demonstrated on legged robots, where the plant changes discontinuously at contact and the correction window is a fraction of a gait cycle [Lee 20, Kumar 21]; there feedback cannot substitute for identification. This machine is the opposite in its time constants, so conditioning buys little from the transient that follows a change. What it buys is the standing operating point. The criterion is therefore better stated in two parts: conditioning on a transient pays where plant variation acts faster than the loop can observe and correct it, and conditioning on the operating point pays wherever a fixed policy must serve a family of plants it cannot otherwise distinguish. The literature states the first and leaves the second implicit.

8. Conclusion and Future Work

This thesis asked how hydraulic actuator dynamics learned from high-fidelity simulation can be reduced, transferred into a computationally efficient robotic simulation and used for reinforcement-learning control, and under what conditions online adaptation adds anything to the robustness those learned models already provide. The six research questions of Section 1.4 are answered in the order they were asked, which is also the order in which the answers depend on one another.

8.1 Summary of Findings

The findings are stated below in the order of the research questions. The first four concern how hydraulic actuation is learned, transferred and controlled. The last two concern online adaptation.

The inverse and forward representation identified in Simscape transfers to the deployment simulation without retraining (RQ1, RQ2). The inverse model was trained on randomised Simscape campaigns and evaluated on all 118 IsaacLab deployment runs, drawn entirely outside that training distribution, where its teacher-forced one-step extension error is millimetre-scale and nearly flat across domains: 2.36 mm on the combined runs against 3.36 mm on nominal, with the held-out group no worse than the domains used in training. The transfer claim is made in that one-step regime, which is how the policy uses the model; fed its own predictions instead, the composite diverges in every run, for the saturation reason set out in Section 5.2.2. No deployment-specific retraining was required, and the inverse model stays frozen everywhere downstream. The reduced-order model is the exception and is treated as one throughout: it received a small bounded calibration against the deployment plant, so it is transferred with a correction rather than without one.

Supervised accuracy is not evidence of control authority (RQ1). The first forward model scored near-perfectly and had learned to ignore the valve command, reconstructing motion from the state history instead. Nothing in the validation loss showed it. Ablating the command exposed it, and redesigning the excitation repaired it. Rollout error fell from 75.2 mm to 12.4 mm at 0.5 s and from 157.9 mm to 27.3 mm at 1.0 s, and removing

the valve input from the repaired model then costs a factor of 1.81. Excitation design and input ablation are therefore reported here as required tests for a learned actuator intended for control. This finding concerns how such a model is validated rather than what it represents, so it applies beyond the machine studied here.

Two actuator representations both support closed-loop control, and they do not produce the same controller (RQ3, RQ5). The command-conditioned reduced-order model reaches $R^2 = 0.998$ on the internal valve state and 0.97 on extension, with a diagonally dominant extension–command Jacobian of condition number 2.14, and it runs inside GPU-parallel training. The inverse–forward chain composes two separately learned models in series, which could have compounded their errors and did not. The two representations differ in the control strategy they induce from the same policy interface. R spends the least command and reverses about 40 times per second, against 10.6 for IF, which converts greater actuator effort into lower lateral error. The choice of representation therefore affects the resulting controller and is not an implementation detail.

The standing of that comparison is stated here because it governs how the three stacks are read throughout. R and IF agree on every recorded training field and differ only in the actuator path, so the contrast between them is controlled and the difference above *is* attributable to the representation. AF is a separately trained run at a different seed, a lower actor learning rate and no frozen-actor warm-up (Section 6.9); it is reported as a deployed controller demonstrating the adaptive stack end to end, not as a ranked competitor, and no claim here rests on its position relative to the other two. Only one training run exists per stack, so seed variance is not estimated even for the matched pair. Claims about the adaptation path are carried instead by a paired within-stack comparison, in which three arms share the frozen base, the architecture, the budget, the reward and the seed, and differ only in whether the residual can see its history.

A learned policy can replace the classical controller at runtime (RQ4). The damped-least-squares controller supplies privileged demonstrations during training and is then removed; the distilled policy emits desired cylinder motion directly into the learned actuator and is the entire deployed controller. Over 33 evaluation cells IF holds a mean lateral error of 70.3 mm, a median of 28.1 mm and 95.29 % ground contact. Reward ascent alone does not deliver this result. Unconstrained fine-tuning raised total return while degrading the primary tracking objective by up to a factor of 12.9, with the critic remaining healthy throughout. The failure is therefore one of objective mismatch rather than of optimisation, and it is the reason the deployed policies are anchored to the demonstrated behaviour.

The robust stacks generalise without online adaptation, but no robustness boundary was located (RQ2, RQ5). Every closed-loop result is measured on held-out meshes, eleven trajectory families scored individually, and plants drawn from sixteen reserved factor configurations disjoint from training by construction. Under those conditions IF holds the lateral criterion in 32 of 33 cells at a median of 0.0281 m, and the frozen inverse model beneath it, identified on Simscape alone, transfers to all 118 deployment runs with millimetre-scale one-step error, flat across domains. This is the broadest claim the work supports and it requires no plant estimation.

Pushing beyond that population establishes less than it appears to. Doubling every deployment-factor deviation moves no stack's median, but the perturbation path cannot

deliver the stress it requests: the inverse/forward plant consumes no factor input, so there the sweep varied tool mass and centre-of-mass height alone, and on R the hydraulic factors are clamped back to their identified ranges above scale 1, with at most 62 % of actuator authority removable by that route (Section 7.4.1). What the closed loop absorbs is therefore established only for the perturbations this work can apply, and the point at which command authority becomes insufficient remains unmeasured.

The remaining findings concern adaptation itself (RQ6).

Under the controller, signal set and excitation regime examined here, plant identity is recoverable only from deliberate excitation, and the limit has a mechanism. Data gathered while this task controller drives the machine does not separate plants, because the command is computed from the state and the two are nearly redundant. This is a statement about the dataset the controller produces, not a claim that closed-loop identification is impossible in general: dedicated closed-loop identification methods exist and were not applied. Open-loop excitation repairs it here. Identity is then legible at low amplitude and gone by amplitude 0.35, and the reason is operating-point drift rather than actuator saturation: by that amplitude the plants have desynchronised in position by half a probe stroke, and a one-second window cannot separate a leaky cylinder from one sitting five centimetres lower. This gives a mechanism-level account of the excitation amplitude at which identification stops working on this plant, which among the excitation-design work reviewed in Chapter 2 is generally left implicit.

The estimator is not the limiting component (H1, H2, RQ6). A state predictor conditioned on an estimated latent lowers free-running prediction error by 24.7% against a matched no-latent baseline in all five seeds (Section 7.5.4), so the residual gap between training and deployment is recoverable without retraining the inverse model, which is what the frozen-inverse architecture requires. The same comparison on controller-driven data put the latent 1.8% behind, so the excitation result above is what makes this one possible. What the estimate carries is temporal rather than a plant label: substituting a per-episode constant costs 13.6% while substituting a constant that is wrong about the plant costs 0.43% (Section 7.5.3). Supplied to the frozen dynamics model the causal estimate is indistinguishable from one read off the true factors, ahead in 18 of 36 held-out runs, while both are clearly ahead of an uninformative latent (33 of 36) and a wrong one (29 of 36) (Section 7.5.5). One expectation it does not meet is that the benefit should grow with the size of the plant deviation: it does not, so the gain is uniform across the family rather than concentrated where the plant is hardest.

Online adaptation pays where the correction has authority to act. Placing a bounded correction on the inverse model's output produced no closed-loop effect, and the reason was delivery rather than estimation: that command already rested on its own clip for most of the scored window, so the correction was removed before it reached the plant. Rebuilding the stack so that the policy emits the valve command itself, with the residual acting on that command, changes the outcome. Under a mid-episode change of load the adaptive arm improves both scored errors against its own frozen base, by 120.1 % on cut height and 78.2 % on lateral, with every paired interval excluding zero (H3, Section 7.6.2).

A capacity-matched control arm, identical but for a zeroed history window, divides that result honestly. Added capacity alone already buys cut-height accuracy, so part of that gain is not adaptation. It does not move lateral error at all, leaving it within half a

millimetre of the baseline, while the adaptive arm improves it by more than three quarters. Lateral error moves only when the history is live, and that is what attributes the effect to online sensing rather than to parameters (H4). The claim is demonstrated on one seed and the pre-registered bar of five is not yet met, so it stands as a demonstration rather than a general result.

8.2 Design Implications

The results carry design consequences for each of the three parts of this work: how a hydraulic actuator is identified and transferred, how a policy is trained to act through it, and how plant adaptation is fitted around both. They are stated below in that order, because that is the order in which they constrain a successor design. Several were established by a measured failure rather than by a successful result, and are stronger for it.

Identification and Transfer

Design the excitation for identifiability, and accept the model on an ablation rather than on a score. A task controller computes its command from the state, so the two are nearly redundant and the valve is not causally identifiable from the resulting data however varied the control effort looks: the closed-loop batch spends a quarter of its time saturated and half at mid-authority, and is still uninformative. A forward model trained on it reached one-step $R^2 \approx 0.999$ while ignoring the valve entirely, and no validation loss computed on that distribution could have revealed it. Zeroing the command did, and open-loop excitation repaired it, at which point removing the valve cost a factor of 1.81. The implication is that a learned actuator intended for control needs a dedicated excitation campaign and an input-ablation acceptance test, and that predictive accuracy should never be the criterion on which such a model is promoted.

Embed the physics that is known and bound what is learned. The reduced-order model propagates pressure, force, velocity and extension through explicit updates with trainable effective parameters, and confines the neural terms to valve realisation, configuration load and residuals bounded to 10 % of the pressure and velocity step limits and 1 cm on position. That bound is what makes the model safe to place inside a training loop: even a badly fitted residual cannot produce an unphysical state jump. The structure also earns its keep at identification time, since encoding the orifice and continuity relations makes the pressure dependence of valve authority architectural rather than something to be rediscovered. Pinning a single global valve gain instead was tried and rejected, because measured authority varies by a factor of 15–75 across pressure quartiles and can change sign.

Prefer a frozen model with a bounded, gated correction to retraining. The inverse model transferred across the deployment plant family with a spread of only a factor of 1.09 between the best and worst domain, and an ordering that does not follow plant severity, so it was deployed frozen everywhere downstream. Where a deployment offset did have to be absorbed, it was absorbed by a correction capped at 3 mm and gated to vanish at zero command, with every hydraulic parameter frozen and the original data retained as replay. That construction preserves the identified physics, keeps a commanded hold a hold, and confines the deployment-specific part of the model to a component whose authority is bounded by construction.

Control Methodology

When the acceptance criterion is not the reward, bound the ascent and price what is gated. Unconstrained PPO from a distilled checkpoint raised total return while degrading the primary tracking objective by up to a factor of 12.9, with critic explained variance healthy throughout. That is the optimizer working correctly on the wrong functional: the reward is a proxy, acceptance is a vector of physical gates, and the proxy's optimum lies outside the feasible set while the distilled policy lies inside it. The remedy is to constrain the update rather than rewrite the objective. A forward-KL anchor to a frozen reference penalises the *abandonment* of reference behaviour, the exploration scale is held fixed so entropy collapse cannot shrink the divergence while the commanded action drifts, and the anchor advances only through checkpoints that have passed the gates.

The same discipline applies to the reward itself: a quantity that is gated but not priced is traded away. The only adaptive checkpoint to clear the ground-contact gate over the full path was the one whose reward priced cylinder headroom explicitly, and an inherited formulation of 53 weighted fields carried three terms working against the task for thirteen campaigns, undetected because logging reported an aggregate and omitted the two largest lateral terms. Shaping must also retain a far-field gradient; the Gaussian form previously used was a factor of 520 weaker at 0.5 m of error, where the failing trajectory families live.

Where no baseline is inherited, the action contract is the safety argument. The deployed policy is not a residual: a zero action leaves the arm stationary, and nothing is summed with the policy output at runtime. Authority is therefore bounded by the chain the action passes through rather than by proximity to a nominal controller, with a rate limit, a filter, a shield that fades authority as travel headroom is consumed, a command scale, and an anti-windup cap on how far the integrated target may lead the measured extension. That last cap is load-bearing and axis-dependent: relaxing it from 18 mm to 50 mm halved lateral error on the families needing a sustained standing offset, while the same relaxation on the head-pivot cylinder drove alignment from 0.9507 to 0.4636.

Plant Adaptation

Measure before you build, and route the latent to the consumer that rewards it. Two cheap measurements precede any estimator. The identifiability envelope establishes whether plant identity is present in the data the machine produces; the delivered-stress check establishes whether the perturbation reaches the plant and whether the correction has authority left to act. Either can end the effort before it is incurred, and on this task the second would have shown that a correction placed on the inverse model's output could not move a command already resting on its own limit, before the estimator, the adapter and the conditioned policy were built. That conversion of an open-ended engineering effort into a measurement is the design contribution this thesis most expects to transfer.

Where the estimate is used matters as much as whether it exists. Its value lies in variation over time rather than in the plant identity it carries, by a factor of about thirty-two, and the two consumers want opposite things from that variation: a one-step predictor gains 13.6 % from its movement, while a memoryless actor integrates nothing, so a moving latent changes the action tick to tick. Filtering on the policy path alone, at fixed content, improves cut-height tracking by 14.5 % on the median in 22 of 33 cells with an interior

optimum at $\tau = 0.796\,\mathrm{s}$. Two qualifications belong with that: the temporal variance of the latent as the actor reads it does not order with τ, so the proposed cause is not the measured one, and the optimum may belong to a policy trained on the unfiltered signal.

8.3 Contributions

Section 1.4 stated five contributions. Their standing at the end of the work is as follows.

Transferable hydraulic actuator learning is delivered. Both representations were identified from Simscape and realised in IsaacLab, and the inverse model transfers without deployment-specific retraining.

The **control-relevant identification and validation** procedure is delivered, and it is the contribution most likely to transfer independently of this machine. It covers both the input-ablation test that exposed a forward model with no command authority and the identifiability audit that established the excitation envelope, including the mechanism behind the upper amplitude limit.

The **reduced-order actuator model** is delivered and carries everything downstream, being accurate enough to serve as the deployment plant and cheap enough to evaluate inside GPU-parallel training, which the stiff implicit solver of the high-fidelity model it replaces cannot support. The qualification stated throughout stands. It was calibrated against IsaacLab and is therefore the reference condition rather than an independent competitor to the inverse and forward pair.

Policy-only control through learned hydraulics is delivered for both representations, with the classical controller absent from the deployed loop.

The measurements of **robustness and the limits of adaptation** are delivered, and their main product is the discipline that produced them rather than any single number. Two findings exist only because of it: that a selection window covering 8.9 of 26 m certifies a third of an episode, and that a quantity which is gated but not priced is traded away by the optimizer. The causal plant estimator and the bounded adaptation path behave as specified, with no privileged access, no future information, bounded output, and reversion to the robust actuator path on invalid input. They remain enablers of this contribution rather than claims in their own right.

The **eight-gate promotion criterion** is delivered as the non-reward criterion the training loop and the checkpoint selection both use, which is the role it was built for and the role every result here gives it. One property is recorded for reuse: five of the eight gates separate the matched pair and carry the comparative claims, while lateral, hard impacts and tool acceleration do not (Section 3.9.1). Re-deriving those three against the widened trajectory library is specification work rather than control work.

8.4 Limitations

The evidence is simulation-to-simulation, and the deployment plant is itself a learned model. No claim here is a claim about hardware.

The differential pressure the estimator consumes is produced by the actuator model rather than measured, so transferring the model-level result to hardware requires instrumentation

the FME 500 does not carry. An observation contract remains unresolved alongside this: every result uses the pressure-inclusive row, while the stricter deployable alternative excludes measured differential pressure. Since the estimator's advantage is concentrated in that channel, the deployable variant should be expected to perform worse, and by how much is not established.

The closed-loop comparisons resolve differences to a few per cent over 33 paired conditions and do not establish equivalence. The available plant variation bounds the disturbance rather than the architecture, and more tightly than the sweep's labels suggest: a stack shown to absorb a 40% reduction in valve effectiveness and a nominal doubling of every deployment factor has been shown to tolerate what actually reached it, which on the inverse/forward plant is a mechanical perturbation only. The three stacks are matched on evaluation but not on training budget, so the comparison describes three deployed controllers rather than three equally tuned training procedures.

Finally, the bound is a bound on this loop. A commanded step cannot complete faster than 417 ms at the command rate limit while the plant answers in 117 ms, so the task response is set downstream of anything a latent could change within a transient. That is close to the least favourable case for adaptation understood as *response to an event*, and it is consistent with the measurement: the adaptive arm's advantage does not widen when the load moves. It says nothing against adaptation understood as continuous conditioning on the operating point, which is the form that pays here. The finding generalises through the mechanism rather than through the number.

8.5 Future Work

The evidence in this thesis is simulation-to-simulation, and the step that would extend it furthest is the one it does not take: placing the learned actuator models and the distilled policy on the machine they were built for.

That step begins with instrumentation, because the FME 500 as delivered carries no cylinder-chamber pressure transducers, and the differential pressure the models propagate exists here only as an actuator-model state. Fitting pressure transducers, extension sensing and a rod load pin converts three modelled quantities into measured ones. The collection instrument is already designed and validated: the open-loop excitation schedule of Chapter 4 was built to make the valve causally identifiable, and the identifiability audit of Section 7.5.1 applies to machine data unchanged.

Fine-tuning on that data is then a well-posed problem rather than an open-ended one. The inverse model transferred across the deployment plant family with a spread of only a factor of 1.09, so the gap to a real machine is plausibly a bounded offset rather than a different function, and the mechanism for absorbing such an offset has already been validated once: a correction capped at 3 mm and gated to vanish at zero command, with every hydraulic parameter frozen and the original data retained as replay. The input-ablation test of Section 5.3.1 decides whether that suffices, since a correction obliged to carry command authority is no longer a correction. For the policy, the reference-anchored construction of Section 6.4 transfers with its guarantee intact, and that guarantee is worth more on a machine than in simulation: it holds the policy within a bounded divergence of a checkpoint verified against physical acceptance and restores it atomically when an update violates that bound.

A. System Parameters and Coordinate Mappings

The kinematic structure, the cylinder-to-joint mapping and the calibrated cutter Jacobian are derived in Section 3.2 and are not repeated here. This appendix records the quantities a reader needs in order to reproduce a run and cannot read off those derivations: the rate of every layer, the limits the command path enforces, and the geometry of the scored course. Every value below is read from the shipped configuration or from a stored evaluation record, and the source is named in each case.

A.1 Rates

Five different rates operate in this system and confusing them is the most common way to misread a result. The control step is 1/60 s, the physics substep is half that because the environment runs with a decimation of two, and every learned component operates on the 20 Hz grid at which the reduced-order model and the identification data were sampled.

Table A.1: The rate of every layer. Control and physics rates are read from the stored evaluation records (`control_dt_s`); the learned-model sample time is stored inside the frozen forward-model checkpoint (`sample_time_s`); the AAM update period is the runtime configuration `update_period_ticks`.

Layer	Rate	Period
PhysX physics substep	120 Hz	8.33 ms
Policy and control step	60 Hz	16.67 ms
Inverse / forward model	20 Hz	50 ms
AAM inference update	20 Hz	50 ms (every 3rd control step)
Simscape logging grid	20 Hz	50 ms

Two derived latencies follow from Table A.1 and are used in Section 7.3. The AAM consumes a history of 20 rows on the 50 ms grid, so its input window is exactly 1.0 s and no estimate at all is available for the first 1.0 s after a reset; until then the adapter is bypassed

and the frozen inverse action is returned unchanged. Its output smoother averages five consecutive predictions, which adds a further 250 ms of group delay.

A.2 Command-Path Limits

The command path enforces four limits, all of which bound the policy rather than advise it. They are listed in Table A.2.

Table A.2: Limits enforced on the command path, per cylinder where applicable. Valve command limits and the adapter residual bound are stored inside the frozen adapter checkpoint (`command_limits`, `residual_bound`); lead caps and the c3 reserve are read from the stored evaluation records.

Limit	c1	c2	c3	c4	Meaning
Valve command limit	0.95	0.95	0.80	0.80	accumulated command is clipped here
Adapter residual bound	0.15	0.15	0.15	0.15	maximum adapter correction
Target lead cap (m)	0.03	0.03	0.03	0.02	how far the target may lead the state
c3 extension reserve	—	0.008 m on c3			solver-enforced stroke reserve

The lead cap is the mechanism by which the reference governor prevents the commanded target from running away from the measured state, and the c3 reserve is enforced by the solver while headroom is still scored against the original stroke, so the gate is not satisfied by moving the goalposts. The adapter residual bound is what makes the adaptive path fail safe: an adapter output that is invalid, uninitialised or late is replaced by zero, which returns the stack to the frozen-inverse configuration rather than to an untested one.

A.3 Course and Episode Geometry

Every closed-loop result in Chapter 7 uses the geometry in Table A.3. The scored window is the full course: the count of control steps differs between evaluations only because the carrier speed does, and it is the travelled distance rather than the step count that defines the window.

Table A.3: Course geometry and the two evaluation speeds that appear in this thesis. Values are read from the stored evaluation records (`tracking_active_length_m`, `max_truck_travel_m`, `steps`).

Quantity	Chapter 7 protocol	Earlier 2 km/h records
Tracking line length	26.0 m	26.0 m
Carrier speed	4.0 km/h	2.0 km/h
Control steps scored	1403	2808
Truck travel achieved	25.98 m	26.00 m
Episode length	47.3 s	46.8 s
Trajectory families	11	8

The right-hand column is included because stored records under it exist in the project and are not comparable to the results of Chapter 7; Section 7.1.4 explains why one such record, which passes all eight gates, is not used.

B. Dataset Manifests and Factor Ranges

B.1 Trajectories

Each run receives its own reference trajectory, sampled rather than scripted. Between twelve and twenty waypoints are drawn by Latin hypercube sampling within the joint limits, and the arm moves between them without dwelling, so that the data contains transitions rather than long static holds. Segment durations are drawn from four speed classes, slow (1.5–2.5 s), medium (0.8–1.5 s), fast (0.3–0.7 s), and variable (0.5–2.5 s), and about a third of the runs receive additional exploratory perturbations on top of the waypoint path.

An earlier version of this generator used four to eight waypoints with dwell periods between them. It produced data in which the arm was stationary for a large fraction of the time, which flatters any model that learns to predict persistence. The current settings were chosen to remove that easy mass of near-static samples, and the increase in waypoint count is the single change that most improved the usefulness of the batch.

B.2 Execution and Logging

Each run simulates 22 s of operation. The solver is the stiff implicit `ode23t` integrator in accelerator mode with relative and absolute tolerances of 10^{-2} and 10^{-3}. The loose tolerances are a deliberate throughput compromise, and their effect is bounded by the consistency checks of Section 4.10. Output is written on a fixed 0.05 s grid, giving 441 logged samples per run, and the batch is checkpointed to disk every hundred runs so that a failure late in a multi-day batch does not cost the whole collection.

For every run the archive stores the time vector, the five valve commands, the four cylinder extensions, the four chamber pressure differences, the four rod forces, the five joint positions and velocities, the reference position and velocity signals, and, crucially, the complete parameter structure that produced the run. That last item is what makes the batch usable for the latent-adaptation work: the privileged plant descriptor $\boldsymbol{\xi}$ of Section 2.6 is not something that has to be reconstructed later, it is recorded alongside the trajectory.

The cost quantifies the argument made in Section 2.1.3. A single 64 s open-loop run takes roughly 2000 s of wall time on the collection machine, a real-time factor of about 0.03. The Simscape plant runs some thirty times slower than the machine it models. No policy-optimisation loop that needs millions of environment steps can be built on that, which is why the learned actuator surrogates exist.

B.3 Parameter Ranges and Dataset Audit

Table B.1: Randomized hydraulic parameters. The seal-wear coordinate is drawn independently for each of the four cylinders and drives three physically linked quantities at once.

Parameter	Nominal	Randomized range	Acts on
Entrained air fraction ε_{air}	0.005	0.001 – 0.080	fluid compliance
Dead-volume fraction ε_{vol}	0.020	0.005 – 0.080	fluid compliance
Discharge coefficient c_d	0.70	0.56 – 0.84	valve flow gain
Seal wear $w_{\text{seal},i}$	0	0 – 0.25 (per cylinder)	clearance, leakage, spool lag
Relief setting p_{set}	27.6 MPa	24.8 MPa – 30.4 MPa	pressure ceiling

Table B.2: Randomized mechanical parameters. The rigid-body perturbations are applied independently to each of the fourteen bodies.

Parameter	Nominal	Randomized range
Body mass	CAD value	±20 %
Centre-of-mass offset	CAD value	±5 %
Inertia tensor and products	CAD value	consistent with mass draw
Joint stiffness, J1 and J2	60 kN/m	40 – 120 kN/m
Joint stiffness, J3 and J4	60 kN/m	40 – 120 kN/m
Joint stiffness, J5	25 kN/m	15 – 60 kN/m
Joint damping	250	100 – 600

Table B.3: The six randomized physical factors of the IsaacLab deployment collections and their final ranges. One factor vector is drawn per run and held fixed for the whole run.

Coordinate	Range	Domain	Acts on
e_{flow}	$[-0.25, 0.25]$	hydraulic	valve flow gain
$e_{\text{compliance}}$	$[-0.25, 0.10]$	hydraulic	effective chamber compliance
e_{seal}	$[0, 1]$	hydraulic	seal wear, leakage, spool lag
e_{pressure}	$[-0.15, 0.15]$	hydraulic	supply/relief pressure level
e_{mass}	$[0.8, 1.5]$	mechanical	cutter mass scale
e_{COM_z}	$[-0.10, 0.05]$	mechanical	cutter centre-of-mass height

B.4 The Frozen Deployment Collections

Two collections are recorded in the deployment plant and both are frozen: the task-controller collection and the open-loop excitation collection of Section 4.7.5. They share

Table B.4: Audit of the strict open-loop excitation export. All checks are computed by the extraction stage and stored with the data; the export is accepted only if every one of them passes.

Check	Result	Interpretation
Rows, runs, columns	25 600 / 4 / 49	four complete runs
Median sample time	0.01 s	as commanded
Maximum sample-time error	9.1×10^{-15} s	numerically exact grid
Maximum Δextension error	0	targets match state differences
Maximum Δvelocity error	0	targets match state differences
Maximum $\Delta(\Delta p)$ error	0 Pa	targets match state differences
Non-finite values	0	no solver escapes
Valid fraction	0.937	startup, gaps, shutdown excluded
Command rank	4	all four channels independent
Lagged excitation rank	80	history matrix full rank
Valve saturation	8.5 % per channel	limits reached but not dominant
Stroke coverage	12 % – 33 %	above floor, below target

their factor design, domain composition and run-level split, and differ only in how the arm is driven. Each manifest records 118 runs, one factor vector per run, held fixed for the duration of that run. The model-level results in this thesis are fitted and scored on the open-loop collection, for the identifiability reason given in Section 7.5.2. The composition common to both is given in Table B.5.

Table B.5: Composition of the frozen deployment collections, counted directly from the manifest. The run-level split is 102 training runs against 16 held-out test runs; the training portion is further divided 82/20 into fitting and validation runs, so that no window from a validation or test run is ever seen during fitting.

Domain	Runs	Factors varied
`combined`	64	all six
`hydraulic`	40	flow, compliance, seal, pressure
`mechanical`	12	mass scale, centre-of-mass height
`nominal`	2	none (factors at nominal)
Total	118	
`train` split	102	further split 82 fit / 20 validation
`test` split	16	held out

The split is at *run* level rather than window level, and this is the single most important property of the collection. Within one run the factor vector is constant, so windows drawn from the same run are highly redundant; a window-level split would place near-duplicates of the same plant on both sides of the boundary and every held-out score in this thesis would be optimistic. The manifest stores the factor key of every run alongside its split label, so this property is checkable rather than asserted.

C. Model Architectures, Normalizers and Checkpoint Hashes

This appendix pins the frozen artifacts. Every model-level and closed-loop result in this thesis was produced with the exact files listed in Table C.1, and the SHA-256 digests are given so that a later reader can establish whether a file on disk is the one that produced a number here. This matters more than usual in this project, because resumed training runs overwrite checkpoints in place.

Table C.1: Frozen artifacts and their SHA-256 digests (first 32 hex digits shown). "Frozen" means the file's weights were not updated by any subsequent stage; the three policy checkpoints are the ones evaluated in Chapter 7.

Artifact	Role	SHA-256 (truncated)
`aam.pt`	AAM, frozen	`cd6a7f0b118b96c7ce66b889b9b3e422`
`zhat_adapter.pt`	ID adapter, frozen	`30629a4f83db5c2bed3c777bcf82dc57`
`best_id_cylinder_control.pt`	inverse model, frozen	`8e26c52e82d89efe193e001d0a8b07a6`
`best_fd_cylinder_strict.pt`	forward model, frozen	`1f41e44309a8f320430a3382ca88ce5f`
`headroom_priced_v1/model_299.pt`	AF policy	`aaf6b4bab7cfeac53fb95a3d15470cde`
`rom_s6_aligned/model_598.pt`	R policy	`ce7e3ab9649b1193f123f9f91107bbf8`
`idfd_s6_aligned/model_598.pt`	IF policy	`06533b8e8614b0ad982dd8c238a17772`

C.1 The Inverse and Forward Models

Both learned actuator models share one architecture: a linear input projection to 512 units, four pre-norm residual blocks of two 512-wide linear layers each, a final normalisation and a linear output projection. They differ only in their input and output layouts, given in Table C.2. Each carries just over 2.12 million parameters. Both are trained on the Simscape campaign, both operate on the 50 ms grid, and both are frozen for every result in Chapters 6 and 7.

Only the predicted extension increment of the forward model is integrated in the control loop. Its velocity output failed validation on two axes (Section 5.3) and is carried as a diagnostic rather than used.

Table C.2: Input and output layouts of the two frozen actuator models, read from the `input_layout` and `output_layout` fields stored in the checkpoints themselves.

	Inverse model	Forward model (strict)
Input width	28	20
Input layout	extension(4), velocity(4), $\Delta p(4)$, force(4), target Δextension(4), target Δvelocity(4), $\Delta t(4)$	valve(4), extension(4), velocity(4), $\Delta p(4)$, force(4)
Output width	4	12
Output layout	valve command(4)	Δextension(4), Δvelocity(4), $\Delta(\Delta p)(4)$
Target mode	normalised Δ valve command	normalised Δ cylinder state
Parameters	2 124 292	2 124 300

C.2 The Adaptation Module

This section describes the offline AAM, the encoder trained against a latent target in Chapter 5. The residual encoder deployed in AF is a different component, trained jointly with its head under the reinforcement objective and against no target (Section 6.7.5).

The AAM is a causal one-dimensional convolutional network over the last 20 rows of the 50 ms deployment observable. The input row is 18 columns wide,

$$h_t = \Big[\, q(5),\ \dot{q}(5),\ \Delta p(4),\ u_{t-1}(4) \,\Big],$$

and the trunk is four convolutional stages of width 64 and kernel three, the first carrying a 1×1 skip projection from 18 to 64 channels, each stage normalised. A two-layer head maps the final 64-channel activation to the 2-dimensional latent $\hat{z}$. The whole module is 42 562 parameters, which is what makes 20 Hz inference affordable alongside 256 parallel environments.

The latent dimension is 2 because the module regresses onto the frozen state predictor's latent, which is 2-dimensional by construction. This is the structural reason the deployed latent decodes only one of the six physical factors, and Section 7.5.3 treats it as a finding rather than a configuration choice.

The ID adapter is deliberately small: a two-layer network of width 64, mapping a 30-dimensional feature row to a 4-dimensional residual, 76 stored tensors' worth of normalisation statistics included. Its feature row is the inverse model's 28 inputs concatenated with the 2 latent components, which is the pressure-inclusive contract discussed in Section 7.1.3. Its output is bounded at 0.15 per cylinder (Table A.2) and scaled by a declared residual factor, 0.25 for every result reported here.

The adapter checkpoint additionally stores the digests of the inverse and forward models it was fitted against, so a latent adapter cannot silently be paired with a different frozen actuator than the one it was trained for.

D. PPO and IsaacLab Configuration

D.1 The Observation Vector

The policy observation is 66 columns for every stack. AF deploys this same actor frozen and unmodified; its adaptation residual reads a history window carried alongside the observation rather than appended to the actor's input (Section 6.7.3). The 66–68 row below records the two latent columns of the earlier latent-conditioned design, retained for reference. Table D.1 gives the layout by index. The final twelve columns are the model-error block, and they exist because the policy is a memoryless multilayer perceptron: it already observes the actuator's prediction and the current measured extension, but forming the error between them requires differencing across two steps, which a feedforward network cannot do. Without these columns the standing model error of a learned actuator is invisible to the policy no matter how long it trains.

Channel 46 caused a campaign-scale failure. It carries truck speed, and with a frozen observation normaliser inherited from a fixed-speed run its recorded standard deviation was zero. The normaliser divides by that standard deviation plus a small constant, which multiplied the channel by roughly one hundred everywhere except at the speed the statistics were collected at. The policy scored perfectly at that one speed and collapsed at every other. Freezing a normaliser is correct when resuming; freezing one whose statistics were gathered under a narrower distribution than the one about to be trained on is not.

D.2 The Action

The action is four-dimensional in every stack: a desired cylinder motion, not a valve command. What converts it to a valve command is the actuator under test, which is the only free variable in the three-way comparison. The limits enforced between the action and the plant are given in Table A.2.

D.3 The Reward

The ten reward terms and their shaping functions are given in Table 6.2 and are not repeated here. Two weights behind that table are the ones a reproduction is most likely to

Table D.1: Policy observation layout. Indices are zero-based and half-open. Columns 0–53 are the task block, 54–65 the model-error block, and 66–67 exist only for AF.

Index	Width	Contents
0–9	9	verge height profile ahead of the cutter
9	1	cut-height error
10	1	lateral error
11	1	cutter vertical velocity
12–16	4	measured cylinder extensions
16–20	4	measured cylinder velocities
20–24	4	cylinder headroom, low side (normalised)
24–28	4	cylinder headroom, high side (normalised)
28–32	4	previous applied action
32–36	4	current valve commands
36–40	4	actuator-predicted Δextension
40–44	4	commanded target lead
44	1	head-down alignment
45	1	head-down alignment rate
46	1	truck speed
47	1	scan valid fraction
48–50	2	world-XY components of the cutter's local $+Z$ axis
50	1	normalised cutter–terrain interaction force
51–54	3	path preview at 0.5, 1.0, 2.0 m ahead
54–58	4	cylinder tracking residual (measured − commanded)
58–62	4	actuator prediction residual
62–66	4	normalised cylinder effort
66–68	2	$\mathbf{z}_{\mathrm{AAM}}$, earlier latent-conditioned design only

get wrong: contact loss is weighted at 20.0 and hard impacts at 20.0, against a support reward of 4.0. The hard-impact weight was introduced after the measurement described in Section 3.9.1: the quantity was gated but not priced, and the optimizer traded it away while its return rose.

D.4 PPO Hyperparameters

Table D.2 lists the optimisation settings of the three evaluated runs, read from their stored run manifests rather than from the scripts that launched them. The manifests share a schema, which is what makes the comparison in Section 7.1 checkable; the fields on which the runs differ are exactly those discussed there.

The dashes are informative and are left visible rather than filled in from the launch scripts. The two reference runs' manifests do not record their truck speed schedule, discount or GAE settings, so those cannot be verified from the frozen record; the adaptive run's manifest does record them. Section 7.1 states the consequence.

Table D.2: PPO configuration of the three evaluated runs, from `run_manifest.json`. A dash marks a field the manifest does not record. Differences between the adaptive run and the matched pair are discussed in Section 7.1.

	R	IF	AF
manifest schema		`terrain_shared_v2`	
seed	17	17	41
terrain base seed	17	17	41
environments	256	256	256
curriculum stage	6	6	6
actor learning rate	3×10^{-4}	3×10^{-4}	3×10^{-6}
entropy coefficient	0.001	0.001	0.001
frozen-actor warm-up	150	150	0
observation normaliser	freeze	freeze	freeze
observation width	66	66	66
discount γ	—	—	0.97
GAE λ	—	—	0.95
steps per environment	—	—	32
episode length	—	—	47.3 s
truck speed	—	—	2–7 km/h, per episode
residual scale	—	—	0.25
trajectory families		all 11	
actor parameters	—	—	59 080
critic parameters	—	—	58 881

E. Supplementary Results

E.1 Per-Family Results on the Reference Mesh

Table E.1 gives the per-family breakdown behind the pooled means of Table 7.3, on mesh 1337, for the adaptive stack. It is included because the pooled statistics of Chapter 7 are carried by a small number of cells, and the reader should be able to see which.

Table E.1: AF per-family results on mesh 1337, full 26 m path at 4 km/h, checkpoint `headroom_priced_v1/299`. Families are ordered by lateral error. Nine of eleven sit near 0.03 m; the distribution is not approximately uniform and its mean is not a typical value.

Family	lateral (m)	cut height (m)	ground contact
`s_curve`	0.0288	0.0014	0.9768
`lateral_sine`	0.0293	0.0012	0.9754
`straight`	0.0309	0.0015	0.9694
`lane_change_hold`	0.0312	0.0026	0.9629
`gaussian_slalom`	0.0313	0.0015	0.9701
`random_spline`	0.0315	0.0020	0.9622
`boom_sweep`	0.0318	0.0016	0.9729
`fourier_meander`	0.0324	0.0013	0.9786
`lateral_chirp`	0.0342	0.0032	0.9615
`double_s_curve`	0.0951	0.0021	0.9772
`offset_return`	**0.2240**	**0.0532**	**0.8485**

Two features of Table E.1 govern how Chapter 7 should be read. The first is the gap between the ninth and eleventh rows: a factor of seven in lateral error separates the typical family from the worst one, so the worst-family aggregation used by the acceptance gates is a different measurement from the mean, not a conservative version of it. The second is that `offset_return` is the family that fails, and it fails on all three metrics at once. Lateral error, cut height and ground contact degrade together, which is the signature of the cutter leaving the ground rather than of a tracking error alone.

E.2 The Latent Substitution Arms

The latent conditions recorded here share one checkpoint and one set of weights, and differ only in what occupies the latent slot. They were scored on the earlier adaptive architecture, in which the residual corrected the inverse model's output, and are retained as a record of that measurement rather than as a result about the deployed stack (Section 7.1.3).

Table E.2: The four latent arms, as evaluated. All four use checkpoint `headroom_priced_v1/299` with identical weights; only the value supplied in the latent slot changes.

Arm	Latent supplied
`estimated`	the deployed AAM estimate, updated at 20 Hz
`true`	privileged encoding of the run's actual factor vector
`mean`	a constant carrying no plant information
`shuffled`	a constant belonging to a *different* plant

The ordering of these four arms is the mechanism result of this thesis. The `shuffled` arm is the control that distinguishes the two candidate explanations: if the deployed estimate were losing because it carries the wrong plant identity, then `shuffled`, which is by definition wrong about the plant, would lose by at least as much. It does not. What separates the estimate from both constants is not its information content but its movement.

E.3 Reproduction

Every closed-loop number in Chapter 7 is produced by the experiment scripts stored with the project under `experiments/E1` through `experiments/E5`, each of which records the checkpoint, the mesh set, the family set, the speed and the scored horizon in its own manifest. The model-level results come from the frozen collection of Appendix B.4. The artifacts named in Table C.1 are sufficient, together with those scripts, to regenerate every table in this thesis.

Bibliography

[Åström 95] K. J. Åström, B. Wittenmark, *Adaptive Control*, 2nd ed., Reading, MA: Addison-Wesley, 1995.

[Ba 16] J. L. Ba, J. R. Kiros, G. E. Hinton, "Layer Normalization", arXiv preprint arXiv:1607.06450, 2016.

[Bengio 15] S. Bengio, O. Vinyals, N. Jaitly, N. Shazeer, "Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28. 2015, pp. 1171–1179.

[Benner 15] P. Benner, S. Gugercin, K. Willcox, "A Survey of Projection-Based Model Reduction Methods for Parametric Dynamical Systems", *SIAM Review*, vol. 57, no. 4, pp. 483–531, 2015.

[Cho 14] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, Y. Bengio, "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation", in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2014, pp. 1724–1734.

[Chua 18] K. Chua, R. Calandra, R. McAllister, S. Levine, "Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31. 2018.

[Egli 20] P. Egli, M. Hutter, "Towards RL-Based Hydraulic Excavator Automation", in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2020, pp. 2692–2697.

[Egli 22] P. Egli, M. Hutter, "A General Approach for the Automation of Hydraulic Excavator Arms Using Reinforcement Learning", *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 5679–5686, 2022.

[Elfwing 18] S. Elfwing, E. Uchibe, K. Doya, "Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning", *Neural Networks*, vol. 107, pp. 3–11, 2018.

[He 16] K. He, X. Zhang, S. Ren, J. Sun, "Deep Residual Learning for Image Recognition", in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778.

[Hochreiter 97] S. Hochreiter, J. Schmidhuber, "Long Short-Term Memory", *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[Hu 26] P. Hu, T. Wen, D. Zhang, H. Chen, J. Gong, "Bayesian reinforcement learning for adaptive control of energy recuperation in hydraulic excavator arms", *Scientific Reports*, vol. 16, p. 6195, 2026.

[Huber 64] P. J. Huber, "Robust Estimation of a Location Parameter", *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964.

[Hwangbo 19] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, M. Hutter, "Learning Agile and Dynamic Motor Skills for Legged Robots", *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.

[Isermann 11] R. Isermann, M. Münchhof, *Identification of Dynamic Systems: An Introduction with Applications*, Berlin, Heidelberg: Springer, 2011.

[Jelali 03] M. Jelali, A. Kroll, *Hydraulic Servo-systems: Modelling, Identification and Control*, London: Springer, 2003.

[Ji 22] G. Ji, J. Mun, H. Kim, J. Hwangbo, "Concurrent Training of a Control Policy and a State Estimator for Dynamic and Robust Legged Locomotion", *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 4630–4637, 2022.

[Johannink 19] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, S. Levine, "Residual Reinforcement Learning for Robot Control", in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2019, pp. 6023–6029.

[Kakade 02] S. Kakade, J. Langford, "Approximately Optimal Approximate Reinforcement Learning", in *Proceedings of the 19th International Conference on Machine Learning (ICML)*. 2002, pp. 267–274.

[Karniadakis 21] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, L. Yang, "Physics-Informed Machine Learning", *Nature Reviews Physics*, vol. 3, no. 6, pp. 422–440, 2021.

[Kingma 15] D. P. Kingma, J. Ba, "Adam: A Method for Stochastic Optimization", in *International Conference on Learning Representations (ICLR)*. 2015.

[Kumar 21] A. Kumar, Z. Fu, D. Pathak, J. Malik, "RMA: Rapid Motor Adaptation for Legged Robots", in *Proceedings of Robotics: Science and Systems (RSS)*. 2021.

[Lee 20] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, M. Hutter, "Learning Quadrupedal Locomotion over Challenging Terrain", *Science Robotics*, vol. 5, no. 47, p. eabc5986, 2020.

[Liu 25] X. Liu, C. Lu, H. Wu, B. Hu, X. Li, Z. Li, X. Guo, "FRMA: Four-Phase Rapid Motor Adaptation Framework", *Machines*, vol. 13, no. 10, p. 885, 2025.

[Ljung 99] L. Ljung, *System Identification: Theory for the User*, 2nd ed., Upper Saddle River, NJ: Prentice Hall, 1999.

[Lutter 19] M. Lutter, C. Ritter, J. Peters, "Deep Lagrangian Networks: Using Physics as Model Prior for Deep Learning", in *International Conference on Learning Representations (ICLR)*. 2019.

[Maitan 26] L. Maitan, L. Toschi, C. Zanette, E. G. Vergamini, L. F. Santos, T. Boaventura, "Contraction Metric Based Safe Reinforcement Learning Force Control for a Hydraulic Actuator with Real-World Training", arXiv preprint arXiv:2602.08977, 2026.

[Makoviychuk 21] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, G. State, "Isaac Gym: High Performance GPU-Based Physics Simulation for Robot Learning", arXiv preprint arXiv:2108.10470, 2021.

[Mattila 17] J. Mattila, J. Koivumäki, D. G. Caldwell, C. Semini, "A Survey on Control of Hydraulic Robotic Manipulators With Projection to Future Trends", *IEEE/ASME Transactions on Mechatronics*, vol. 22, no. 2, pp. 669–680, 2017.

[McKay 79] M. D. McKay, R. J. Beckman, W. J. Conover, "A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code", *Technometrics*, vol. 21, no. 2, pp. 239–245, 1979.

[Merritt 67] H. E. Merritt, *Hydraulic Control Systems*, New York: John Wiley & Sons, 1967.

[Mittal 23] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlekar, B. Babich, G. State, M. Hutter, A. Garg, "Orbit: A Unified Simulation Framework for Interactive Robot Learning Environments", *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3740–3747, 2023.

[MULAG Fahrzeugwerk 26] MULAG Fahrzeugwerk, "FME 500 Front Mower — Product Documentation", Heinz Wössner GmbH u. Co. KG, Oppenau, Germany. `https://www.mulag.de/en/roadside-equipment/products/unimog-equipment/front-mower/fme-500/`, 2026. Accessed July 2026.

[Nagabandi 18] A. Nagabandi, G. Kahn, R. S. Fearing, S. Levine, "Neural Network Dynamics for Model-Based Deep Reinforcement Learning with Model-Free Fine-Tuning", in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2018, pp. 7559–7566.

[Nakamura 86] Y. Nakamura, H. Hanafusa, "Inverse Kinematic Solutions With Singularity Robustness for Robot Manipulator Control", *ASME Journal of Dynamic Systems, Measurement, and Control*, vol. 108, no. 3, pp. 163–171, 1986.

[Nguyen-Tuong 11] D. Nguyen-Tuong, J. Peters, "Model Learning for Robot Control: A Survey", *Cognitive Processing*, vol. 12, no. 4, pp. 319–340, 2011.

[Paszke 19] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32. 2019, pp. 8024–8035.

[Peng 18] X. B. Peng, M. Andrychowicz, W. Zaremba, P. Abbeel, "Sim-to-Real Transfer of Robotic Control with Dynamics Randomization", in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2018, pp. 3803–3810.

[Rajeswaran 18] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, S. Levine, "Learning Complex Dexterous Manipulation with Deep Reinforcement Learning and Demonstrations", in *Proceedings of Robotics: Science and Systems (RSS)*. 2018.

[Ross 11] S. Ross, G. J. Gordon, J. A. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning", in *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*. 2011, pp. 627–635.

[Rudin 22] N. Rudin, D. Hoeller, P. Reist, M. Hutter, "Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning", in *Proceedings of the Conference on Robot Learning (CoRL)*. 2022, pp. 91–100.

[Schoukens 19] J. Schoukens, L. Ljung, "Nonlinear System Identification: A User-Oriented Road Map", *IEEE Control Systems Magazine*, vol. 39, no. 6, pp. 28–99, 2019.

[Schulman 15] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, P. Abbeel, "Trust Region Policy Optimization", in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 37. 2015, pp. 1889–1897.

[Schulman 16] J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation", in *International Conference on Learning Representations (ICLR)*. 2016.

[Schulman 17] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, "Proximal Policy Optimization Algorithms", arXiv preprint arXiv:1707.06347, 2017.

[Siciliano 09] B. Siciliano, L. Sciavicco, L. Villani, G. Oriolo, *Robotics: Modelling, Planning and Control*, London: Springer, 2009.

[Silver 18] T. Silver, K. Allen, J. Tenenbaum, L. Kaelbling, "Residual Policy Learning", arXiv preprint arXiv:1812.06298, 2018.

[Slotine 91] J.-J. E. Slotine, W. Li, *Applied Nonlinear Control*, Englewood Cliffs, NJ: Prentice Hall, 1991.

[Sobol' 67] I. M. Sobol', "On the Distribution of Points in a Cube and the Approximate Evaluation of Integrals", *USSR Computational Mathematics and Mathematical Physics*, vol. 7, no. 4, pp. 86–112, 1967.

[Sofrony 24] J. Sofrony, M. C. Turner, C. M. Richards, "Anti-windup compensation for model reference adaptive control schemes", *International Journal of Robust and Nonlinear Control*, vol. 34, no. 15, pp. 10176–10193, 2024.

[Spinelli 24] F. A. Spinelli, P. Egli, J. Nubert, F. Nan, T. Bleumer, P. Goegler, S. Brockes, F. Hofmann, M. Hutter, "Reinforcement Learning Control for Autonomous Hydraulic Material Handling Machines with Underactuated Tools", in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2024. arXiv:2410.05093.

[Srivastava 14] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting", *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[Sutton 18] R. S. Sutton, A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., Cambridge, MA: MIT Press, 2018.

[The MathWorks, Inc. 24a] The MathWorks, Inc , "Simscape Fluids: User's Guide", Natick, MA, USA, 2024. `https://www.mathworks.com/products/simscape-fluids.html`.

[The MathWorks, Inc. 24b] The MathWorks, Inc , "Simscape Multibody: User's Guide", Natick, MA, USA, 2024. `https://www.mathworks.com/products/simscape-multibody.html`.

[Tobin 17] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, P. Abbeel, "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World", in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2017, pp. 23–30.

[von Szadkowski 20] K. von Szadkowski, S. Reichel, "Phobos: A Tool for Creating Complex Robot Models", *Journal of Open Source Software*, vol. 5, no. 45, p. 1326, 2020.

[Wampler 86] C. W. Wampler, "Manipulator Inverse Kinematic Solutions Based on Vector Formulations and Damped Least-Squares Methods", *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 16, no. 1, pp. 93–101, 1986.

[Yamamori 26] S. Yamamori, K. Ishihara, K. Minamikawa, R. Ohmori, T. Yasaki, N. Sugimoto, J. Morimoto, "Actuator Reality Shaping for Zero-Shot Sim-to-Real Robot Learning", arXiv preprint arXiv:2607.02205, 2026.

[Yang 26] N. Yang, C. Ji, J. Chen, H. Zheng, "State-Constrained Control for Hydraulic Manipulator Position Servo Systems with Valve Dead-Band Compensation", *Actuators*, vol. 15, no. 4, p. 196, 2026.

[Yin 25] P. Yin, T. Westenbroek, S. Bagaria, K. Huang, C.-A. Cheng, A. Kolobov, A. Gupta, "Rapidly Adapting Policies to the Real World via Simulation-Guided Fine-Tuning", arXiv preprint arXiv:2502.02705, 2025.

[Yu 26] S.-Y. Yu, J.-W. Wang, M. Horii, M. Tomizuka, V. Garg, "Learning Context-Aware Neural ODE Dynamics for Adaptive Robotic Control", arXiv preprint arXiv:2606.15469, 2026.